Facultad de Ingeniería

Curso Integrador I: Sistemas - Software

Avance de Proyecto Final 3 (APF3)

Sistema Web de Ventas por Kilo, Créditos y Cuadre Diario para Canomar

– negocio de productos hidrobiológicos –

Integrantes:

Coronel Lopez, Franco Raul — Ingeniería de Sistemas e Informática

Doria Bautista, Imanol Jeffrey — Ingeniería de Software

Ciclo: 5.° · Periodo 2026-II · Sección 39120

Docente: Mg. Milagros Elisa Leonardo Ramos

Lima, 30 de octubre de 2026

Tabla de participación

En cada celda se indica el porcentaje de aporte en clase / fuera de clase. Cada fila suma 100 % entre los integrantes. El equipo está formado por dos integrantes, por eso las columnas 3 y 4 no se usan. El integrante 1 no participó en la elaboración del APF3: no asistió a las sesiones de trabajo ni registra commits en el repositorio (sección 3.3), por lo que todo el avance fue desarrollado por el integrante 2.

Punto trabajado	Integrante 1 Coronel Lopez, F.	Integrante 2 Doria Bautista, I.	Integrante 3	Integrante 4
1. Diseño: arquitectura (1.1 a 1.6)	0 / 0	100 / 100	—	—
1. Diseño: Propuesta 2 (1.7)	0 / 0	100 / 100	—	—
1. Diseño: Propuesta 3 (1.8)	0 / 0	100 / 100	—	—
2. Pruebas TDD	0 / 0	100 / 100	—	—
3. Control de versiones	0 / 0	100 / 100	—	—
4. Verificación de interfaces	0 / 0	100 / 100	—	—
5. Modelo de datos final	0 / 0	100 / 100	—	—
5. Construcción del producto	0 / 0	100 / 100	—	—
6. Preparación de la sustentación	0 / 0	100 / 100	—	—
Participación total	0 / 0	100 / 100	—	—
Firmas			—	—

PENDIENTE DEL EQUIPO: firmar la tabla.

Registro de actividades

- **En clase:** revisión de la rúbrica y de la guía del APF3 en las sesiones de las semanas 9, 10 y 11.
- **Fuera de clase:** construcción del sistema, pruebas TDD (149 pruebas automatizadas), verificación UX/UI de las 20 pantallas, modelo de datos final, propuestas con hardware y preparación de la sustentación.

Control de cambios

Observaciones recibidas en el APF2 y las que el equipo detectó al construir el sistema, con el lugar donde se corrigieron.

Observación recibida en el APF2	Cómo se corrigió	Dónde está (sección o anexo)
El prototipo no se presentó en clase (criterio de interfaces sin puntaje).	Las 20 pantallas ya están construidas y conectadas a la base de datos; se verificaron con las 4 capas y se muestran con capturas reales.	Capítulo 4 y Anexo C
No se realizó la sustentación oral del APF2.	Se preparó la distribución de la exposición, la demostración en vivo y las preguntas probables por integrante.	Capítulo 6
Los CUS tenían 2 eventos alternativos por RF.	Cada CUS tiene ahora 1 evento exitoso y 3 alternativos, ajustados al comportamiento real del sistema.	Sección 2.2 y Anexo A
El modelo físico del APF2 guardaba datos calculados (compra.total, venta.total, detalle_venta.importe, cuadre.diferencia) que la normalización ya había eliminado.	Se retiraron del modelo final y se obtienen con consultas y con las vistas SQL v_venta_total y v_saldo_cliente.	Secciones 5.2.2 y 5.2.3, Anexo E
Los mensajes del prototipo no coincidían con todos los del código.	Las fichas de prueba usan el mensaje exacto que muestra el sistema; cada pantalla lleva su rótulo P## y sus RF.	Sección 2.3, Anexo B, hallazgo H03
La secuencia del RF principal no estaba ligada al código.	Cada objeto de la secuencia de RF18 se relaciona con su clase y carpeta del repositorio.	Sección 1.2

Índice de contenido

CAPÍTULO 1. DISEÑO DE LA SOLUCIÓN....14
Propuesta 1: arquitectura del producto....14
1.1. Arquitectura....14
1.2. Del diagrama de secuencias al código....15
1.3. Patrón DAO....16
1.4. Principios SOLID....17
1.5. Preparado para TDD....17
1.6. Seguridad....18
Propuestas 2 y 3: evolución con hardware....18
CAPÍTULO 2. PRUEBAS TDD Y LIBRERÍAS....19
2.1. Librerías....19
2.2. Momento 1: escenarios....21
2.3. Momento 2: ficha de casos de prueba....23
2.4. Momento 3: clasificación de aptitud....24
2.5. Evidencia de tests automatizados....26
RF01 Registrar usuario · RF01RegistrarUsuarioTest....27
RF02 Iniciar sesión · RF02IniciarSesionTest....28
RF04 Cambiar contraseña · RF04CambiarContrasenaTest....30
RF05 Desactivar usuario · RF05DesactivarUsuarioTest....31
RF06 Registrar producto · RF06RegistrarProductoTest....32
RF07 Editar producto · RF07EditarProductoTest....34
RF08 Registrar cliente · RF08RegistrarClienteTest....35
RF09 Editar cliente · RF09EditarClienteTest....37
RF10 Registrar proveedor · RF10RegistrarProveedorTest....38
RF11 Registrar compra del día · RF11RegistrarCompraDelDiaTest....39
RF12 Calcular costo promedio por kilo · RF12CalcularCostoPromedioPorKiloTest....41
RF13 Registrar pago a proveedor · RF13RegistrarPagoAProveedorTest....42
RF15 Registrar precios de venta del día · RF15RegistrarPreciosDeVentaDelDiaTest....44
RF17 Registrar venta al contado · RF17RegistrarVentaAlContadoTest....45

RF18 Registrar venta al crédito · RF18RegistrarVentaAlCreditoTest.....46
RF19 Calcular importe por peso · RF19CalcularImportePorPesoTest.....48
RF20 Anular venta · RF20AnularVentaTest.....49
RF21 Registrar venta de remate · RF21RegistrarVentaDeRemateTest.....51
RF22 Registrar autoconsumo · RF22RegistrarAutoconsumoTest.....52
RF25 Sincronizar ventas pendientes · RF25SincronizarVentasPendientesTest 53
RF27 Registrar saldo para el día siguiente · RF27RegistrarSaldoParaElDiaSiguienteTest.....55
RF28 Consultar saldo por antigüedad · RF28ConsultarSaldoPorAntiguedadTest 56
RF29 Alertar mercadería antigua · RF29AlertarMercaderiaAntiguaTest.....57
RF30 Registrar ajuste de saldo · RF30RegistrarAjusteDeSaldoTest.....58
RF31 Registrar abono de cliente · RF31RegistrarAbonoDeClienteTest.....60
RF33 Calcular antigüedad de la deuda · RF33CalcularAntiguedadDeLaDeudaTest.....61
RF34 Alertar deudas vencidas · RF34AlertarDeudasVencidasTest.....62
RF35 Registrar compromiso de pago · RF35RegistrarCompromisoDePagoTest 64
RF36 Generar estado de cuenta · RF36GenerarEstadoDeCuentaTest.....65
RF38 Registrar gasto menor · RF38RegistrarGastoMenorTest.....66
RF39 Realizar cuadre de caja · RF39RealizarCuadreDeCajaTest.....68
RF40 Calcular ganancia del día · RF40CalcularGananciaDelDiaTest.....69
RF42 Registrar factura emitida · RF42RegistrarFacturaEmitidaTest.....70
RF43 Asociar ventas a factura · RF43AsociarVentasAFacturaTest.....72
RF45 Generar cierre mensual · RF45GenerarCierreMensualTest.....73
RF47 Consultar ganancia por producto · RF47ConsultarGananciaPorProductoTest.....75
RF48 Exportar reportes a Excel · RF48ExportarReportesAExcelTest.....76
RF49 Generar respaldo de la información · RF49GenerarRespaldoDeLaInformacionTest.....77
2.6. Evidencia de pruebas manuales.....78
2.7. Resumen de resultados.....104
CAPÍTULO 3. CONTROL DE VERSIONES.....105

3.1. Repositorio y ramas.....105
3.2. Entregables frente al repositorio.....105
3.3. Contribuidores.....106
3.4. Archivos excluidos.....108
CAPÍTULO 4. VERIFICACIÓN DE INTERFACES.....109
4.1. Capa 1: trazabilidad vista / RF.....109
4.2. Matriz de hallazgos.....110
4.3. Levantamiento de hallazgos.....111
4.4. Capa 4: navegación por rol.....113
4.5. Cobertura.....114
CAPÍTULO 5. PRODUCTO CONSTRUIDO Y MODELO DE DATOS FINAL.....115
5.1. Completo.....115
5.2. Coherente: documentación y código alineados.....115
5.2.1. Normalización de las 5 vistas físicas.....115
Vista 1: Nueva compra (P08 · RF11, RF12).....116
Vista 2: Vender (P10 · RF17, RF18, RF19).....117
Vista 3: Cobranza (P14 · RF31, RF35).....119
Vista 4: Gastos y cuadre (P16 · RF38, RF39).....120
Vista 5: Facturas (P18 · RF42, RF43).....122
5.2.2. Comparativo pantallas frente a base de datos.....123
5.2.3. Modelo de datos final.....125
5.2.4. Trazabilidad de punta a punta.....139
5.3. Buenas prácticas.....139
5.4. Autoría.....140
CAPÍTULO 6. SUSTENTACIÓN.....142
REFERENCIAS.....143
ANEXOS.....144

Índice de figuras

Figura 1. *Arquitectura en capas de SVC-Canomar y recorrido de una petición (RF18 Registrar venta al crédito)*.....14

Figura 2. *Archivo de dependencias pom.xml del repositorio*.....20

Figura 3. *Ejecución de RF01RegistrarUsuarioTest en rojo y en verde*.....28

Figura 4. *Ejecución de RF02IniciarSesionTest en rojo y en verde*.....30

Figura 5. *Ejecución de RF04CambiarContrasenaTest en rojo y en verde*.....31

Figura 6. *Ejecución de RF05DesactivarUsuarioTest en rojo y en verde*.....32

Figura 7. *Ejecución de RF06RegistrarProductoTest en rojo y en verde*.....34

Figura 8. *Ejecución de RF07EditarProductoTest en rojo y en verde*.....35

Figura 9. *Ejecución de RF08RegistrarClienteTest en rojo y en verde*.....36

Figura 10. *Ejecución de RF09EditarClienteTest en rojo y en verde*.....38

Figura 11. *Ejecución de RF10RegistrarProveedorTest en rojo y en verde*.....39

Figura 12. *Ejecución de RF11RegistrarCompraDelDiaTest en rojo y en verde*.....41

Figura 13. *Ejecución de RF12CalcularCostoPromedioPorKiloTest en rojo y en verde*.....42

Figura 14. *Ejecución de RF13RegistrarPagoAProveedorTest en rojo y en verde*.....43

Figura 15. *Ejecución de RF15RegistrarPreciosDeVentaDelDiaTest en rojo y en verde*.....45

Figura 16. *Ejecución de RF17RegistrarVentaAlContadoTest en rojo y en verde*.....46

Figura 17. *Ejecución de RF18RegistrarVentaAlCreditoTest en rojo y en verde*.....48

Figura 18. *Ejecución de RF19CalcularImportePorPesoTest en rojo y en verde*.....49

Figura 19. *Ejecución de RF20AnularVentaTest en rojo y en verde*.....50

Figura 20. *Ejecución de RF21RegistrarVentaDeRemateTest en rojo y en verde*.....52

Figura 21. *Ejecución de RF22RegistrarAutoconsumoTest en rojo y en verde*.....53

Figura 22. *Ejecución de RF25SincronizarVentasPendientesTest en rojo y en verde*.....55

Figura 23. *Ejecución de RF27RegistrarSaldoParaElDiaSiguienteTest en rojo y en verde*.....56

Figura 24. *Ejecución de RF28ConsultarSaldoPorAntiguedadTest en rojo y en verde*.....57

Figura 25. *Ejecución de RF29AlertarMercaderiaAntiguaTest en rojo y en verde*.....58

Figura 26. *Ejecución de RF30RegistrarAjusteDeSaldoTest en rojo y en verde*.....60

Figura 27. *Ejecución de RF31RegistrarAbonoDeClienteTest en rojo y en verde*.....61

Figura 28. *Ejecución de RF33CalcularAntiguedadDeLaDeudaTest en rojo y en verde* 62
Figura 29. *Ejecución de RF34AlertarDeudasVencidasTest en rojo y en verde*.....63
Figura 30. *Ejecución de RF35RegistrarCompromisoDePagoTest en rojo y en verde* 65
Figura 31. *Ejecución de RF36GenerarEstadoDeCuentaTest en rojo y en verde*……66
Figura 32. *Ejecución de RF38RegistrarGastoMenorTest en rojo y en verde*.....67
Figura 33. *Ejecución de RF39RealizarCuadreDeCajaTest en rojo y en verde*.....69
Figura 34. *Ejecución de RF40CalcularGananciaDelDiaTest en rojo y en verde*.....70
Figura 35. *Ejecución de RF42RegistrarFacturaEmitidaTest en rojo y en verde*.....72
Figura 36. *Ejecución de RF43AsociarVentasAFacturaTest en rojo y en verde*.....73
Figura 37. *Ejecución de RF45GenerarCierreMensualTest en rojo y en verde*.....74
Figura 38. *Ejecución de RF47ConsultarGananciaPorProductoTest en rojo y en verde* 76
Figura 39. *Ejecución de RF48ExportarReportesAExcelTest en rojo y en verde*.....77
Figura 40. *Ejecución de RF49GenerarRespaldoDeLaInformacionTest en rojo y en verde*..... 78
Figura 41. *Ramas del repositorio en GitHub (Code > Branches)*.....105
Figura 42. *Release apf3 publicado en GitHub*.....106
Figura 43. *Historial de commits de la rama main (pestaña Commits)*.....107
Figura 44. *Contribuidores del repositorio (Insights > Contributors)*.....107
Figura 45. *H01 · Vender: productos sin saldo (antes y después)*.....111
Figura 46. *H02 · Nueva compra: tipo de pago por defecto (antes y después)*.....112
Figura 47. *Mapa de navegación actualizado: pantallas y rol que las ve*.....113
Figura 48. *Menú de cada rol: Inicio (arriba) y Más opciones (abajo)*.....114
Figura 49. *Pantalla real P08 Nueva compra con datos del día*.....116
Figura 50. *Pantalla real P10 Vender con datos del día*.....117
Figura 51. *Pantalla real P14 Cobranza con datos del día*.....119
Figura 52. *Pantalla real P16 Gastos y cuadre con datos del día*.....120
Figura 53. *Pantalla real P18 Facturas con datos del día*.....122
Figura 54. *Diagrama entidad-relación final completo (vista general)*.....125
Figura 55. *DER final, parte A: compras, precios y saldo de mercadería (der_parte_a.drawio)*.....126

Figura 56. *DER final, parte B: ventas, créditos, caja, facturas y auditoría (der_parte_b.drawio)*.....127

Figura 57. *Ejecución del script SQL del Anexo E: 20 tablas y 2 vistas creadas sin errores*.....138

Figura 58. *Pull requests unidos con el check de GitHub Actions en verde (2/2)*.....140

Índice de tablas

Tabla 1. *Capas del sistema y su tecnología*.....15

Tabla 2. *Objetos del diagrama de secuencias de RF18 (flujo exitoso, bloques 1 a 4) y su lugar en el código*.....15

Tabla 3. *Interfaz VentaDao (extracto)*.....16

Tabla 4. *Implementación JdbcVentaDao: inserción con parámetros (extracto)*.....16

Tabla 5. *Principios SOLID aplicados en el código del repositorio*.....17

Tabla 6. *Controles de seguridad implementados*.....18

Tabla 7. *Ubicación de las secciones 1.7 a 1.9 en el Anexo F (confidencial)*.....18

Tabla 8. *Librerías de pruebas y de apoyo*.....19

Tabla 9. *Resumen de escenarios alternativos por RF*.....21

Tabla 10. *Ficha de casos de prueba de RF18 Registrar venta al crédito*.....23

Tabla 11. *Clasificación de aptitud de los 50 RF*.....24

Tabla 12. *Clase base de las pruebas: DAO falsos y servicios reales (extracto de TddBase)*.....26

Tabla 13. *Pruebas manuales de RF03 Cerrar sesión*.....79

Tabla 14. *Pruebas manuales de RF14 Consultar deuda con proveedores*.....81

Tabla 15. *Pruebas manuales de RF16 Consultar precios del día*.....83

Tabla 16. *Pruebas manuales de RF23 Consultar ventas del día*.....85

Tabla 17. *Pruebas manuales de RF24 Guardar venta sin conexión*.....87

Tabla 18. *Pruebas manuales de RF26 Mostrar estado de sincronización*.....89

Tabla 19. *Pruebas manuales de RF29 Alertar mercadería antigua*.....91

Tabla 20. *Pruebas manuales de RF32 Consultar saldo por cliente*.....91

Tabla 21. *Pruebas manuales de RF34 Alertar deudas vencidas*.....93

Tabla 22. *Pruebas manuales de RF37 Compartir estado de cuenta*.....94

Tabla 23. *Pruebas manuales de RF41 Consultar historial de cuadres*.....96

Tabla 24. *Pruebas manuales de RF44 Consultar ventas pendientes de facturar*.....98

Tabla 25. *Pruebas manuales de RF46 Consultar panel del negocio*.....100

Tabla 26. *Pruebas manuales de RF49 Generar respaldo de la información*.....102

Tabla 27. *Pruebas manuales de RF50 Consultar bitácora de auditoría*.....102

Tabla 28. *Resumen de las pruebas*.....104

Tabla 29. *Ramas del repositorio*.....105

Tabla 30. *Entregables del proyecto y su carpeta en el repositorio*.....105
Tabla 31. *Matriz de trazabilidad vista / RF (RF01 a RF15)*.....109
Tabla 32. *Matriz de hallazgos de las 4 capas*.....110
Tabla 33. *Módulos del alcance del APF2 y su estado*.....115
Tabla 34. *Vista 1 · 0FN*.....116
Tabla 35. *Vista 1 · 1FN*.....116
Tabla 36. *Vista 1 · 2FN*.....116
Tabla 37. *Vista 1 · 3FN*.....117
Tabla 38. *Vista 2 · 0FN*.....117
Tabla 39. *Vista 2 · 1FN*.....118
Tabla 40. *Vista 2 · 2FN*.....118
Tabla 41. *Vista 2 · 3FN*.....118
Tabla 42. *Vista 3 · 0FN*.....119
Tabla 43. *Vista 3 · 1FN*.....119
Tabla 44. *Vista 3 · 2FN*.....120
Tabla 45. *Vista 3 · 3FN*.....120
Tabla 46. *Vista 4 · 0FN*.....121
Tabla 47. *Vista 4 · 1FN*.....121
Tabla 48. *Vista 4 · 2FN*.....121
Tabla 49. *Vista 4 · 3FN*.....121
Tabla 50. *Vista 5 · 0FN*.....122
Tabla 51. *Vista 5 · 1FN*.....122
Tabla 52. *Vista 5 · 2FN*.....122
Tabla 53. *Vista 5 · 3FN*.....123
Tabla 54. *Campos de pantalla que quedaban libres y acción tomada*.....123
Tabla 55. *Relaciones del modelo final leídas de ida y vuelta*.....128
Tabla 56. *Esquema lógico del modelo final*.....128
Tabla 57. *Diccionario de datos: USUARIO — Persona que usa el sistema con un rol*.....129
Tabla 58. *Diccionario de datos: CONFIGURACION — Parámetros del negocio (una sola fila)*.....129
Tabla 59. *Diccionario de datos: PRODUCTO — Especie o producto que se vende por kilo*.....129

Tabla 60. *Diccionario de datos: CLIENTE — Restaurante o cliente frecuente con crédito*.....130

Tabla 61. *Diccionario de datos: PROVEEDOR — Mayorista o comerciante del terminal*.....130

Tabla 62. *Diccionario de datos: COMPRA — Compra de mercadería de la madrugada*.....130

Tabla 63. *Diccionario de datos: DETALLE_COMPRA — Producto comprado en una compra*.....130

Tabla 64. *Diccionario de datos: PAGO_PROVEEDOR — Pago total o parcial a un proveedor*.....131

Tabla 65. *Diccionario de datos: PRECIO_DIA — Precio de venta y costo promedio de un producto en un día*.....131

Tabla 66. *Diccionario de datos: LOTE_SALDO — Kilos de un producto según su fecha de ingreso (saldo con antigüedad)*.....131

Tabla 67. *Diccionario de datos: AJUSTE_SALDO — Cierre del puesto, autoconsumo o ajuste de kilos*.....131

Tabla 68. *Diccionario de datos: FACTURA — Factura emitida en SUNAT y registrada por el contador*.....132

Tabla 69. *Diccionario de datos: VENTA — Venta registrada en el puesto (contado, crédito o remate)*.....132

Tabla 70. *Diccionario de datos: DETALLE_VENTA — Producto vendido en una venta* 132

Tabla 71. *Diccionario de datos: ABONO — Pago total o parcial de un cliente*.....132

Tabla 72. *Diccionario de datos: ABONO_VENTA — Parte de un abono aplicada a una venta al crédito*.....133

Tabla 73. *Diccionario de datos: COMPROMISO_PAGO — Fecha y monto que el cliente promete pagar*.....133

Tabla 74. *Diccionario de datos: GASTO — Gasto menor del día*.....133

Tabla 75. *Diccionario de datos: CUADRE — Cuadre de caja y ganancia de un día* 133

Tabla 76. *Diccionario de datos: AUDITORIA — Bitácora de operaciones (RF50)*...134

Tabla 77. *Vistas SQL v_venta_total y v_saldo_cliente*.....134

Tabla 78. *Script SQL del modelo final (schema.sql)*.....134

Tabla 79. *Trazabilidad de los 10 RF principales: documento, código, base de datos y prueba*.....139

Tabla 80. *Responsable de cada módulo*.....140

Tabla 81*. Distribución de la sustentación*.....142

CAPÍTULO 1. DISEÑO DE LA SOLUCIÓN

Este capítulo muestra cómo está construido por dentro el producto (Propuesta 1: el sistema web SVC-Canomar que el equipo ya desarrolla) y presenta dos propuestas con hardware para el semillero, que resuelven problemas técnicos del mismo negocio.

Propuesta 1: arquitectura del producto

1.1. Arquitectura

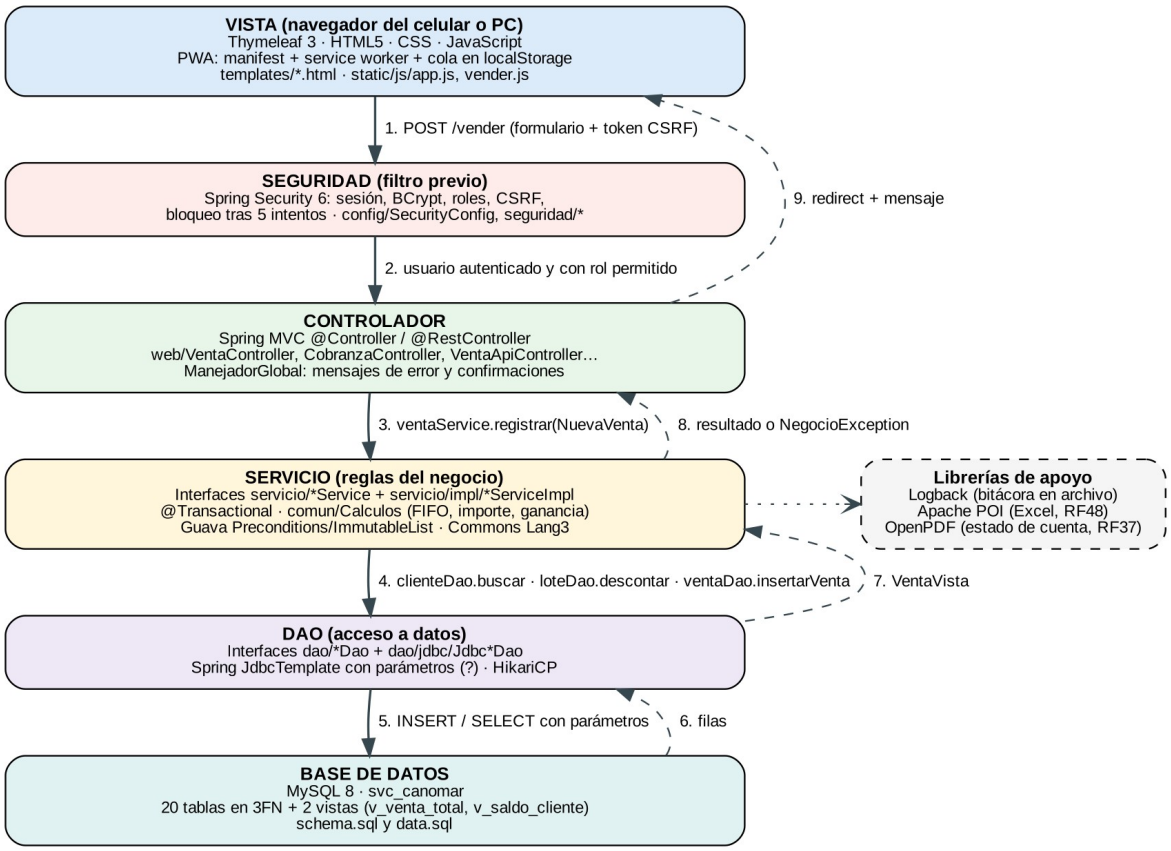


Figura 1. *Arquitectura en capas de SVC-Canomar y recorrido de una petición (RF18 Registrar venta al crédito)*

Fuente: Elaboración propia.

Una petición nace en la vista: el vendedor llena la pantalla Vender en el navegador del celular y envía el formulario con su token CSRF (1). Spring Security comprueba la sesión y el rol antes de que la petición llegue a la aplicación (2). El controlador VentaController no tiene reglas: arma el objeto NuevaVenta y llama a la interfaz VentaService (3). VentaServiceImpl aplica las reglas (precio, saldo, límite de crédito, deuda vencida) y usa solo interfaces DAO para leer y escribir (4); las implementaciones JDBC ejecutan sentencias con parámetros en MySQL (5). La respuesta vuelve por el mismo camino (6 a 9): la vista nunca llega directo a la base de datos.

Tabla 1*. Capas del sistema y su tecnología*

Capa	Tecnología	Carpeta en el repositorio
Vista	Thymeleaf 3.1, HTML5, CSS, JavaScript; PWA (manifest, service worker, cola en localStorage)	src/main/resources/ templates, static
Seguridad	Spring Security 6.3: sesión, BCrypt (10), roles, CSRF, bloqueo por intentos	config, seguridad
Controlador	Spring MVC 6 (@Controller, @RestController para la sincronización)	web
Servicio	Interfaces + implementaciones @Service, @Transactional; Guava, Commons Lang3, Logback	servicio, servicio/impl, comun
DAO	Interfaces + Spring JdbcTemplate con parámetros; HikariCP	dao, dao/jdbc
Base de datos	MySQL 8: 20 tablas en 3FN y 2 vistas	src/main/resources/ schema.sql, data.sql

1.2. Del diagrama de secuencias al código

El requerimiento principal del proyecto es RF18 Registrar venta al crédito: es el que más reglas reúne (precio del día, saldo, límite de crédito y deuda vencida) y es el que originó el sistema. Su diagrama de secuencias (APF2, figura 11 y archivo secuencia_RF18.drawio) tiene cuatro bloques en el flujo exitoso: acceso a la pantalla, ingreso y validación, registro y confirmación, y respuesta al actor. La tabla indica en qué clase y carpeta quedó cada objeto.

Tabla 2*. Objetos del diagrama de secuencias de RF18 (flujo exitoso, bloques 1 a 4) y su lugar en el código*

Objeto en la secuencia	Bloque	Capa MVC	Clase o archivo en el código	Carpeta
AS_Vendedor	1-4	Actor (rol)	Rol VENDEDOR en UsuarioSesion / SecurityConfig	seguridad, config
I_Inicio	1	Vista	inicio.html	resources/ templates
I_Venta	1-4	Vista	vender.html + vender.js (importe en vivo, cola sin señal)	resources/ templates, static/js
C_Venta	2-4	Controlador + Servicio	VentaController.registrar() → VentaServiceImpl.registrar()	web, servicio/impl
calcularImporte (auto-mensaje)	2	Servicio (modelo)	Calculos.importe() y NuevaVenta.total()	comun, modelo
C_Credito	2	Servicio	CreditoService.saldo() → CreditoServiceImpl	servicio, servicio/impl
C_Alerta (estaVencido)	2	Servicio	SaldoCliente.conVencido() usado en VentaServiceImpl	modelo, servicio/impl
E_Cliente	2	Modelo + DAO	Cliente, ClienteDao → JdbcClienteDao (tabla cliente)	modelo, dao, dao/jdbc
E_Precio	2	Modelo + DAO	PrecioDao → JdbcPrecioDao (tabla precio_dia)	dao, dao/jdbc
E_Venta	3	Modelo + DAO	NuevaVenta, VentaVista, VentaDao → JdbcVentaDao (venta, detalle_venta)	modelo, dao, dao/jdbc
E_LoteSaldo	3	Modelo + DAO	LoteDao → JdbcLoteDao (lote_saldo) y Calculos.consumirFifo()	dao, dao/jdbc, comun
Mensaje «Venta al crédito registrada…»	4	Vista	RedirectAttributes «ok» en VentaController, fragments/base.html :: mensajes	web, resources/templates

1.3. Patrón DAO

La entidad principal del sistema es la venta (tablas venta y detalle_venta del modelo físico, capítulo 5). Su interfaz VentaDao declara qué datos se necesitan; JdbcVentaDao implementa el cómo con JdbcTemplate y siempre con parámetros (?), lo que evita la inyección SQL.

Tabla 3*. Interfaz VentaDao (extracto)*

```
/** Acceso a venta y detalle_venta, y consultas de créditos y ganancias. */
public interface VentaDao {
    long insertarVenta(NuevaVenta venta, int idUsuario, LocalDateTime
fechaSincronizacion);

    void insertarDetalle(long idVenta, int idProducto, BigDecimal kilos,
BigDecimal precioKg, BigDecimal costoKg);

    boolean existeUuid(String uuid);

    Optional<VentaVista> buscar(long idVenta);

    List<DetalleVentaVista> detalle(long idVenta);

    List<VentaVista> listarEntre(LocalDate desde, LocalDate hasta);

    void anular(long idVenta, String motivo);
    // … consultas de créditos, ganancias y facturas
}
```

Corresponde a las tablas **venta** y **detalle_venta** (sección 5.2.3).

Tabla 4*. Implementación JdbcVentaDao: inserción con parámetros (extracto)*

```
@Repository
public class JdbcVentaDao extends JdbcBase implements VentaDao {

    // …
    @Override
    public long insertarVenta(NuevaVenta v, int idUsuario, LocalDateTime
fechaSincronizacion) {
        return insertarConLlave("INSERT INTO venta (uuid_dispositivo, id_usuario,
id_cliente, fecha_hora, fecha_sincronizacion, tipo_pago, medio_pago, es_remate,
estado) "
                        + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'VIGENTE')",
                v.uuid(), idUsuario, v.idCliente(), v.fechaHora(),
fechaSincronizacion, v.tipoPago(), v.medioPago(), v.esRemate());
    }

    @Override
    public void insertarDetalle(long idVenta, int idProducto, BigDecimal kilos,
BigDecimal precioKg, BigDecimal costoKg) {
        jdbc.update("INSERT INTO detalle_venta (id_venta, id_producto, kilos,
precio_kg, costo_kg) VALUES (?, ?, ?, ?, ?)",
                idVenta, idProducto, kilos, precioKg, costoKg);
    }
    // …
}
```

El servicio recibe la interfaz y no la implementación. Spring inyecta JdbcVentaDao al arrancar; en las pruebas se pasa un DAO falso:

```
    private final VentaDao ventaDao;
    private final ProductoDao productoDao;
    private final ClienteDao clienteDao;
    private final PrecioDao precioDao;
```

```
    private final LoteDao loteDao;
    private final CreditoService credito;
    private final AuditoriaService auditoria;
    private final TransactionTemplate tx;

    public VentaServiceImpl(VentaDao ventaDao, ProductoDao productoDao,
ClienteDao clienteDao, PrecioDao precioDao, LoteDao loteDao,
                            CreditoService credito, AuditoriaService auditoria,
PlatformTransactionManager txManager) {
        this.ventaDao = ventaDao;
        this.productoDao = productoDao;
        this.clienteDao = clienteDao;
        this.precioDao = precioDao;
        this.loteDao = loteDao;
        this.credito = credito;
        this.auditoria = auditoria;
        this.tx = new TransactionTemplate(txManager);
    }
```

1.4. Principios SOLID

Tabla 5*. Principios SOLID aplicados en el código del repositorio*

Principio	Clase donde se aplica	Fragmento de código	Explicación
S · Responsabilidad única	Calculos (comun)	public static BigDecimal importe(BigDecimal kilos, BigDecimal precioKg)	Solo calcula (importe, FIFO, tramos, ganancia); no accede a la base ni a la pantalla.
O · Abierto/cerrado	ManejadorGlobal (web)	@ExceptionHandler(NegocioException .class) public String negocio(NegocioException e, …)	Cualquier regla nueva lanza NegocioException y el mensaje llega a la pantalla sin modificar ningún controlador.
L · Sustitución de Liskov	JdbcVentaDao y el DAO falso de las pruebas	public class JdbcVentaDao extends JdbcBase implements VentaDao	VentaServiceImpl funciona igual con la implementación JDBC o con el DAO falso de Mockito: ambos cumplen el contrato de VentaDao.
I · Segregación de interfaces	CompromisoDao (dao)	int insertar(…); List<CompromisoVista> listarVigentes(); int marcarIncumplidos(LocalDate hoy); void marcarCumplidos(int idCliente);	Una interfaz pequeña por entidad: cada servicio depende solo de los métodos que usa.
D · Inversión de dependencias	VentaServiceImpl (servicio/impl)	public VentaServiceImpl(VentaDao ventaDao, ProductoDao productoDao, …, CreditoService credito, …)	El servicio depende de abstracciones (interfaces) que recibe por el constructor, no de clases concretas.

1.5. Preparado para TDD

Las reglas no están en la pantalla ni en los controladores: viven en las clases Service y en la clase de cálculos. Para el RF principal, la clase es **VentaServiceImpl** (servicio/impl), que decide el precio aplicado, el aviso de saldo, el límite de crédito y la deuda vencida, y usa Calculos para el importe y el descuento FIFO. Como VentaServiceImpl recibe interfaces DAO por el constructor, en las pruebas se reemplazan por DAO falsos creados con Mockito: no se necesita la base de datos ni Spring para probar una regla. La evidencia de los 149 tests automatizados, con sus capturas en rojo y en verde, está en el capítulo 2.

1.6. Seguridad

Tabla 6*. Controles de seguridad implementados*

Riesgo	Control	Dónde está
Robo de contraseñas	Se guardan cifradas con BCrypt (costo 10); nunca en texto plano.	SecurityConfig.passwordEncoder(), UsuarioServiceImpl
Fuerza bruta en el acceso	Al quinto intento fallido el usuario queda bloqueado 5 minutos; el mensaje no revela qué dato falló.	ManejadorAcceso (RF02, tests RF02-CP02 y CP03)
Acceso a opciones de otro rol	Rutas limitadas por rol en el filtro y @PreAuthorize en los controladores; los menús solo muestran las opciones del rol.	SecurityConfig, controladores, sec:authorize en plantillas
Envío de formularios falsos (CSRF)	Token CSRF en cada formulario y en la sincronización.	Spring Security, app.js (cabecera del token)
Inyección SQL	Todas las consultas usan parámetros (?) con JdbcTemplate.	dao/jdbc/*
Credenciales en el repositorio	La contraseña de MySQL se lee de la variable DB_PASSWORD; application-local.properties y .env están en .gitignore.	application.properties, .gitignore (sección 3.4)
Ventas duplicadas al sincronizar	Cada venta del celular tiene un identificador único (UNIQUE en la base).	venta.uuid_dispositivo, VentaServiceImpl.sincronizar() (RF25)
Cambios sin rastro	Bitácora de auditoría en la base y en archivo diario (Logback).	AuditoriaServiceImpl, logback-spring.xml (RF50)

Propuestas 2 y 3: evolución con hardware

Las dos propuestas con componente físico para el semillero (secciones 1.7 a 1.9) están completas en el **Anexo F, que se entrega solo por UTP+Class y no se sube a GitHub**, para proteger la novedad de las ideas antes de postularlas. El anexo contiene, para cada propuesta, la ficha de 10 campos alineada al Anexo B de las bases, el diagrama de bloques en draw.io, los antecedentes (patente, artículo y producto comercial), el presupuesto por fase y la viabilidad en 6 meses, además del comparativo y la decisión.

Tabla 7*. Ubicación de las secciones 1.7 a 1.9 en el Anexo F (confidencial)*

Sección	Contenido	Dónde está
1.7. Propuesta 2	Ficha de 10 campos, diagrama de bloques y presupuesto	Anexo F, Propuesta 2
1.8. Propuesta 3	Ficha de 10 campos, diagrama de bloques y presupuesto	Anexo F, Propuesta 3
1.9. Comparativo	Tabla de criterios y propuesta que se postula al semillero	Anexo F, Comparativo y propuesta que se postula

CAPÍTULO 2. PRUEBAS TDD Y LIBRERÍAS

Este capítulo reúne el trabajo de la semana 11 en sus tres momentos: los escenarios de cada CUS (momento 1), las fichas de casos de prueba (momento 2) y la clasificación de aptitud con la evidencia de los tests (momento 3). Los 50 CUS y las 50 fichas completas están en los Anexos A y B (Excel).

2.1. Librerías

Tabla 8*. Librerías de pruebas y de apoyo*

Librería	Versión	Para qué se usa	Clase donde se usa	Consideración de seguridad
JUnit 5 (Jupiter)	5.10 (spring-boot-starter-test)	Ejecutar los 149 tests de los CP de Regla e Infraestructura	pe.canomar.svc.tdd.RF##…Test	Solo en alcance test: no viaja en el .jar de producción.
Mockito	5.11 (spring-boot-starter-test)	Crear los DAO falsos para probar los servicios sin base de datos	TddBase (mock(VentaDao.class)…)	Alcance test; los datos de prueba son ficticios.
Guava	33.3.1-jre	Validar argumentos (Preconditions) y devolver listas inmodificables (ImmutableList)	Calculos	Las listas inmodificables evitan que otra capa altere los lotes o abonos ya calculados.
Apache Commons Lang3	3.14 (gestionada por Spring Boot)	Validar y limpiar textos (isBlank, normalizeSpace, left)	UsuarioServiceImpl, VentaServiceImpl, ClienteServiceImpl	Recorta y normaliza entradas antes de guardarlas; limita la longitud de motivos y descripciones.
Logback (SLF4J)	1.5 (spring-boot-starter)	Registro en consola y en archivo diario logs/svc-canomar.log	VentaServiceImpl, AuditoriaServiceImpl, ManejadorAcceso	No registra contraseñas ni tokens; la carpeta logs/ está en .gitignore.
Apache POI	5.3.0	Exportar reportes a Excel (RF48)	ReporteServiceImpl	Genera el archivo en memoria; no ejecuta fórmulas ni macros de terceros.
Spring Security Crypto (BCrypt)	6.3 (spring-boot-starter-security)	Cifrar y comprobar contraseñas	SecurityConfig, UsuarioServiceImpl	Hash con sal y costo 10; nunca se guarda la contraseña.

pom.xml · dependencias (svc-canomar)

```xml
    <dependencies>
        <!-- Web MVC + vistas Thymeleaf -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
        <dependency>
            <groupId>org.thymeleaf.extras</groupId>
            <artifactId>thymeleaf-extras-springsecurity6</artifactId>
        </dependency>
        <!-- Seguridad: login, roles y BCrypt -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <!-- Acceso a datos con el patrón DAO (JdbcTemplate) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-jdbc</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- Librerías de apoyo exigidas por el curso -->
        <dependency>
            <groupId>com.google.guava</groupId>
            <artifactId>guava</artifactId>
            <version>${guava.version}</version>
        </dependency>
        <dependency>
            <groupId>org.apache.commons</groupId>
            <artifactId>commons-lang3</artifactId>
        </dependency>
        <dependency>
            <groupId>org.apache.poi</groupId>
            <artifactId>poi-ooxml</artifactId>
            <version>${poi.version}</version>
        </dependency>
        <!-- Logback viene incluido en spring-boot-starter (configurado en logback-spring.xml) -->

        <!-- PDF del estado de cuenta (RF36, RF37) -->
        <dependency>
            <groupId>com.github.librepdf</groupId>
            <artifactId>openpdf</artifactId>
            <version>${openpdf.version}</version>
        </dependency>

        <!-- Pruebas (TDD): JUnit 5 + Mockito -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.security</groupId>
            <artifactId>spring-security-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

Figura 2. *Archivo de dependencias pom.xml del repositorio*

Fuente: Elaboración propia.

2.2. Momento 1: escenarios

Cada CUS tiene un evento exitoso y tres eventos alternativos. La tabla resume los 50 RF; el detalle (precondición, flujo básico, eventos, postcondición y reglas) está en el Anexo A.

Tabla 9*. Resumen de escenarios alternativos por RF*

RF	Nombre	N° alternativos	Eventos alternativos (E2, E3, E4)
RF01	Registrar usuario	3	El nombre de usuario ya existe: el sistema pide otro. · La contraseña tiene menos de 8 caracteres: el sistema la rechaza. · El rol no es vendedor, administrador ni contador: el sistema lo rechaza.
RF02	Iniciar sesión	3	La contraseña es incorrecta: se muestra un mensaje genérico y se suma un intento fallido. · Es el quinto intento fallido: el acceso se bloquea 5 minutos. · La cuenta está desactivada: el sistema no permite el ingreso.
RF03	Cerrar sesión	3	Hay ventas sin enviar: el sistema pide confirmar; si el usuario cancela, la sesión sigue activa. · Después de cerrar sesión, una página protegida pide ingresar de nuevo. · No hay ventas pendientes: Mi cuenta indica que se puede cerrar sesión con seguridad.
RF04	Cambiar contraseña	3	Las contraseñas nuevas no coinciden: el sistema lo indica. · La contraseña actual es incorrecta: el sistema rechaza el cambio. · La nueva contraseña tiene menos de 8 caracteres: el sistema la rechaza.
RF05	Desactivar usuario	3	El administrador intenta desactivar su propia cuenta: el sistema lo impide. · Es el único administrador activo: el sistema lo impide. · El usuario inactivo se vuelve a activar: recupera el acceso.
RF06	Registrar producto	3	No se indica nombre corto: el sistema usa el nombre (hasta 20 letras) para la pantalla de venta. · El producto ya existe: el sistema lo informa. · Falta el nombre: el sistema lo solicita.
RF07	Editar producto	3	Se inactiva un producto sin saldo: deja de mostrarse en la venta. · El producto tiene saldo de mercadería: el sistema no permite inactivarlo. · El nuevo nombre duplica otro producto: el sistema lo rechaza.
RF08	Registrar cliente	3	El cliente no tiene RUC: se registra sin RUC y no podrá recibir factura. · El RUC no tiene 11 dígitos: el sistema lo rechaza. · El RUC ya pertenece a otro cliente: el sistema lo rechaza.
RF09	Editar cliente	3	Se reduce el límite por debajo del saldo actual: el sistema advierte que no podrá comprar al crédito hasta abonar. · El plazo es mayor a 90 días: el sistema lo rechaza. · El límite es negativo: el sistema lo rechaza.
RF10	Registrar proveedor	3	Es un comerciante ocasional del terminal: se registra solo con su nombre. · El proveedor ya existe: el sistema lo informa. · El RUC no tiene 11 dígitos: el sistema lo rechaza.
RF11	Registrar compra del día	3	La compra es a crédito: queda como deuda con el proveedor. · Los kilos o el costo son cero: el sistema los rechaza. · El producto está inactivo: el sistema no permite comprarlo.
RF12	Calcular costo promedio por kilo	3	No hay saldo anterior: el costo promedio es el de la compra del día. · No hay compra ni saldo del producto: el costo es cero y no se guarda. · El producto no tiene precio hoy: el costo se guarda con el último precio como sugerido.
RF13	Registrar pago a proveedor	3	El pago es parcial: el saldo pendiente se mantiene. · El monto supera la deuda: el sistema lo rechaza. · El monto es cero: el sistema lo rechaza.
RF14	Consultar deuda con proveedores	3	Se muestra la deuda total con proveedores en la parte superior. · Se muestran los pagos recientes debajo de la lista. · El vendedor intenta entrar: el sistema muestra «Sin permiso».
RF15	Registrar precios de venta del día	3	El vendedor mantiene los precios de ayer con un toque. · Un precio es menor que el costo promedio: el sistema advierte la pérdida y pide confirmar. · Un precio es cero: el sistema lo rechaza.
RF16	Consultar precios del día	3	Un producto sin precio hoy aparece en gris con «—». · La pantalla de precios muestra el costo promedio y el precio de ayer como referencia. · El contador intenta vender: el sistema muestra «Sin permiso».
RF17	Registrar venta al contado	3	El cliente lleva varios productos: se agregan a la misma venta. · Los kilos superan el saldo disponible: el sistema advierte y pide confirmar. · No se ingresan los kilos de un producto: el sistema los pide.

RF	Nombre	N° alternativos	Eventos alternativos (E2, E3, E4)
RF18	Registrar venta al crédito	3	El restaurante tiene deuda vencida: el sistema lo advierte y pide confirmar. · La venta supera el límite de crédito: el vendedor no puede continuar sin autorización del administrador del negocio. · El cliente no tiene crédito habilitado (límite S/ 0): el sistema solo permite venta al contado.
RF19	Calcular importe por peso	3	Se aplica el precio de remate: el sistema usa el precio indicado. · Los kilos tienen más de dos decimales: el sistema los redondea. · El producto no tiene precio del día: el sistema pide ingresarlo.
RF20	Anular venta	3	La venta tenía factura: la bitácora advierte que se debe emitir una nota de crédito en SUNAT. · La venta al crédito ya tiene abonos: el sistema no permite anularla. · Falta el motivo: el sistema lo solicita.
RF21	Registrar venta de remate	3	El remate es a un restaurante con crédito: se registra al crédito. · El precio de remate es igual o mayor que el del día: el sistema lo rechaza. · El producto no tiene precio de remate ni del día: el sistema pide ingresarlo.
RF22	Registrar autoconsumo	3	Hay lotes de varios días: el sistema toma primero el más antiguo (FIFO). · Los kilos superan el saldo: el sistema lo rechaza. · Los kilos son cero: el sistema los pide.
RF23	Consultar ventas del día	3	Se elige otra fecha: el sistema muestra las ventas de esa fecha. · No hay ventas en la fecha: el sistema lo informa. · Se abre una venta: se ve su detalle por producto.
RF24	Guardar venta sin conexión	3	Se registran varias ventas seguidas sin señal: todas quedan en cola. · Se cierra y se vuelve a abrir la página: las ventas pendientes se conservan. · La pantalla Vender avisa que no hay conexión.
RF25	Sincronizar ventas pendientes	3	Una venta se envía dos veces: el servidor la reconoce por su identificador y no la duplica. · Una venta no cumple una regla: se rechaza con su motivo y las demás se guardan. · Una venta llega sin identificador: se rechaza.
RF26	Mostrar estado de sincronización	3	Sin señal y con ventas guardadas: el indicador se muestra en rojo con su número («Sin señal · 2»); con señal pero sin enviar, en naranja («1 pendiente»). · Vuelve la conexión: las ventas se envían solas, se informa cuántas se sincronizaron y el indicador vuelve a «Enviado». · Una venta fue rechazada al sincronizar: se muestra el motivo.
RF27	Registrar saldo para el día siguiente	3	Sobró más de lo esperado: la diferencia se suma al saldo. · Los kilos reales difieren más del 5 %: el sistema pide el motivo. · Los kilos reales son negativos: el sistema los rechaza.
RF28	Consultar saldo por antigüedad	3	El lote está a un día de la alerta: se muestra en naranja. · El lote llegó a los días de alerta: se muestra en rojo. · Se consulta el saldo: cada lote trae sus días calculados desde su ingreso.
RF29	Alertar mercadería antigua	3	No hay mercadería antigua: no se muestra alerta. · El administrador cambia los días de alerta: la alerta usa el nuevo valor. · La alerta se muestra en Inicio con enlaces a remate y autoconsumo.
RF30	Registrar ajuste de saldo	3	El ajuste es positivo por un error de conteo: el sistema lo suma. · Falta el motivo: el sistema lo solicita. · El ajuste deja el saldo negativo: el sistema lo rechaza.
RF31	Registrar abono de cliente	3	El abono cubre la deuda completa: el cliente queda al día y sus compromisos se cumplen. · El monto supera el saldo: el sistema lo rechaza. · No se elige el medio de pago: el sistema lo pide.
RF32	Consultar saldo por cliente	3	Se abre un cliente: se ve su saldo, disponible y vencido. · El cliente no tiene ventas al crédito: se muestra con saldo S/ 0.00. · El vendedor consulta la lista de clientes: ve los saldos para saber a quién vender al crédito.
RF33	Calcular antigüedad de la deuda	3	Toda la deuda es reciente: solo el primer tramo tiene monto. · Un abono reciente cubre ventas antiguas: los tramos se recalculan sin esas ventas. · Se consulta la antigüedad de un cliente que no existe: el sistema lo informa.
RF34	Alertar deudas vencidas	3	Ningún cliente está vencido: no hay alerta. · Un cliente está sobre su límite aunque no tenga vencido: también se alerta. · La alerta se muestra en Inicio y en Cobranza con el monto vencido.
RF35	Registrar compromiso de pago	3	Llega la fecha sin abono: el compromiso se marca como incumplido al consultar. · La fecha es anterior a hoy: el sistema la rechaza. · El cliente no tiene deudas: el sistema no permite registrar el compromiso.
RF36	Generar estado de cuenta	3	El cliente no tiene movimientos en el periodo: el saldo final es igual al inicial. · La fecha inicial es mayor que la final: el sistema pide revisar el rango. · El periodo es mayor a un año: el sistema pide reducirlo.

RF	Nombre	N° alternativos	Eventos alternativos (E2, E3, E4)
RF37	Compartir estado de cuenta	3	En el celular, el botón Compartir abre las aplicaciones instaladas (WhatsApp, correo). · El navegador no permite compartir: el botón descarga el PDF. · El PDF muestra cliente, periodo, movimientos y saldo final.
RF38	Registrar gasto menor	3	El concepto no está en la lista: se elige Otro y se describe. · El monto es cero: el sistema lo rechaza. · Se elige Otro sin descripción: el sistema la pide.
RF39	Realizar cuadre de caja	3	Hay una diferencia menor a la tolerancia (S/ 5): se guarda sin pedir motivo. · Hay una diferencia mayor: el sistema exige el motivo. · El día ya tiene cuadre: el sistema no permite otro.
RF40	Calcular ganancia del día	3	Hubo remates, mermas o autoconsumo: entran como pérdidas. · El día aún no tiene cuadre: el resultado se marca como provisional. · No hubo ventas: el margen es 0 %.
RF41	Consultar historial de cuadres	3	Se elige un rango: se muestran solo esos días. · No hay cuadres en el rango: el sistema lo informa. · La fecha inicial es mayor que la final: el sistema lo indica.
RF42	Registrar factura emitida	3	La factura está sujeta a detracción: se marca para su control. · El número de factura ya existe: el sistema lo rechaza. · El cliente no tiene RUC: el sistema no permite registrar factura.
RF43	Asociar ventas a factura	3	La suma de las ventas difiere del importe: el sistema muestra la diferencia y pide confirmar. · Una venta ya está asociada a otra factura o es de otro cliente: el sistema la rechaza. · No se selecciona ninguna venta: el sistema lo pide.
RF44	Consultar ventas pendientes de facturar	3	Se elige otro mes: el sistema muestra solo ese mes. · No hay pendientes: el sistema lo informa. · Desde la lista se abre el cliente para registrar su factura.
RF45	Generar cierre mensual	3	Hay días con ventas sin cuadre: se cuentan como advertencia. · El mes aún no termina: el cierre se marca como parcial. · El mes aún no empieza: el sistema lo rechaza.
RF46	Consultar panel del negocio	3	Es temprano y no hay ventas: el panel muestra S/ 0.00 en ventas y el resto de datos. · Hay mercadería antigua: el panel muestra la alerta. · Hay clientes vencidos: el panel muestra la alerta con el enlace a Cobranza.
RF47	Consultar ganancia por producto	3	Un producto se vendió por debajo de su costo (remates): su ganancia es negativa. · La fecha inicial es mayor que la final: el sistema pide revisar el rango. · Un producto no tuvo ventas: su margen es 0 %.
RF48	Exportar reportes a Excel	3	Se exporta el cierre mensual: el archivo tiene la hoja del cierre y la hoja por producto. · Se exporta la ganancia por producto con sus columnas de margen. · El rango de fechas está invertido: el sistema no genera el archivo.
RF49	Generar respaldo de la información	3	Cada respaldo queda registrado en la bitácora con su tamaño. · El respaldo conserva tildes y eñes (UTF-8). · La página de respaldo solo la ve el administrador del negocio.
RF50	Consultar bitácora de auditoría	3	Se filtra por operación (por ejemplo ANULAR_VENTA): se muestran solo esas. · No hay eventos en el filtro: el sistema lo informa. · La fecha inicial es mayor que la final: el sistema lo indica.

2.3. Momento 2: ficha de casos de prueba

Ficha completa del RF principal, RF18 Registrar venta al crédito. Las 50 fichas están en el Anexo B.

Tabla 10*. Ficha de casos de prueba de RF18 Registrar venta al crédito*

CP	Escenario	Precondición	Acción del actor	Resultado esperado	Resultado obtenido
RF18-CP01	E1: La venta al crédito queda registrada a nombre del restaurante y suma a su saldo.	Cevichería El Muelle (cliente 1): límite S/ 1,500, saldo S/ 228, sin vencido.	Vender 10 kg de corvina al crédito.	Se guarda la venta CREDITO del cliente 1 por S/ 320.00, sin medio de pago.	Cumple. El test rf18_cp01_venta Credito pasa (verde); antes de programar fallaba (rojo).

CP	Escenario	Precondición	Acción del actor	Resultado esperado	Resultado obtenido
RF18-CP02	E2: El restaurante tiene deuda vencida: el sistema lo advierte y pide confirmar.	Restaurante Sabor Norteño (cliente 2) con S/ 1,372 vencido y S/ 128 disponibles.	Vender 2 kg de corvina al crédito sin confirmar.	Pide confirmar: «Restaurante Sabor Norteño tiene deuda vencida de S/ 1372.00. ¿Registrar igual?».	Cumple. El test rf18_cp02_clienteConVencido pasa (verde); antes de programar fallaba (rojo).
RF18-CP03	E3: La venta supera el límite de crédito: el vendedor no puede continuar sin autorización del administrador del negocio.	Cliente 2: disponible S/ 128.	El vendedor vende 10 kg de corvina (S/ 320) al crédito.	Mensaje «Supera el límite de crédito: disponible S/ 128.00 de S/ 1500.00. Requiere autorización del administrador».	Cumple. El test rf18_cp03_superaLimiteVendedor pasa (verde); antes de programar fallaba (rojo).
RF18-CP04	E4: El cliente no tiene crédito habilitado (límite S/ 0): el sistema solo permite venta al contado.	Pollería Don Lucho (cliente 3) con límite S/ 0.	Vender al crédito a la pollería.	Mensaje «Este cliente solo puede comprar al contado».	Cumple. El test rf18_cp04_sinCreditoHabilitado pasa (verde); antes de programar fallaba (rojo).

2.4. Momento 3: clasificación de aptitud

Un RF es apto para TDD si su resultado esperado se puede escribir como un sí/no, un número o un cambio de estado. Los RF que solo muestran o listan datos se prueban de forma manual con captura. Regla e Infraestructura se prueban con código; Pantalla, a mano.

Tabla 11*. Clasificación de aptitud de los 50 RF*

RF	Nombre	¿Qué decide?	¿Apto para TDD?	CP de Regla	CP de Infraestructura	CP de Pantalla
RF01	Registrar usuario	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF02	Iniciar sesión	Cambia estado	Sí	—	CP01, CP02, CP03, CP04	—
RF03	Cerrar sesión	Solo muestra	No	—	—	CP01, CP02, CP03, CP04
RF04	Cambiar contraseña	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF05	Desactivar usuario	Cambia estado	Sí	CP01, CP02, CP03, CP04	—	—
RF06	Registrar producto	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF07	Editar producto	Cambia estado	Sí	CP01, CP02, CP03, CP04	—	—
RF08	Registrar cliente	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF09	Editar cliente	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF10	Registrar proveedor	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF11	Registrar compra del día	Cambia estado	Sí	CP01, CP02, CP03, CP04	—	—
RF12	Calcular costo promedio por kilo	Calcula	Sí	CP01, CP02, CP03, CP04	—	—
RF13	Registrar pago a proveedor	Cambia estado	Sí	CP01, CP02, CP03, CP04	—	—
RF14	Consultar deuda con proveedores	Solo muestra	No	—	—	CP01, CP02, CP03, CP04

RF	Nombre	¿Qué decide?	¿Apto para TDD?	CP de Regla	CP de Infraestructura	CP de Pantalla
RF15	Registrar precios de venta del día	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF16	Consultar precios del día	Solo muestra	No	—	—	CP01, CP02, CP03, CP04
RF17	Registrar venta al contado	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF18	Registrar venta al crédito	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF19	Calcular importe por peso	Calcula	Sí	CP01, CP02, CP03, CP04	—	—
RF20	Anular venta	Cambia estado	Sí	CP01, CP02, CP03, CP04	—	—
RF21	Registrar venta de remate	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF22	Registrar autoconsumo	Calcula	Sí	CP01, CP02, CP03, CP04	—	—
RF23	Consultar ventas del día	Solo muestra	No	—	—	CP01, CP02, CP03, CP04
RF24	Guardar venta sin conexión	Solo muestra	No	—	—	CP01, CP02, CP03, CP04
RF25	Sincronizar ventas pendientes	Cambia estado	Sí	—	CP01, CP02, CP03, CP04	—
RF26	Mostrar estado de sincronización	Solo muestra	No	—	—	CP01, CP02, CP03, CP04
RF27	Registrar saldo para el día siguiente	Cambia estado	Sí	CP01, CP02, CP03, CP04	—	—
RF28	Consultar saldo por antigüedad	Calcula	Sí	CP01, CP02, CP03, CP04	—	—
RF29	Alertar mercadería antigua	Valida	Sí	CP01, CP02, CP03	—	CP04
RF30	Registrar ajuste de saldo	Cambia estado	Sí	CP01, CP02, CP03, CP04	—	—
RF31	Registrar abono de cliente	Cambia estado	Sí	CP01, CP02, CP03, CP04	—	—
RF32	Consultar saldo por cliente	Solo muestra	No	—	—	CP01, CP02, CP03, CP04
RF33	Calcular antigüedad de la deuda	Calcula	Sí	CP01, CP02, CP03, CP04	—	—
RF34	Alertar deudas vencidas	Valida	Sí	CP01, CP02, CP03	—	CP04
RF35	Registrar compromiso de pago	Cambia estado	Sí	CP01, CP02, CP03, CP04	—	—
RF36	Generar estado de cuenta	Calcula	Sí	CP01, CP02, CP03, CP04	—	—
RF37	Compartir estado de cuenta	Solo muestra	No	—	—	CP01, CP02, CP03, CP04
RF38	Registrar gasto menor	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF39	Realizar cuadre de caja	Calcula	Sí	CP01, CP02, CP03, CP04	—	—
RF40	Calcular ganancia del día	Calcula	Sí	CP01, CP02, CP03, CP04	—	—
RF41	Consultar historial de cuadres	Solo muestra	No	—	—	CP01, CP02, CP03, CP04
RF42	Registrar factura emitida	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF43	Asociar ventas a factura	Valida	Sí	CP01, CP02, CP03, CP04	—	—
RF44	Consultar ventas pendientes de facturar	Solo muestra	No	—	—	CP01, CP02, CP03, CP04

RF	Nombre	¿Qué decide?	¿Apto para TDD?	CP de Regla	CP de Infraestructura	CP de Pantalla
RF45	Generar cierre mensual	Calcula	Sí	CP01, CP02, CP03, CP04	—	—
RF46	Consultar panel del negocio	Solo muestra	No	—	—	CP01, CP02, CP03, CP04
RF47	Consultar ganancia por producto	Calcula	Sí	CP01, CP02, CP03, CP04	—	—
RF48	Exportar reportes a Excel	Cambia estado	Sí	—	CP01, CP02, CP03, CP04	—
RF49	Generar respaldo de la información	Cambia estado	Sí	—	CP01, CP02, CP03	CP04
RF50	Consultar bitácora de auditoría	Solo muestra	No	—	—	CP01, CP02, CP03, CP04
Total			38 RF aptos	134 CP	15 CP	51 CP

2.5. Evidencia de tests automatizados

Por cada RF apto hay una clase de prueba en src/test/java/pe/canomar/svc/tdd con un test por CP de tipo Regla o Infraestructura. El nombre del método lleva el código del CP (por ejemplo rf18_cp03_…) y su @DisplayName lo muestra completo (RF18-CP03). Cada test tiene los comentarios ARRANGE / ACT / ASSERT y usa los DAO falsos de la clase base TddBase.

Cómo se obtuvo la captura en rojo: se ejecutó cada clase de prueba contra una copia del código donde los métodos de las clases Service y de Calculos todavía no tienen implementación (lanzan UnsupportedOperationException("Pendiente de implementar")), igual que cuando se escribe el test antes que el código. La captura en verde es la misma ejecución con el código implementado. Las dos ejecuciones se hicieron con JUnit 5 desde la consola; en IntelliJ se obtiene lo mismo con clic derecho > Run sobre la clase.

Tabla 12*. Clase base de las pruebas: DAO falsos y servicios reales (extracto de TddBase)*

```
abstract class TddBase {

    // ---------- DAO falsos ----------
    protected final UsuarioDao usuarioDao = mock(UsuarioDao.class);
    protected final ProductoDao productoDao = mock(ProductoDao.class);
    protected final ClienteDao clienteDao = mock(ClienteDao.class);
    protected final ProveedorDao proveedorDao = mock(ProveedorDao.class);
    protected final CompraDao compraDao = mock(CompraDao.class);
    protected final LoteDao loteDao = mock(LoteDao.class);
    protected final PrecioDao precioDao = mock(PrecioDao.class);
    protected final VentaDao ventaDao = mock(VentaDao.class);
    protected final AbonoDao abonoDao = mock(AbonoDao.class);
    protected final CompromisoDao compromisoDao = mock(CompromisoDao.class);
    protected final GastoDao gastoDao = mock(GastoDao.class);
    protected final CuadreDao cuadreDao = mock(CuadreDao.class);
    protected final FacturaDao facturaDao = mock(FacturaDao.class);
    protected final ConfiguracionDao configDao = mock(ConfiguracionDao.class);
    protected final AuditoriaDao auditoriaDao = mock(AuditoriaDao.class);
    protected final RespaldoDao respaldoDao = mock(RespaldoDao.class);
    // … servicios vecinos simulados y datos de prueba
    protected VentaServiceImpl ventaService() {
        return new VentaServiceImpl(ventaDao, productoDao, clienteDao, precioDao,
loteDao, credito, auditoria, txManager);
```

```
    }
}
```

RF01 Registrar usuario · RF01RegistrarUsuarioTest

```
@DisplayName("RF01 Registrar usuario")
class RF01RegistrarUsuarioTest extends TddBase {

    /** RF01-CP01 (E1). Se guarda el usuario con la clave cifrada (empieza con $2a$) y se
registra REGISTRAR_USUARIO en la bitácora. */
    @Test
    @DisplayName("RF01-CP01 Registrar «Caja del puesto», usuario «caja2», rol VENDEDOR,
clave «Pescado2026».")
    void rf01_cp01_registraUsuarioConClaveCifrada() throws Exception {
        // ARRANGE
        when(usuarioDao.existeUsuario("caja2")).thenReturn(false);
        when(usuarioDao.insertar(anyString(), anyString(), anyString(),
anyString())).thenReturn(4);
        // ACT
        int id = usuarioService().registrar("Caja del puesto", "caja2", "VENDEDOR",
"Pescado2026", 1);
        // ASSERT
        assertEquals(4, id);
        ArgumentCaptor<String> hash = ArgumentCaptor.forClass(String.class);
        verify(usuarioDao).insertar(eq("Caja del puesto"), eq("caja2"), hash.capture(),
eq("VENDEDOR"));
        assertTrue(hash.getValue().startsWith("$2a$"), "la clave debe guardarse cifrada con
BCrypt");
        verify(auditoria).registrar(1, "REGISTRAR_USUARIO", "caja2 · VENDEDOR");
    }

    /** RF01-CP02 (E2). Mensaje «El usuario ya existe, ingrese otro»; no se inserta nada. */
    @Test
    @DisplayName("RF01-CP02 Registrar otra cuenta con el usuario «vendedor».")
    void rf01_cp02_usuarioDuplicado() throws Exception {
        // ARRANGE
        when(usuarioDao.existeUsuario("vendedor")).thenReturn(true);
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> usuarioService().registrar("Otro vendedor", "vendedor", "VENDEDOR",
"Pescado2026", 1));
        // ASSERT
        assertEquals("El usuario ya existe, ingrese otro", e.getMessage());
        verify(usuarioDao, never()).insertar(anyString(), anyString(), anyString(),
anyString());
    }

    /** RF01-CP03 (E3). Mensaje «La contraseña debe tener al menos 8 caracteres». */
    @Test
    @DisplayName("RF01-CP03 Registrar el usuario con la clave «abc123» (6 caracteres).")
    void rf01_cp03_claveCorta() throws Exception {
        // ARRANGE
        when(usuarioDao.existeUsuario("caja2")).thenReturn(false);
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> usuarioService().registrar("Caja del puesto", "caja2", "VENDEDOR",
"abc123", 1));
        // ASSERT
        assertEquals("La contraseña debe tener al menos 8 caracteres", e.getMessage());
    }

    /** RF01-CP04 (E4). Mensaje «Elija un rol válido». */
    @Test
    @DisplayName("RF01-CP04 Registrar el usuario con el rol «GERENTE».")
    void rf01_cp04_rolInvalido() throws Exception {
        // ARRANGE (no se necesita preparar el DAO: la regla falla antes)
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> usuarioService().registrar("Caja del puesto", "caja2", "GERENTE",
"Pescado2026", 1));
        // ASSERT
        assertEquals("Elija un rol válido", e.getMessage());
        verify(auditoria, never()).registrar(any(), any(), any());
    }
```

```
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF01RegistrarUsuarioTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF01 Registrar usuario ✔
│     ├─ RF01-CP03 Registrar el usuario con la clave «abc123» (6 caracteres). ✘ Unexpected exception type thrown, expected: <NegocioException> but wa…
│     ├─ RF01-CP01 Registrar «Caja del puesto», usuario «caja2», rol VENDEDOR, clave «Pes... ✘ Pendiente de implementar
│     ├─ RF01-CP04 Registrar el usuario con el rol «GERENTE». ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedO…
│     └─ RF01-CP02 Registrar otra cuenta con el usuario «vendedor». ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <Unsupp…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF01RegistrarUsuarioTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF01 Registrar usuario ✔
│     ├─ RF01-CP03 Registrar el usuario con la clave «abc123» (6 caracteres). ✔
│     ├─ RF01-CP01 Registrar «Caja del puesto», usuario «caja2», rol VENDEDOR, clave «Pes... ✔
│     ├─ RF01-CP04 Registrar el usuario con el rol «GERENTE». ✔
│     └─ RF01-CP02 Registrar otra cuenta con el usuario «vendedor». ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 3. *Ejecución de RF01RegistrarUsuarioTest en rojo y en verde*

RF02 Iniciar sesión · RF02IniciarSesionTest

```
@DisplayName("RF02 Iniciar sesión")
class RF02IniciarSesionTest extends TddBase {

    /** RF02-CP01 (E1). Intentos en 0, registro INICIAR_SESION en la bitácora y redirección
a /inicio. */
    @Test
    @DisplayName("RF02-CP01 Ingresar con usuario y contraseña correctos.")
    void rf02_cp01_ingresoCorrectoReiniciaIntentos() throws Exception {
        // ARRANGE
        Usuario u = usuario(2, "vendedor", "VENDEDOR", "ACTIVO", 2);
        Authentication auth = mock(Authentication.class);
        when(auth.getPrincipal()).thenReturn(new UsuarioSesion(u));
        HttpServletRequest req = mock(HttpServletRequest.class);
        HttpServletResponse resp = mock(HttpServletResponse.class);
        when(req.getContextPath()).thenReturn("");
        // ACT
        manejadorAcceso().onAuthenticationSuccess(req, resp, auth);
        // ASSERT
        verify(usuarioDao).actualizarIntentos(2, 0, null);
        verify(auditoriaDao).registrar(2, "INICIAR_SESION", "Ingreso de vendedor");
        verify(resp).sendRedirect("/inicio");
    }

    /** RF02-CP02 (E2). Se guardan 2 intentos y se redirige a /login?error=credenciales
(«Usuario o contraseña incorrectos»). */
    @Test
    @DisplayName("RF02-CP02 Ingresar con la contraseña «clave-mala».")
    void rf02_cp02_claveIncorrectaSumaIntento() throws Exception {
        // ARRANGE
        when(usuarioDao.buscarPorUsuario("vendedor")).thenReturn(Optional.of(usuario(2,
"vendedor", "VENDEDOR", "ACTIVO", 1)));
        HttpServletRequest req = mock(HttpServletRequest.class);
        HttpServletResponse resp = mock(HttpServletResponse.class);
        when(req.getParameter("username")).thenReturn("vendedor");
        when(req.getContextPath()).thenReturn("");
        // ACT
        manejadorAcceso().onAuthenticationFailure(req, resp, new
BadCredentialsException("x"));
        // ASSERT
        verify(usuarioDao).actualizarIntentos(2, 2, null);
```

```
        verify(resp).sendRedirect("/login?error=credenciales");
    }

    /** RF02-CP03 (E3). El usuario queda bloqueado hasta dentro de 5 minutos y se registra
BLOQUEO_ACCESO. */
    @Test
    @DisplayName("RF02-CP03 Ingresar por quinta vez con una contraseña incorrecta.")
    void rf02_cp03_quintoIntentoBloquea() throws Exception {
        // ARRANGE
        when(usuarioDao.buscarPorUsuario("vendedor")).thenReturn(Optional.of(usuario(2,
"vendedor", "VENDEDOR", "ACTIVO", 4)));
        HttpServletRequest req = mock(HttpServletRequest.class);
        HttpServletResponse resp = mock(HttpServletResponse.class);
        when(req.getParameter("username")).thenReturn("vendedor");
        when(req.getContextPath()).thenReturn("");
        // ACT
        manejadorAcceso().onAuthenticationFailure(req, resp, new
BadCredentialsException("x"));
        // ASSERT
        ArgumentCaptor<LocalDateTime> hasta = ArgumentCaptor.forClass(LocalDateTime.class);
        verify(usuarioDao).actualizarIntentos(eq(2), eq(0), hasta.capture());
        long minutos = java.time.Duration.between(Fechas.ahora(),
hasta.getValue()).toMinutes();
        assertTrue(minutos >= 4 && minutos <= 5, "el bloqueo debe durar 5 minutos");
        verify(auditoriaDao).registrar(2, "BLOQUEO_ACCESO", "Bloqueado 5 min por intentos
fallidos");
    }

    /** RF02-CP04 (E4). Redirección a /login?error=inactivo («Su cuenta está desactivada»);
no se cuentan intentos. */
    @Test
    @DisplayName("RF02-CP04 Ingresar con el usuario desactivado.")
    void rf02_cp04_cuentaDesactivada() throws Exception {
        // ARRANGE
        HttpServletRequest req = mock(HttpServletRequest.class);
        HttpServletResponse resp = mock(HttpServletResponse.class);
        when(req.getParameter("username")).thenReturn("antiguo");
        when(req.getContextPath()).thenReturn("");
        // ACT
        manejadorAcceso().onAuthenticationFailure(req, resp, new
DisabledException("inactivo"));
        // ASSERT
        verify(resp).sendRedirect("/login?error=inactivo");
        verify(usuarioDao, never()).actualizarIntentos(anyInt(), anyInt(), any());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF02IniciarSesionTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF02 Iniciar sesión ✔
│     ├─ RF02-CP01 Ingresar con usuario y contraseña correctos. ✘ Pendiente de implementar
│     ├─ RF02-CP03 Ingresar por quinta vez con una contraseña incorrecta. ✘ Pendiente de implementar
│     ├─ RF02-CP02 Ingresar con la contraseña «clave-mala». ✘ Pendiente de implementar
│     └─ RF02-CP04 Ingresar con el usuario desactivado. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF02IniciarSesionTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF02 Iniciar sesión ✔
│     ├─ RF02-CP01 Ingresar con usuario y contraseña correctos. ✔
│     ├─ RF02-CP03 Ingresar por quinta vez con una contraseña incorrecta. ✔
│     ├─ RF02-CP02 Ingresar con la contraseña «clave-mala». ✔
│     └─ RF02-CP04 Ingresar con el usuario desactivado. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 4. *Ejecución de RF02IniciarSesionTest en rojo y en verde*

RF04 Cambiar contraseña · RF04CambiarContrasenaTest

```
@DisplayName("RF04 Cambiar contraseña")
class RF04CambiarContrasenaTest extends TddBase {

    /** RF04-CP01 (E1). Se guarda la nueva clave cifrada y se registra CAMBIAR_CLAVE. */
    @Test
    @DisplayName("RF04-CP01 Cambiar a «Corvina2027» (repetida igual).")
    void rf04_cp01_cambiaClave() throws Exception {
        // ARRANGE
        when(usuarioDao.buscarPorId(2)).thenReturn(Optional.of(usuarioConClave(2,
"vendedor", "Canomar2026")));
        // ACT
        usuarioService().cambiarClave(2, "Canomar2026", "Corvina2027", "Corvina2027");
        // ASSERT
        ArgumentCaptor<String> hash = ArgumentCaptor.forClass(String.class);
        verify(usuarioDao).actualizarClave(eq(2), hash.capture());
        assertTrue(encoder.matches("Corvina2027", hash.getValue()));
        verify(auditoria).registrar(2, "CAMBIAR_CLAVE", "vendedor");
    }

    /** RF04-CP02 (E2). Mensaje «Las contraseñas no coinciden». */
    @Test
    @DisplayName("RF04-CP02 Nueva «Corvina2027», repetir «Corvina2028».")
    void rf04_cp02_noCoinciden() throws Exception {
        // ARRANGE (la regla se valida antes de consultar el DAO)
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> usuarioService().cambiarClave(2, "Canomar2026", "Corvina2027",
"Corvina2028"));
        // ASSERT
        assertEquals("Las contraseñas no coinciden", e.getMessage());
        verify(usuarioDao, never()).actualizarClave(anyInt(), anyString());
    }

    /** RF04-CP03 (E3). Mensaje «La contraseña actual es incorrecta». */
    @Test
    @DisplayName("RF04-CP03 Clave actual «Canomar2025».")
    void rf04_cp03_actualIncorrecta() throws Exception {
        // ARRANGE
        when(usuarioDao.buscarPorId(2)).thenReturn(Optional.of(usuarioConClave(2,
"vendedor", "Canomar2026")));
```

```
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> usuarioService().cambiarClave(2, "Canomar2025", "Corvina2027",
"Corvina2027"));
        // ASSERT
        assertEquals("La contraseña actual es incorrecta", e.getMessage());
    }

    /** RF04-CP04 (E4). Mensaje «La contraseña debe tener al menos 8 caracteres». */
    @Test
    @DisplayName("RF04-CP04 Nueva clave «pez» repetida igual.")
    void rf04_cp04_nuevaCorta() throws Exception {
        // ARRANGE
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> usuarioService().cambiarClave(2, "Canomar2026", "pez", "pez"));
        // ASSERT
        assertEquals("La contraseña debe tener al menos 8 caracteres", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF04CambiarContrasenaTest
FASE ROJA: clases Service sin implementar

.
├─ JUnit Jupiter ✔
│  └─ RF04 Cambiar contraseña ✔
│     ├─ RF04-CP02 Nueva «Corvina2027», repetir «Corvina2028». ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <Unsupported…
│     ├─ RF04-CP04 Nueva clave «pez» repetida igual. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationE…
│     ├─ RF04-CP01 Cambiar a «Corvina2027» (repetida igual). ✘ Pendiente de implementar
│     └─ RF04-CP03 Clave actual «Canomar2025». ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExcepti…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF04CambiarContrasenaTest
FASE VERDE: código implementado

.
├─ JUnit Jupiter ✔
│  └─ RF04 Cambiar contraseña ✔
│     ├─ RF04-CP02 Nueva «Corvina2027», repetir «Corvina2028». ✔
│     ├─ RF04-CP04 Nueva clave «pez» repetida igual. ✔
│     ├─ RF04-CP01 Cambiar a «Corvina2027» (repetida igual). ✔
│     └─ RF04-CP03 Clave actual «Canomar2025». ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 5. *Ejecución de RF04CambiarContrasenaTest en rojo y en verde*

RF05 Desactivar usuario · RF05DesactivarUsuarioTest

```
@DisplayName("RF05 Desactivar usuario")
class RF05DesactivarUsuarioTest extends TddBase {

    /** RF05-CP01 (E1). Estado del usuario 2 = INACTIVO; bitácora DESACTIVAR_USUARIO. */
    @Test
    @DisplayName("RF05-CP01 El administrador 1 desactiva al usuario 2.")
    void rf05_cp01_desactivaUsuario() throws Exception {
        // ARRANGE
        when(usuarioDao.buscarPorId(2)).thenReturn(Optional.of(usuario(2, "vendedor",
"VENDEDOR", "ACTIVO", 0)));
        // ACT
        usuarioService().desactivar(2, 1);
        // ASSERT
        verify(usuarioDao).cambiarEstado(2, "INACTIVO");
        verify(auditoria).registrar(1, "DESACTIVAR_USUARIO", "vendedor");
    }

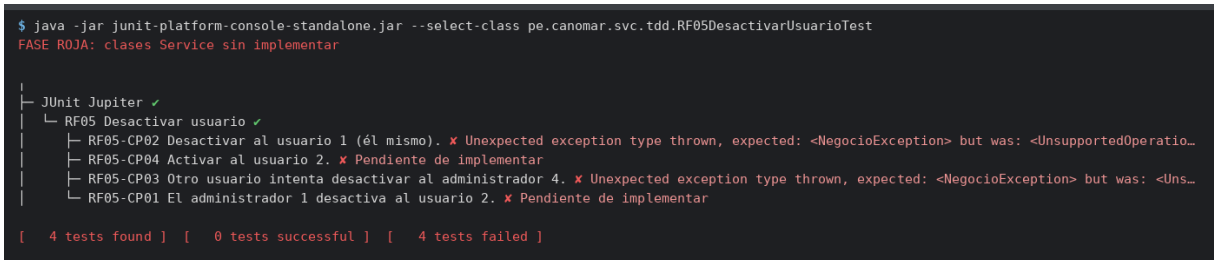
    /** RF05-CP02 (E2). Mensaje «No puede desactivar su propia cuenta». */
    @Test
    @DisplayName("RF05-CP02 Desactivar al usuario 1 (él mismo).")
    void rf05_cp02_propiaCuenta() throws Exception {
```

```
        // ARRANGE
        when(usuarioDao.buscarPorId(1)).thenReturn(Optional.of(usuario(1, "cuentas",
"ADMINISTRADOR", "ACTIVO", 0)));
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
usuarioService().desactivar(1, 1));
        // ASSERT
        assertEquals("No puede desactivar su propia cuenta", e.getMessage());
        verify(usuarioDao, never()).cambiarEstado(anyInt(), anyString());
    }

    /** RF05-CP03 (E3). Mensaje «No puede desactivar al único administrador». */
    @Test
    @DisplayName("RF05-CP03 Otro usuario intenta desactivar al administrador 4.")
    void rf05_cp03_unicoAdministrador() throws Exception {
        // ARRANGE
        when(usuarioDao.buscarPorId(4)).thenReturn(Optional.of(usuario(4, "dueña",
"ADMINISTRADOR", "ACTIVO", 0)));
        when(usuarioDao.contarAdministradoresActivos()).thenReturn(1);
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
usuarioService().desactivar(4, 1));
        // ASSERT
        assertEquals("No puede desactivar al único administrador", e.getMessage());
    }

    /** RF05-CP04 (E4). Estado del usuario 2 = ACTIVO; bitácora ACTIVAR_USUARIO. */
    @Test
    @DisplayName("RF05-CP04 Activar al usuario 2.")
    void rf05_cp04_activaUsuario() throws Exception {
        // ARRANGE
        when(usuarioDao.buscarPorId(2)).thenReturn(Optional.of(usuario(2, "vendedor",
"VENDEDOR", "INACTIVO", 0)));
        // ACT
        usuarioService().activar(2, 1);
        // ASSERT
        verify(usuarioDao).cambiarEstado(2, "ACTIVO");
        verify(auditoria).registrar(1, "ACTIVAR_USUARIO", "vendedor");
    }
}
```

Antes de programar (rojo):



Con el código implementado (verde):

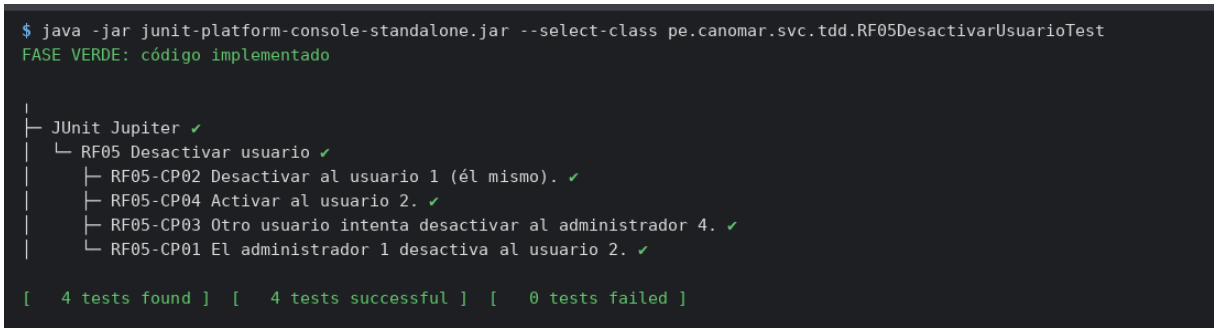


Figura 6. *Ejecución de RF05DesactivarUsuarioTest en rojo y en verde*

RF06 Registrar producto · RF06RegistrarProductoTest

```
@DisplayName("RF06 Registrar producto")
```

```
class RF06RegistrarProductoTest extends TddBase {

    /** RF06-CP01 (E1). Se inserta «Caballa» ACTIVO (nombre con mayúscula inicial). */
    @Test
    @DisplayName("RF06-CP01 Registrar «caballa», nombre corto «Caballa», categoría
PESCADO.")
    void rf06_cp01_registraProducto() throws Exception {
        // ARRANGE
        when(productoDao.existeNombre("Caballa", null)).thenReturn(false);
        when(productoDao.insertar(any())).thenReturn(8);
        // ACT
        int id = productoService().registrar("caballa", "Caballa", "PESCADO", 1);
        // ASSERT
        assertEquals(8, id);
        verify(productoDao).insertar(new Producto(null, "Caballa", "Caballa", "PESCADO",
"ACTIVO"));
    }

    /** RF06-CP02 (E2). El nombre corto queda «Concha de abanico» (máx. 20 caracteres). */
    @Test
    @DisplayName("RF06-CP02 Registrar sin nombre corto.")
    void rf06_cp02_nombreCortoPorDefecto() throws Exception {
        // ARRANGE
        when(productoDao.existeNombre("Concha de abanico", null)).thenReturn(false);
        // ACT
        productoService().registrar("concha de abanico", "", "MOLUSCO", 1);
        // ASSERT
        ArgumentCaptor<Producto> p = ArgumentCaptor.forClass(Producto.class);
        verify(productoDao).insertar(p.capture());
        assertEquals("Concha de abanico", p.getValue().nombreCorto());
    }

    /** RF06-CP03 (E3). Mensaje «El producto ya existe». */
    @Test
    @DisplayName("RF06-CP03 Registrar «Bonito».")
    void rf06_cp03_productoDuplicado() throws Exception {
        // ARRANGE
        when(productoDao.existeNombre("Bonito", null)).thenReturn(true);
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
productoService().registrar("Bonito", null, "PESCADO", 1));
        // ASSERT
        assertEquals("El producto ya existe", e.getMessage());
    }

    /** RF06-CP04 (E4). Mensaje «Ingrese el nombre del producto». */
    @Test
    @DisplayName("RF06-CP04 Registrar con el nombre en blanco.")
    void rf06_cp04_faltaNombre() throws Exception {
        // ARRANGE
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
productoService().registrar("   ", null, "PESCADO", 1));
        // ASSERT
        assertEquals("Ingrese el nombre del producto", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF06RegistrarProductoTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF06 Registrar producto ✔
│     ├─ RF06-CP02 Registrar sin nombre corto. ✘ Pendiente de implementar
│     ├─ RF06-CP03 Registrar «Bonito». ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>
│     ├─ RF06-CP01 Registrar «caballa», nombre corto «Caballa», categoría PESCADO. ✘ Pendiente de implementar
│     └─ RF06-CP04 Registrar con el nombre en blanco. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperation…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF06RegistrarProductoTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF06 Registrar producto ✔
│     ├─ RF06-CP02 Registrar sin nombre corto. ✔
│     ├─ RF06-CP03 Registrar «Bonito». ✔
│     ├─ RF06-CP01 Registrar «caballa», nombre corto «Caballa», categoría PESCADO. ✔
│     └─ RF06-CP04 Registrar con el nombre en blanco. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 7. *Ejecución de RF06RegistrarProductoTest en rojo y en verde*

RF07 Editar producto · RF07EditarProductoTest

```
@DisplayName("RF07 Editar producto")
class RF07EditarProductoTest extends TddBase {

    /** RF07-CP01 (E1). Se actualiza el producto 3 y se registra EDITAR_PRODUCTO. */
    @Test
    @DisplayName("RF07-CP01 Cambiar el nombre corto a «Pota fresca».")
    void rf07_cp01_editaProducto() throws Exception {
        // ARRANGE
        when(productoDao.buscar(3)).thenReturn(Optional.of(new Producto(3, "Pota", "Pota",
"MOLUSCO", "ACTIVO")));
        // ACT
        productoService().editar(3, "Pota", "Pota fresca", "MOLUSCO", "ACTIVO", 1);
        // ASSERT
        verify(productoDao).actualizar(new Producto(3, "Pota", "Pota fresca", "MOLUSCO",
"ACTIVO"));
        verify(auditoria).registrar(1, "EDITAR_PRODUCTO", "Pota · ACTIVO");
    }

    /** RF07-CP02 (E2). El producto queda INACTIVO. */
    @Test
    @DisplayName("RF07-CP02 Cambiar su estado a INACTIVO.")
    void rf07_cp02_inactivaSinSaldo() throws Exception {
        // ARRANGE
        when(productoDao.buscar(6)).thenReturn(Optional.of(new Producto(6, "Lenguado",
"Lenguado", "PESCADO", "ACTIVO")));
        when(loteDao.kilosDisponibles(6)).thenReturn(d("0.00"));
        // ACT
        productoService().editar(6, "Lenguado", "Lenguado", "PESCADO", "INACTIVO", 1);
        // ASSERT
        verify(productoDao).actualizar(new Producto(6, "Lenguado", "Lenguado", "PESCADO",
"INACTIVO"));
    }

    /** RF07-CP03 (E3). Mensaje «No se puede inactivar: tiene 8.00 kg en saldo». */
    @Test
    @DisplayName("RF07-CP03 Cambiar su estado a INACTIVO.")
    void rf07_cp03_noInactivaConSaldo() throws Exception {
        // ARRANGE
        when(productoDao.buscar(1)).thenReturn(Optional.of(producto(1, "Bonito")));
        when(loteDao.kilosDisponibles(1)).thenReturn(d("8.00"));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> productoService().editar(1, "Bonito", "Bonito", "PESCADO", "INACTIVO",
```

```
1));
        // ASSERT
        assertEquals("No se puede inactivar: tiene 8.00 kg en saldo", e.getMessage());
        verify(productoDao, never()).actualizar(any());
    }

    /** RF07-CP04 (E4). Mensaje «Ya existe otro producto con ese nombre». */
    @Test
    @DisplayName("RF07-CP04 Renombrar el producto 5 como «Pota».")
    void rf07_cp04_nombreDuplicado() throws Exception {
        // ARRANGE
        when(productoDao.buscar(5)).thenReturn(Optional.of(producto(5, "Jurel")));
        when(productoDao.existeNombre("Pota", 5)).thenReturn(true);
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> productoService().editar(5, "Pota", "Pota", "PESCADO", "ACTIVO", 1));
        // ASSERT
        assertEquals("Ya existe otro producto con ese nombre", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF07EditarProductoTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF07 Editar producto ✔
│     ├─ RF07-CP01 Cambiar el nombre corto a «Pota fresca». ✘ Pendiente de implementar
│     ├─ RF07-CP04 Renombrar el producto 5 como «Pota». ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperati…
│     ├─ RF07-CP02 Cambiar su estado a INACTIVO. ✘ Pendiente de implementar
│     └─ RF07-CP03 Cambiar su estado a INACTIVO. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExcep…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF07EditarProductoTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF07 Editar producto ✔
│     ├─ RF07-CP01 Cambiar el nombre corto a «Pota fresca». ✔
│     ├─ RF07-CP04 Renombrar el producto 5 como «Pota». ✔
│     ├─ RF07-CP02 Cambiar su estado a INACTIVO. ✔
│     └─ RF07-CP03 Cambiar su estado a INACTIVO. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 8. *Ejecución de RF07EditarProductoTest en rojo y en verde*

RF08 Registrar cliente · RF08RegistrarClienteTest

```
@DisplayName("RF08 Registrar cliente")
class RF08RegistrarClienteTest extends TddBase {

    /** RF08-CP01 (E1). Se inserta el cliente ACTIVO y la bitácora dice «límite S/ 1000.00».
*/
    @Test
    @DisplayName("RF08-CP01 Registrar «Cevichería La Ola», RUC 20605555551, límite S/ 1,000,
plazo 15.")
    void rf08_cp01_registraCliente() throws Exception {
        // ARRANGE
        when(clienteDao.existeRuc("20605555551", null)).thenReturn(false);
        when(clienteDao.insertar(any())).thenReturn(4);
        // ACT
        int id = clienteService().registrar("Cevichería La Ola", "20605555551", "988777666",
d("1000"), 15, 1);
        // ASSERT
        assertEquals(4, id);
        verify(clienteDao).insertar(new Cliente(null, "Cevichería La Ola", "20605555551",
"988777666", d("1000.00"), 15, "ACTIVO"));
        verify(auditoria).registrar(1, "REGISTRAR_CLIENTE", "Cevichería La Ola · límite S/
1000.00");
```

```
    }

    /** RF08-CP02 (E2). Se inserta con RUC vacío (null). */
    @Test
    @DisplayName("RF08-CP02 Registrar «Juguería Rosita» sin RUC, límite S/ 300.")
    void rf08_cp02_sinRuc() throws Exception {
        // ARRANGE
        // ACT
        clienteService().registrar("Juguería Rosita", "", null, d("300"), 7, 1);
        // ASSERT
        ArgumentCaptor<Cliente> c = ArgumentCaptor.forClass(Cliente.class);
        verify(clienteDao).insertar(c.capture());
        assertNull(c.getValue().ruc());
    }

    /** RF08-CP03 (E3). Mensaje «El RUC debe tener 11 dígitos». */
    @Test
    @DisplayName("RF08-CP03 Registrar con RUC «2060123».")
    void rf08_cp03_rucInvalido() throws Exception {
        // ARRANGE
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> clienteService().registrar("Cevichería La Ola", "2060123", null,
d("1000"), 15, 1));
        // ASSERT
        assertEquals("El RUC debe tener 11 dígitos", e.getMessage());
    }

    /** RF08-CP04 (E4). Mensaje «El cliente ya está registrado con ese RUC». */
    @Test
    @DisplayName("RF08-CP04 Registrar otro cliente con ese RUC.")
    void rf08_cp04_rucDuplicado() throws Exception {
        // ARRANGE
        when(clienteDao.existeRuc("20601234561", null)).thenReturn(true);
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> clienteService().registrar("El Muelle 2", "20601234561", null,
d("500"), 7, 1));
        // ASSERT
        assertEquals("El cliente ya está registrado con ese RUC", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF08RegistrarClienteTest
FASE ROJA: clases Service sin implementar

.
├─ JUnit Jupiter ✔
│  └─ RF08 Registrar cliente ✔
│     ├─ RF08-CP02 Registrar «Juguería Rosita» sin RUC, límite S/ 300. ✘ Pendiente de implementar
│     ├─ RF08-CP01 Registrar «Cevichería La Ola», RUC 20605555551, límite S/ 1,000, plazo... ✘ Pendiente de implementar
│     ├─ RF08-CP04 Registrar otro cliente con ese RUC. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperatio…
│     └─ RF08-CP03 Registrar con RUC «2060123». ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExcept…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF08RegistrarClienteTest
FASE VERDE: código implementado

.
├─ JUnit Jupiter ✔
│  └─ RF08 Registrar cliente ✔
│     ├─ RF08-CP02 Registrar «Juguería Rosita» sin RUC, límite S/ 300. ✔
│     ├─ RF08-CP01 Registrar «Cevichería La Ola», RUC 20605555551, límite S/ 1,000, plazo... ✔
│     ├─ RF08-CP04 Registrar otro cliente con ese RUC. ✔
│     └─ RF08-CP03 Registrar con RUC «2060123». ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 9. *Ejecución de RF08RegistrarClienteTest en rojo y en verde*

RF09 Editar cliente · RF09EditarClienteTest

```
@DisplayName("RF09 Editar cliente")
class RF09EditarClienteTest extends TddBase {

    /** RF09-CP01 (E1). Bitácora CAMBIO_LIMITE «S/ 1500.00 → S/ 2000.00»; sin advertencia.
*/
    @Test
    @DisplayName("RF09-CP01 Subir el límite a S/ 2,000.")
    void rf09_cp01_cambiaLimite() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(1)).thenReturn(Optional.of(cliente(1, "Cevichería El Muelle",
"1500.00", 15)));
        when(credito.saldo(1)).thenReturn(saldoCliente(1, "228.00", "1500.00", "0.00"));
        // ACT
        String aviso = clienteService().editar(1, "Cevichería El Muelle", "20601234561",
"987654321", d("2000"), 15, "ACTIVO", 1);
        // ASSERT
        assertNull(aviso);
        verify(auditoria).registrar(1, "CAMBIO_LIMITE", "Cevichería El Muelle: S/ 1500.00 →
S/ 2000.00");
    }

    /** RF09-CP02 (E2). Aviso «El saldo actual (S/ 1372.00) supera el nuevo límite: no podrá
comprar al crédito hasta abonar». */
    @Test
    @DisplayName("RF09-CP02 Bajar su límite a S/ 1,000.")
    void rf09_cp02_limiteMenorQueSaldo() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(2)).thenReturn(Optional.of(cliente(2, "Restaurante Sabor
Norteño", "1500.00", 7)));
        when(credito.saldo(2)).thenReturn(saldoCliente(2, "1372.00", "1000.00", "1372.00"));
        // ACT
        String aviso = clienteService().editar(2, "Restaurante Sabor Norteño",
"20601234571", null, d("1000"), 7, "ACTIVO", 1);
        // ASSERT
        assertEquals("El saldo actual (S/ 1372.00) supera el nuevo límite: no podrá comprar
al crédito hasta abonar", aviso);
    }

    /** RF09-CP03 (E3). Mensaje «El plazo debe estar entre 0 y 90 días». */
    @Test
    @DisplayName("RF09-CP03 Cambiar el plazo a 120 días.")
    void rf09_cp03_plazoFueraDeRango() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(1)).thenReturn(Optional.of(cliente(1, "Cevichería El Muelle",
"1500.00", 15)));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> clienteService().editar(1, "Cevichería El Muelle", null, null,
d("1500"), 120, "ACTIVO", 1));
        // ASSERT
        assertEquals("El plazo debe estar entre 0 y 90 días", e.getMessage());
    }

    /** RF09-CP04 (E4). Mensaje «El límite de crédito no puede ser negativo». */
    @Test
    @DisplayName("RF09-CP04 Cambiar el límite a S/ -50.")
    void rf09_cp04_limiteNegativo() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(1)).thenReturn(Optional.of(cliente(1, "Cevichería El Muelle",
"1500.00", 15)));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> clienteService().editar(1, "Cevichería El Muelle", null, null, d("-
50"), 15, "ACTIVO", 1));
        // ASSERT
        assertEquals("El límite de crédito no puede ser negativo", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF09EditarClienteTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF09 Editar cliente ✔
│     ├─ RF09-CP02 Bajar su límite a S/ 1,000. ✘ Pendiente de implementar
│     ├─ RF09-CP03 Cambiar el plazo a 120 días. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExcept…
│     ├─ RF09-CP01 Subir el límite a S/ 2,000. ✘ Pendiente de implementar
│     └─ RF09-CP04 Cambiar el límite a S/ -50. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExcepti…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF09EditarClienteTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF09 Editar cliente ✔
│     ├─ RF09-CP02 Bajar su límite a S/ 1,000. ✔
│     ├─ RF09-CP03 Cambiar el plazo a 120 días. ✔
│     ├─ RF09-CP01 Subir el límite a S/ 2,000. ✔
│     └─ RF09-CP04 Cambiar el límite a S/ -50. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 10*. Ejecución de RF09EditarClienteTest en rojo y en verde*

RF10 Registrar proveedor · RF10RegistrarProveedorTest

```
@DisplayName("RF10 Registrar proveedor")
class RF10RegistrarProveedorTest extends TddBase {

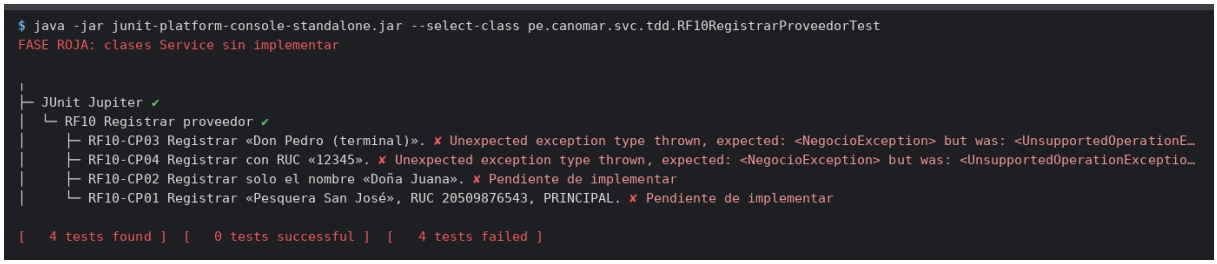
    /** RF10-CP01 (E1). Se inserta el proveedor PRINCIPAL. */
    @Test
    @DisplayName("RF10-CP01 Registrar «Pesquera San José», RUC 20509876543, PRINCIPAL.")
    void rf10_cp01_registraProveedor() throws Exception {
        // ARRANGE
        when(proveedorDao.insertar(any())).thenReturn(3);
        // ACT
        int id = compraService().registrarProveedor("Pesquera San José", "20509876543",
"PRINCIPAL", "999000111", 1);
        // ASSERT
        assertEquals(3, id);
        verify(proveedorDao).insertar(new Proveedor(null, "Pesquera San José",
"20509876543", "PRINCIPAL", "999000111"));
    }

    /** RF10-CP02 (E2). Se inserta como OCASIONAL, sin RUC ni teléfono. */
    @Test
    @DisplayName("RF10-CP02 Registrar solo el nombre «Doña Juana».")
    void rf10_cp02_ocasionalSoloNombre() throws Exception {
        // ARRANGE
        // ACT
        compraService().registrarProveedor("Doña Juana", null, null, null, 1);
        // ASSERT
        verify(proveedorDao).insertar(new Proveedor(null, "Doña Juana", null, "OCASIONAL",
null));
    }

    /** RF10-CP03 (E3). Mensaje «El proveedor ya está registrado». */
    @Test
    @DisplayName("RF10-CP03 Registrar «Don Pedro (terminal)».")
    void rf10_cp03_proveedorDuplicado() throws Exception {
        // ARRANGE
        when(proveedorDao.existeNombre("Don Pedro (terminal)")).thenReturn(true);
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> compraService().registrarProveedor("Don Pedro (terminal)", null,
"OCASIONAL", null, 1));
        // ASSERT
        assertEquals("El proveedor ya está registrado", e.getMessage());
    }
```

```
    /** RF10-CP04 (E4). Mensaje «El RUC debe tener 11 dígitos». */
    @Test
    @DisplayName("RF10-CP04 Registrar con RUC «12345».")
    void rf10_cp04_rucInvalido() throws Exception {
        // ARRANGE
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> compraService().registrarProveedor("Pesquera San José", "12345",
"PRINCIPAL", null, 1));
        // ASSERT
        assertEquals("El RUC debe tener 11 dígitos", e.getMessage());
    }
}
```

Antes de programar (rojo):



Con el código implementado (verde):

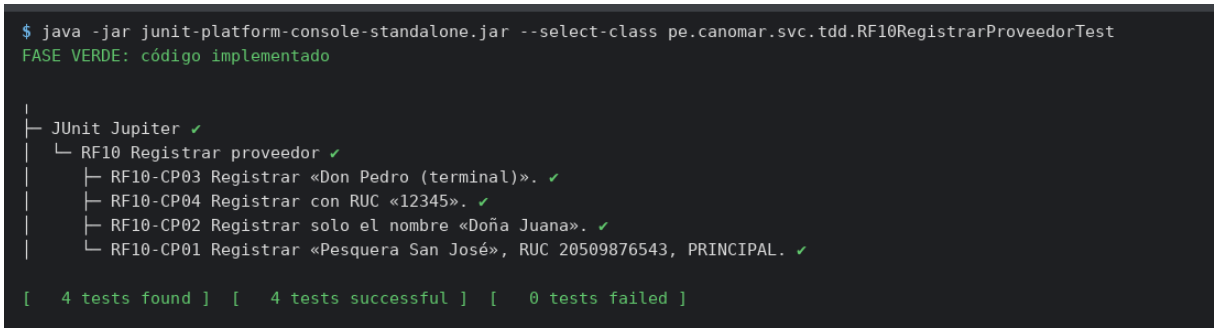


Figura 11*. Ejecución de RF10RegistrarProveedorTest en rojo y en verde*

RF11 Registrar compra del día · RF11RegistrarCompraDelDiaTest

```
@DisplayName("RF11 Registrar compra del día")
class RF11RegistrarCompraDelDiaTest extends TddBase {

    /** RF11-CP01 (E1). Se crea el lote de 40.00 kg a S/ 26.00 por kg y se recalcula el
costo de la corvina. */
    @Test
    @DisplayName("RF11-CP01 Comprar 40 kg de corvina por S/ 1,040 al contado.")
    void rf11_cp01_compraCreaLote() throws Exception {
        // ARRANGE
        when(proveedorDao.buscar(1)).thenReturn(Optional.of(new Proveedor(1, "Distribuidora
Pesquera Lurín", null, "PRINCIPAL", null)));
        when(productoDao.buscar(2)).thenReturn(Optional.of(producto(2, "Corvina")));
        when(compraDao.insertarCompra(anyInt(), anyInt(), any(),
anyString())).thenReturn(9);
        when(compraDao.insertarDetalle(anyInt(), any())).thenReturn(15);
        // ACT
        compraService().registrarCompra(1, "CONTADO", List.of(new ItemCompra(2, d("40"),
d("1040"))), 2);
        // ASSERT
        verify(loteDao).insertar(2, 15, Fechas.hoy(), d("40.00"), d("26.00"));
        verify(precioService).recalcularCosto(2);
    }

    /** RF11-CP02 (E2). La compra se guarda con tipo de pago CREDITO. */
    @Test
    @DisplayName("RF11-CP02 Comprar 20 kg de jurel por S/ 100 a crédito.")
    void rf11_cp02_compraCredito() throws Exception {
```

```
        // ARRANGE
        when(proveedorDao.buscar(2)).thenReturn(Optional.of(new Proveedor(2, "Don Pedro
(terminal)", null, "OCASIONAL", null)));
        when(productoDao.buscar(5)).thenReturn(Optional.of(producto(5, "Jurel")));
        // ACT
        compraService().registrarCompra(2, "CREDITO", List.of(new ItemCompra(5, d("20"),
d("100"))), 2);
        // ASSERT
        verify(compraDao).insertarCompra(2, 2, Fechas.hoy(), "CREDITO");
    }

    /** RF11-CP03 (E3). Mensaje «Los kilos y el costo deben ser mayores que cero»; no se
crea la compra. */
    @Test
    @DisplayName("RF11-CP03 Comprar 0 kg de corvina por S/ 100.")
    void rf11_cp03_kilosCero() throws Exception {
        // ARRANGE
        when(proveedorDao.buscar(1)).thenReturn(Optional.of(new Proveedor(1, "Distribuidora
Pesquera Lurín", null, "PRINCIPAL", null)));
        when(productoDao.buscar(2)).thenReturn(Optional.of(producto(2, "Corvina")));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> compraService().registrarCompra(1, "CONTADO", List.of(new
ItemCompra(2, d("0"), d("100"))), 2));
        // ASSERT
        assertEquals("Los kilos y el costo deben ser mayores que cero", e.getMessage());
        verify(compraDao, never()).insertarCompra(anyInt(), anyInt(), any(), anyString());
    }

    /** RF11-CP04 (E4). Mensaje «Lenguado está inactivo». */
    @Test
    @DisplayName("RF11-CP04 Comprar 10 kg de lenguado.")
    void rf11_cp04_productoInactivo() throws Exception {
        // ARRANGE
        when(proveedorDao.buscar(1)).thenReturn(Optional.of(new Proveedor(1, "Distribuidora
Pesquera Lurín", null, "PRINCIPAL", null)));
        when(productoDao.buscar(6)).thenReturn(Optional.of(new Producto(6, "Lenguado",
"Lenguado", "PESCADO", "INACTIVO")));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> compraService().registrarCompra(1, "CONTADO", List.of(new
ItemCompra(6, d("10"), d("400"))), 2));
        // ASSERT
        assertEquals("Lenguado está inactivo", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF11RegistrarCompraDelDiaTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF11 Registrar compra del día ✔
│     ├─ RF11-CP02 Comprar 20 kg de jurel por S/ 100 a crédito. ✘ Pendiente de implementar
│     ├─ RF11-CP01 Comprar 40 kg de corvina por S/ 1,040 al contado. ✘ Pendiente de implementar
│     ├─ RF11-CP03 Comprar 0 kg de corvina por S/ 100. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperatio…
│     └─ RF11-CP04 Comprar 10 kg de lenguado. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExceptio…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF11RegistrarCompraDelDiaTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF11 Registrar compra del día ✔
│     ├─ RF11-CP02 Comprar 20 kg de jurel por S/ 100 a crédito. ✔
│     ├─ RF11-CP01 Comprar 40 kg de corvina por S/ 1,040 al contado. ✔
│     ├─ RF11-CP03 Comprar 0 kg de corvina por S/ 100. ✔
│     └─ RF11-CP04 Comprar 10 kg de lenguado. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 12. *Ejecución de RF11RegistrarCompraDelDiaTest en rojo y en verde*

RF12 Calcular costo promedio por kilo · RF12CalcularCostoPromedioPorKiloTest

```
@DisplayName("RF12 Calcular costo promedio por kilo")
class RF12CalcularCostoPromedioPorKiloTest extends TddBase {

    /** RF12-CP01 (E1). Costo promedio = (8×6.50 + 52×7.00) ÷ 60 = S/ 6.93. */
    @Test
    @DisplayName("RF12-CP01 Recalcular el costo del bonito.")
    void rf12_cp01_promedioPonderado() throws Exception {
        // ARRANGE
        when(loteDao.disponiblesFifo(1)).thenReturn(List.of(new LoteKg(1, d("8.00"),
d("6.50")), new LoteKg(7, d("52.00"), d("7.00"))));
        when(precioDao.precioDe(eq(1), any())).thenReturn(Optional.of(d("9.00")));
        // ACT
        BigDecimal costo = precioServiceReal().recalcularCosto(1);
        // ASSERT
        assertEquals(d("6.93"), costo);
        verify(precioDao).guardarCosto(1, Fechas.hoy(), d("6.93"), d("9.00"));
    }

    /** RF12-CP02 (E2). Costo promedio = S/ 26.00. */
    @Test
    @DisplayName("RF12-CP02 Recalcular el costo de la corvina.")
    void rf12_cp02_sinSaldoAnterior() throws Exception {
        // ARRANGE
        when(loteDao.disponiblesFifo(2)).thenReturn(List.of(new LoteKg(9, d("40.00"),
d("26.00"))));
        when(precioDao.precioDe(eq(2), any())).thenReturn(Optional.of(d("32.00")));
        // ACT
        BigDecimal costo = precioServiceReal().recalcularCosto(2);
        // ASSERT
        assertEquals(d("26.00"), costo);
    }

    /** RF12-CP03 (E3). Costo = 0.00 y no se guarda en precio_dia. */
    @Test
    @DisplayName("RF12-CP03 Recalcular el costo del perico.")
    void rf12_cp03_sinLotes() throws Exception {
        // ARRANGE
        when(loteDao.disponiblesFifo(7)).thenReturn(List.of());
        // ACT
        BigDecimal costo = precioServiceReal().recalcularCosto(7);
        // ASSERT
```

```
        assertEquals(d("0.00"), costo);
        verify(precioDao, never()).guardarCosto(anyInt(), any(), any(), any());
    }

    /** RF12-CP04 (E4). Se guarda costo S/ 5.50 con precio sugerido S/ 8.00 (el de ayer). */
    @Test
    @DisplayName("RF12-CP04 Recalcular el costo de la pota.")
    void rf12_cp04_precioDeAyerSugerido() throws Exception {
        // ARRANGE
        when(loteDao.disponiblesFifo(3)).thenReturn(List.of(new LoteKg(4, d("30.00"),
d("5.50"))));
        when(precioDao.precioDe(eq(3), any())).thenReturn(Optional.empty());
        when(precioDao.ultimoPrecio(eq(3), any())).thenReturn(Optional.of(d("8.00")));
        // ACT
        precioServiceReal().recalcularCosto(3);
        // ASSERT
        verify(precioDao).guardarCosto(3, Fechas.hoy(), d("5.50"), d("8.00"));
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF12CalcularCostoPromedioPorKiloTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF12 Calcular costo promedio por kilo ✔
│     ├─ RF12-CP03 Recalcular el costo del perico. ✘ Pendiente de implementar
│     ├─ RF12-CP04 Recalcular el costo de la pota. ✘ Pendiente de implementar
│     ├─ RF12-CP02 Recalcular el costo de la corvina. ✘ Pendiente de implementar
│     └─ RF12-CP01 Recalcular el costo del bonito. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF12CalcularCostoPromedioPorKiloTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF12 Calcular costo promedio por kilo ✔
│     ├─ RF12-CP03 Recalcular el costo del perico. ✔
│     ├─ RF12-CP04 Recalcular el costo de la pota. ✔
│     ├─ RF12-CP02 Recalcular el costo de la corvina. ✔
│     └─ RF12-CP01 Recalcular el costo del bonito. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 13. *Ejecución de RF12CalcularCostoPromedioPorKiloTest en rojo y en verde*

RF13 Registrar pago a proveedor · RF13RegistrarPagoAProveedorTest

```
@DisplayName("RF13 Registrar pago a proveedor")
class RF13RegistrarPagoAProveedorTest extends TddBase {

    /** RF13-CP01 (E1). Se registra el pago y la deuda queda en S/ 0.00. */
    @Test
    @DisplayName("RF13-CP01 Pagar S/ 390 por transferencia.")
    void rf13_cp01_pagoTotal() throws Exception {
        // ARRANGE
        when(compraDao.deudas()).thenReturn(List.of(deudaProveedor(1, "Distribuidora
Pesquera Lurín", "390.00", "0.00")));
        // ACT
        BigDecimal queda = compraService().registrarPago(1, d("390"), "TRANSFERENCIA", 1);
        // ASSERT
        assertEquals(d("0.00"), queda);
        verify(compraDao).insertarPago(1, 1, Fechas.hoy(), d("390.00"), "TRANSFERENCIA");
    }

    /** RF13-CP02 (E2). La deuda queda en S/ 60.00. */
    @Test
    @DisplayName("RF13-CP02 Pagar S/ 40 en efectivo.")
```

```
    void rf13_cp02_pagoParcial() throws Exception {
        // ARRANGE
        when(compraDao.deudas()).thenReturn(List.of(deudaProveedor(2, "Don Pedro
(terminal)", "100.00", "0.00")));
        // ACT
        BigDecimal queda = compraService().registrarPago(2, d("40"), "EFECTIVO", 1);
        // ASSERT
        assertEquals(d("60.00"), queda);
    }

    /** RF13-CP03 (E3). Mensaje «El pago supera la deuda (S/ 100.00)». */
    @Test
    @DisplayName("RF13-CP03 Pagar S/ 150.")
    void rf13_cp03_pagoSuperaDeuda() throws Exception {
        // ARRANGE
        when(compraDao.deudas()).thenReturn(List.of(deudaProveedor(2, "Don Pedro
(terminal)", "100.00", "0.00")));
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
compraService().registrarPago(2, d("150"), "EFECTIVO", 1));
        // ASSERT
        assertEquals("El pago supera la deuda (S/ 100.00)", e.getMessage());
    }

    /** RF13-CP04 (E4). Mensaje «Ingrese un monto mayor que cero». */
    @Test
    @DisplayName("RF13-CP04 Pagar S/ 0.")
    void rf13_cp04_montoCero() throws Exception {
        // ARRANGE
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
compraService().registrarPago(2, d("0"), "EFECTIVO", 1));
        // ASSERT
        assertEquals("Ingrese un monto mayor que cero", e.getMessage());
        verify(compraDao, never()).insertarPago(anyInt(), anyInt(), any(), any(),
anyString());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF13RegistrarPagoAProveedorTest
FASE ROJA: clases Service sin implementar

.
├─ JUnit Jupiter ✔
│  └─ RF13 Registrar pago a proveedor ✔
│     ├─ RF13-CP01 Pagar S/ 390 por transferencia. ✘ Pendiente de implementar
│     ├─ RF13-CP02 Pagar S/ 40 en efectivo. ✘ Pendiente de implementar
│     ├─ RF13-CP03 Pagar S/ 150. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>
│     └─ RF13-CP04 Pagar S/ 0. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF13RegistrarPagoAProveedorTest
FASE VERDE: código implementado

.
├─ JUnit Jupiter ✔
│  └─ RF13 Registrar pago a proveedor ✔
│     ├─ RF13-CP01 Pagar S/ 390 por transferencia. ✔
│     ├─ RF13-CP02 Pagar S/ 40 en efectivo. ✔
│     ├─ RF13-CP03 Pagar S/ 150. ✔
│     └─ RF13-CP04 Pagar S/ 0. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 14. *Ejecución de RF13RegistrarPagoAProveedorTest en rojo y en verde*

RF15 Registrar precios de venta del día · RF15RegistrarPreciosDeVentaDelDiaTest

```
@DisplayName("RF15 Registrar precios de venta del día")
class RF15RegistrarPreciosDeVentaDelDiaTest extends TddBase {

    /** RF15-CP01 (E1). Se guarda 1 precio para hoy. */
    @Test
    @DisplayName("RF15-CP01 Guardar precio de corvina S/ 32.00.")
    void rf15_cp01_guardaPrecios() throws Exception {
        // ARRANGE
        when(precioDao.preciosDe(any())).thenReturn(List.of(precioVista(2, "Corvina",
"26.00", null, "32.00")));
        // ACT
        int n = precioServiceReal().guardarPrecios(Map.of(2, d("32")), false, 2);
        // ASSERT
        assertEquals(1, n);
        verify(precioDao).guardarPrecio(2, Fechas.hoy(), d("32.00"), d("26.00"));
    }

    /** RF15-CP02 (E2). Se copia S/ 9.00 como precio de hoy del bonito. */
    @Test
    @DisplayName("RF15-CP02 Tocar «Usar precios de ayer».")
    void rf15_cp02_usarPreciosDeAyer() throws Exception {
        // ARRANGE
        when(precioDao.preciosDe(any())).thenReturn(List.of(precioVista(1, "Bonito", "6.50",
null, "9.00")));
        // ACT
        int n = precioServiceReal().usarPreciosDeAyer(2);
        // ASSERT
        assertEquals(1, n);
        verify(precioDao).guardarPrecio(1, Fechas.hoy(), d("9.00"), d("6.50"));
    }

    /** RF15-CP03 (E3). Pide confirmar: «Corvina: S/ 24.00 está por debajo del costo S/
26.00. ¿Confirmar los precios?». */
    @Test
    @DisplayName("RF15-CP03 Guardar corvina a S/ 24.00 sin confirmar.")
    void rf15_cp03_precioBajoCosto() throws Exception {
        // ARRANGE
        when(precioDao.preciosDe(any())).thenReturn(List.of(precioVista(2, "Corvina",
"26.00", null, "32.00")));
        // ACT
        ConfirmacionException e = assertThrows(ConfirmacionException.class,
                () -> precioServiceReal().guardarPrecios(Map.of(2, d("24")), false, 2));
        // ASSERT
        assertEquals("Corvina: S/ 24.00 está por debajo del costo S/ 26.00. ¿Confirmar los
precios?", e.getMessage());
        verify(precioDao, never()).guardarPrecio(anyInt(), any(), any(), any());
    }

    /** RF15-CP04 (E4). Mensaje «Jurel: el precio debe ser mayor que cero». */
    @Test
    @DisplayName("RF15-CP04 Guardar jurel a S/ 0.")
    void rf15_cp04_precioCero() throws Exception {
        // ARRANGE
        when(precioDao.preciosDe(any())).thenReturn(List.of(precioVista(5, "Jurel", "5.00",
null, "8.00")));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> precioServiceReal().guardarPrecios(Map.of(5, d("0")), false, 2));
        // ASSERT
        assertEquals("Jurel: el precio debe ser mayor que cero", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF15RegistrarPreciosDeVentaDelDiaTest
FASE ROJA: clases Service sin implementar

+
├─ JUnit Jupiter ✔
│  └─ RF15 Registrar precios de venta del día ✔
│     ├─ RF15-CP01 Guardar precio de corvina S/ 32.00. ✘ Pendiente de implementar
│     ├─ RF15-CP04 Guardar jurel a S/ 0. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>
│     ├─ RF15-CP03 Guardar corvina a S/ 24.00 sin confirmar. ✘ Unexpected exception type thrown, expected: <ConfirmacionException> but was: <Unsuppor…
│     └─ RF15-CP02 Tocar «Usar precios de ayer». ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF15RegistrarPreciosDeVentaDelDiaTest
FASE VERDE: código implementado

+
├─ JUnit Jupiter ✔
│  └─ RF15 Registrar precios de venta del día ✔
│     ├─ RF15-CP01 Guardar precio de corvina S/ 32.00. ✔
│     ├─ RF15-CP04 Guardar jurel a S/ 0. ✔
│     ├─ RF15-CP03 Guardar corvina a S/ 24.00 sin confirmar. ✔
│     └─ RF15-CP02 Tocar «Usar precios de ayer». ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 15*. Ejecución de RF15RegistrarPreciosDeVentaDelDiaTest en rojo y en verde*

RF17 Registrar venta al contado · RF17RegistrarVentaAlContadoTest

```
@DisplayName("RF17 Registrar venta al contado")
class RF17RegistrarVentaAlContadoTest extends TddBase {

    /** RF17-CP01 (E1). Se guarda la venta CONTADO · YAPE, se descuentan 1.85 kg del lote 10
y el detalle queda con costo S/ 26.00. */
    @Test
    @DisplayName("RF17-CP01 Vender 1.85 kg de corvina al contado por Yape.")
    void rf17_cp01_ventaContadoYape() throws Exception {
        // ARRANGE
        prepararCorvina();
        // ACT
        ventaService().registrar(contado("YAPE", item(2, "1.85")), 2, false, false);
        // ASSERT
        verify(ventaDao).insertarVenta(any(), eq(2), isNull());
        verify(loteDao).descontar(10, d("1.85"));
        verify(ventaDao).insertarDetalle(7L, 2, d("1.85"), d("32.00"), d("26.00"));
    }

    /** RF17-CP02 (E2). Se guarda una venta con 2 detalles; total S/ 91.00. */
    @Test
    @DisplayName("RF17-CP02 Vender 2 kg de corvina y 3 kg de bonito en una venta.")
    void rf17_cp02_variosProductos() throws Exception {
        // ARRANGE
        prepararCorvina();
        prepararBonito();
        // ACT
        ventaService().registrar(contado("EFECTIVO", item(2, "2"), item(1, "3")), 2, false,
false);
        // ASSERT
        ArgumentCaptor<NuevaVenta> v = ArgumentCaptor.forClass(NuevaVenta.class);
        verify(ventaDao).insertarVenta(v.capture(), eq(2), isNull());
        assertEquals(d("91.00"), v.getValue().total());
        verify(ventaDao, times(2)).insertarDetalle(eq(7L), anyInt(), any(), any(), any());
    }

    /** RF17-CP03 (E3). Pide confirmar: «Solo quedan 15.00 kg de corvina. ¿Registrar
igual?»; no se guarda. */
    @Test
    @DisplayName("RF17-CP03 Vender 20 kg sin confirmar.")
    void rf17_cp03_superaSaldoPideConfirmar() throws Exception {
        // ARRANGE
        prepararCorvina();
        // ACT
```

```
        ConfirmacionException e = assertThrows(ConfirmacionException.class,
                () -> ventaService().registrar(contado("EFECTIVO", item(2, "20")), 2, false,
false));
        // ASSERT
        assertEquals("Solo quedan 15.00 kg de corvina. ¿Registrar igual?", e.getMessage());
        verify(ventaDao, never()).insertarVenta(any(), anyInt(), any());
    }

    /** RF17-CP04 (E4). Mensaje «Corvina: ingrese los kilos de la balanza». */
    @Test
    @DisplayName("RF17-CP04 Registrar la corvina con 0 kg.")
    void rf17_cp04_faltanKilos() throws Exception {
        // ARRANGE
        prepararCorvina();
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> ventaService().registrar(contado("EFECTIVO", item(2, "0")), 2, false,
false));
        // ASSERT
        assertEquals("Corvina: ingrese los kilos de la balanza", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF17RegistrarVentaAlContadoTest
FASE ROJA: clases Service sin implementar

.
├─ JUnit Jupiter ✔
│  └─ RF17 Registrar venta al contado ✔
│     ├─ RF17-CP03 Vender 20 kg sin confirmar. ✘ Unexpected exception type thrown, expected: <ConfirmacionException> but was: <UnsupportedOperationEx…
│     ├─ RF17-CP01 Vender 1.85 kg de corvina al contado por Yape. ✘ Pendiente de implementar
│     ├─ RF17-CP02 Vender 2 kg de corvina y 3 kg de bonito en una venta. ✘ Pendiente de implementar
│     └─ RF17-CP04 Registrar la corvina con 0 kg. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExce…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF17RegistrarVentaAlContadoTest
FASE VERDE: código implementado

.
├─ JUnit Jupiter ✔
│  └─ RF17 Registrar venta al contado ✔
│     ├─ RF17-CP03 Vender 20 kg sin confirmar. ✔
│     ├─ RF17-CP01 Vender 1.85 kg de corvina al contado por Yape. ✔
│     ├─ RF17-CP02 Vender 2 kg de corvina y 3 kg de bonito en una venta. ✔
│     └─ RF17-CP04 Registrar la corvina con 0 kg. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 16. *Ejecución de RF17RegistrarVentaAlContadoTest en rojo y en verde*

RF18 Registrar venta al crédito · RF18RegistrarVentaAlCreditoTest

```
@DisplayName("RF18 Registrar venta al crédito")
class RF18RegistrarVentaAlCreditoTest extends TddBase {

    /** RF18-CP01 (E1). Se guarda la venta CREDITO del cliente 1 por S/ 320.00, sin medio de
pago. */
    @Test
    @DisplayName("RF18-CP01 Vender 10 kg de corvina al crédito.")
    void rf18_cp01_ventaCredito() throws Exception {
        // ARRANGE
        prepararCorvina();
        when(clienteDao.buscar(1)).thenReturn(Optional.of(cliente(1, "Cevichería El Muelle",
"1500.00", 15)));
        when(credito.saldo(1)).thenReturn(saldoCliente(1, "228.00", "1500.00", "0.00"));
        // ACT
        ventaService().registrar(ventaCredito(1, item(2, "10")), 2, false, false);
        // ASSERT
        ArgumentCaptor<NuevaVenta> v = ArgumentCaptor.forClass(NuevaVenta.class);
        verify(ventaDao).insertarVenta(v.capture(), eq(2), isNull());
        assertEquals(1, v.getValue().idCliente());
```

```
        assertNull(v.getValue().medioPago());
        assertEquals(d("320.00"), v.getValue().total());
    }

    /** RF18-CP02 (E2). Pide confirmar: «Restaurante Sabor Norteño tiene deuda vencida de S/
1372.00. ¿Registrar igual?». */
    @Test
    @DisplayName("RF18-CP02 Vender 2 kg de corvina al crédito sin confirmar.")
    void rf18_cp02_clienteConVencido() throws Exception {
        // ARRANGE
        prepararCorvina();
        when(clienteDao.buscar(2)).thenReturn(Optional.of(cliente(2, "Restaurante Sabor
Norteño", "1500.00", 7)));
        when(credito.saldo(2)).thenReturn(saldoCliente(2, "1372.00", "1500.00", "1372.00"));
        // ACT
        ConfirmacionException e = assertThrows(ConfirmacionException.class,
                () -> ventaService().registrar(ventaCredito(2, item(2, "2")), 2, false,
false));
        // ASSERT
        assertEquals("Restaurante Sabor Norteño tiene deuda vencida de S/ 1372.00.
¿Registrar igual?", e.getMessage());
    }

    /** RF18-CP03 (E3). Mensaje «Supera el límite de crédito: disponible S/ 128.00 de S/
1500.00. Requiere autorización del administrador». */
    @Test
    @DisplayName("RF18-CP03 El vendedor vende 10 kg de corvina (S/ 320) al crédito.")
    void rf18_cp03_superaLimiteVendedor() throws Exception {
        // ARRANGE
        prepararCorvina();
        when(clienteDao.buscar(2)).thenReturn(Optional.of(cliente(2, "Restaurante Sabor
Norteño", "1500.00", 7)));
        when(credito.saldo(2)).thenReturn(saldoCliente(2, "1372.00", "1500.00", "0.00"));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> ventaService().registrar(ventaCredito(2, item(2, "10")), 2, false,
true));
        // ASSERT
        assertEquals("Supera el límite de crédito: disponible S/ 128.00 de S/ 1500.00.
Requiere autorización del administrador", e.getMessage());
        verify(ventaDao, never()).insertarVenta(any(), anyInt(), any());
    }

    /** RF18-CP04 (E4). Mensaje «Este cliente solo puede comprar al contado». */
    @Test
    @DisplayName("RF18-CP04 Vender al crédito a la pollería.")
    void rf18_cp04_sinCreditoHabilitado() throws Exception {
        // ARRANGE
        prepararCorvina();
        when(clienteDao.buscar(3)).thenReturn(Optional.of(cliente(3, "Pollería Don Lucho",
"0.00", 7)));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> ventaService().registrar(ventaCredito(3, item(2, "2")), 2, false,
false));
        // ASSERT
        assertEquals("Este cliente solo puede comprar al contado", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF18RegistrarVentaAlCreditoTest
FASE ROJA: clases Service sin implementar

.
├─ JUnit Jupiter ✔
│  └─ RF18 Registrar venta al crédito ✔
│     ├─ RF18-CP02 Vender 2 kg de corvina al crédito sin confirmar. ✘ Unexpected exception type thrown, expected: <ConfirmacionException> but was: <U…
│     ├─ RF18-CP01 Vender 10 kg de corvina al crédito. ✘ Pendiente de implementar
│     ├─ RF18-CP03 El vendedor vende 10 kg de corvina (S/ 320) al crédito. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: …
│     └─ RF18-CP04 Vender al crédito a la pollería. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationEx…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF18RegistrarVentaAlCreditoTest
FASE VERDE: código implementado

.
├─ JUnit Jupiter ✔
│  └─ RF18 Registrar venta al crédito ✔
│     ├─ RF18-CP02 Vender 2 kg de corvina al crédito sin confirmar. ✔
│     ├─ RF18-CP01 Vender 10 kg de corvina al crédito. ✔
│     ├─ RF18-CP03 El vendedor vende 10 kg de corvina (S/ 320) al crédito. ✔
│     └─ RF18-CP04 Vender al crédito a la pollería. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 17. *Ejecución de RF18RegistrarVentaAlCreditoTest en rojo y en verde*

RF19 Calcular importe por peso · RF19CalcularImportePorPesoTest

```
@DisplayName("RF19 Calcular importe por peso")
class RF19CalcularImportePorPesoTest extends TddBase {

    /** RF19-CP01 (E1). Importe S/ 59.20. */
    @Test
    @DisplayName("RF19-CP01 Calcular 1.85 kg.")
    void rf19_cp01_importeKilosPorPrecio() throws Exception {
        // ARRANGE
        BigDecimal kilos = d("1.85");
        BigDecimal precio = d("32.00");
        // ACT
        BigDecimal importe = Calculos.importe(kilos, precio);
        // ASSERT
        assertEquals(d("59.20"), importe);
    }

    /** RF19-CP02 (E2). Importe S/ 24.00. */
    @Test
    @DisplayName("RF19-CP02 Calcular 4 kg de remate.")
    void rf19_cp02_importeRemate() throws Exception {
        // ARRANGE
        BigDecimal precioRemate = d("6.00");
        // ACT
        BigDecimal importe = Calculos.importe(d("4"), precioRemate);
        // ASSERT
        assertEquals(d("24.00"), importe);
    }

    /** RF19-CP03 (E3). Kilos 1.86. */
    @Test
    @DisplayName("RF19-CP03 Normalizar los kilos.")
    void rf19_cp03_redondeaKilos() throws Exception {
        // ARRANGE
        BigDecimal balanza = d("1.855");
        // ACT
        BigDecimal kilos = Calculos.kilos(balanza);
        // ASSERT
        assertEquals(d("1.86"), kilos);
    }

    /** RF19-CP04 (E4). Mensaje «Pota: ingrese el precio de hoy de este producto». */
    @Test
    @DisplayName("RF19-CP04 Registrar 2 kg de pota.")
```

```
    void rf19_cp04_sinPrecioDelDia() throws Exception {
        // ARRANGE
        when(productoDao.buscar(3)).thenReturn(Optional.of(producto(3, "Pota")));
        when(precioDao.precioDe(eq(3), any())).thenReturn(Optional.empty());
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> ventaService().registrar(contado("EFECTIVO", item(3, "2")), 2, false,
false));
        // ASSERT
        assertEquals("Pota: ingrese el precio de hoy de este producto", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF19CalcularImportePorPesoTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF19 Calcular importe por peso ✔
│     ├─ RF19-CP04 Registrar 2 kg de pota. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>
│     ├─ RF19-CP02 Calcular 4 kg de remate. ✘ Pendiente de implementar
│     ├─ RF19-CP01 Calcular 1.85 kg. ✘ Pendiente de implementar
│     └─ RF19-CP03 Normalizar los kilos. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF19CalcularImportePorPesoTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF19 Calcular importe por peso ✔
│     ├─ RF19-CP04 Registrar 2 kg de pota. ✔
│     ├─ RF19-CP02 Calcular 4 kg de remate. ✔
│     ├─ RF19-CP01 Calcular 1.85 kg. ✔
│     └─ RF19-CP03 Normalizar los kilos. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 18*. Ejecución de RF19CalcularImportePorPesoTest en rojo y en verde*

RF20 Anular venta · RF20AnularVentaTest

```
@DisplayName("RF20 Anular venta")
class RF20AnularVentaTest extends TddBase {

    /** RF20-CP01 (E1). Se devuelven 1.85 kg al lote 10 y la venta queda anulada. */
    @Test
    @DisplayName("RF20-CP01 Anular con motivo «Error de peso».")
    void rf20_cp01_anulaYDevuelveKilos() throws Exception {
        // ARRANGE
        when(ventaDao.buscar(7L)).thenReturn(Optional.of(venta(7, null, "CONTADO", "59.20",
"VIGENTE", null)));
        when(ventaDao.detalle(7L)).thenReturn(List.of(new DetalleVentaVista(2, "Corvina",
d("1.85"), d("32.00"), d("26.00"))));
        when(loteDao.ultimoLote(2)).thenReturn(Optional.of(10));
        // ACT
        ventaService().anular(7L, "Error de peso", 1);
        // ASSERT
        verify(loteDao).sumar(10, d("1.85"));
        verify(ventaDao).anular(7L, "Error de peso");
    }

    /** RF20-CP02 (E2). La bitácora termina en «tenía factura: emitir nota de crédito en
SUNAT». */
    @Test
    @DisplayName("RF20-CP02 Anular la venta 8.")
    void rf20_cp02_ventaConFactura() throws Exception {
        // ARRANGE
        when(ventaDao.buscar(8L)).thenReturn(Optional.of(venta(8, null, "CONTADO", "96.00",
"VIGENTE", 3)));
        // ACT
```

```
        ventaService().anular(8L, "Cliente devolvió", 1);
        // ASSERT
        ArgumentCaptor<String> det = ArgumentCaptor.forClass(String.class);
        verify(auditoria).registrar(eq(1), eq("ANULAR_VENTA"), det.capture());
        assertTrue(det.getValue().endsWith("tenía factura: emitir nota de crédito en
SUNAT"));
    }

    /** RF20-CP03 (E3). Mensaje «La venta ya tiene abonos aplicados (S/ 50.00); no se puede
anular». */
    @Test
    @DisplayName("RF20-CP03 Anular la venta 3.")
    void rf20_cp03_conAbonos() throws Exception {
        // ARRANGE
        when(ventaDao.buscar(3L)).thenReturn(Optional.of(venta(3, 1, "CREDITO", "228.00",
"VIGENTE", null)));
        when(credito.pendientes(1)).thenReturn(List.of(new PendienteVenta(3L,
Fechas.ahora().minusDays(7), d("228.00"), d("50.00"))));
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
ventaService().anular(3L, "Error", 1));
        // ASSERT
        assertEquals("La venta ya tiene abonos aplicados (S/ 50.00); no se puede anular",
e.getMessage());
        verify(ventaDao, never()).anular(anyLong(), anyString());
    }

    /** RF20-CP04 (E4). Mensaje «Indique el motivo de la anulación». */
    @Test
    @DisplayName("RF20-CP04 Anular sin escribir motivo.")
    void rf20_cp04_faltaMotivo() throws Exception {
        // ARRANGE
        when(ventaDao.buscar(7L)).thenReturn(Optional.of(venta(7, null, "CONTADO", "59.20",
"VIGENTE", null)));
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
ventaService().anular(7L, "  ", 1));
        // ASSERT
        assertEquals("Indique el motivo de la anulación", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF20AnularVentaTest
FASE ROJA: clases Service sin implementar

.
├─ JUnit Jupiter ✔
│  └─ RF20 Anular venta ✔
│     ├─ RF20-CP01 Anular con motivo «Error de peso». ✘ Pendiente de implementar
│     ├─ RF20-CP04 Anular sin escribir motivo. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExcepti…
│     ├─ RF20-CP03 Anular la venta 3. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>
│     └─ RF20-CP02 Anular la venta 8. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF20AnularVentaTest
FASE VERDE: código implementado

.
├─ JUnit Jupiter ✔
│  └─ RF20 Anular venta ✔
│     ├─ RF20-CP01 Anular con motivo «Error de peso». ✔
│     ├─ RF20-CP04 Anular sin escribir motivo. ✔
│     ├─ RF20-CP03 Anular la venta 3. ✔
│     └─ RF20-CP02 Anular la venta 8. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 19*. Ejecución de RF20AnularVentaTest en rojo y en verde*

RF21 Registrar venta de remate · RF21RegistrarVentaDeRemateTest

```
@DisplayName("RF21 Registrar venta de remate")
class RF21RegistrarVentaDeRemateTest extends TddBase {

    /** RF21-CP01 (E1). Se guarda la venta marcada como remate con detalle a S/ 6.00 por kg.
*/
    @Test
    @DisplayName("RF21-CP01 Rematar 4 kg a S/ 6.00 al contado.")
    void rf21_cp01_registraRemate() throws Exception {
        // ARRANGE
        prepararBonito();
        // ACT
        ventaService().registrar(remate(null, item(1, "4", "6.00")), 2, false, false);
        // ASSERT
        ArgumentCaptor<NuevaVenta> v = ArgumentCaptor.forClass(NuevaVenta.class);
        verify(ventaDao).insertarVenta(v.capture(), eq(2), isNull());
        assertTrue(v.getValue().esRemate());
        verify(ventaDao).insertarDetalle(7L, 1, d("4.00"), d("6.00"), d("6.50"));
    }

    /** RF21-CP02 (E2). Se guarda la venta CREDITO del cliente 1 marcada como remate. */
    @Test
    @DisplayName("RF21-CP02 Rematar 4 kg de bonito a S/ 6.00 al crédito del cliente 1.")
    void rf21_cp02_remateAlCredito() throws Exception {
        // ARRANGE
        prepararBonito();
        when(clienteDao.buscar(1)).thenReturn(Optional.of(cliente(1, "Cevichería El Muelle",
"1500.00", 15)));
        when(credito.saldo(1)).thenReturn(saldoCliente(1, "228.00", "1500.00", "0.00"));
        // ACT
        ventaService().registrar(remate(1, item(1, "4", "6.00")), 2, false, false);
        // ASSERT
        ArgumentCaptor<NuevaVenta> v = ArgumentCaptor.forClass(NuevaVenta.class);
        verify(ventaDao).insertarVenta(v.capture(), eq(2), isNull());
        assertEquals("CREDITO", v.getValue().tipoPago());
        assertTrue(v.getValue().esRemate());
    }

    /** RF21-CP03 (E3). Mensaje «Bonito: el precio de remate debe ser menor al del día (S/
9.00)». */
    @Test
    @DisplayName("RF21-CP03 Rematar a S/ 9.50.")
    void rf21_cp03_remateNoMenor() throws Exception {
        // ARRANGE
        prepararBonito();
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> ventaService().registrar(remate(null, item(1, "4", "9.50")), 2, false,
false));
        // ASSERT
        assertEquals("Bonito: el precio de remate debe ser menor al del día (S/ 9.00)",
e.getMessage());
    }

    /** RF21-CP04 (E4). Mensaje «Perico: ingrese el precio de hoy de este producto». */
    @Test
    @DisplayName("RF21-CP04 Rematar perico sin escribir precio.")
    void rf21_cp04_remateSinPrecio() throws Exception {
        // ARRANGE
        when(productoDao.buscar(7)).thenReturn(Optional.of(producto(7, "Perico")));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> ventaService().registrar(remate(null, item(7, "3")), 2, false,
false));
        // ASSERT
        assertEquals("Perico: ingrese el precio de hoy de este producto", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF21RegistrarVentaDeRemateTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF21 Registrar venta de remate ✔
│     ├─ RF21-CP02 Rematar 4 kg de bonito a S/ 6.00 al crédito del cliente 1. ✘ Pendiente de implementar
│     ├─ RF21-CP04 Rematar perico sin escribir precio. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperatio…
│     ├─ RF21-CP01 Rematar 4 kg a S/ 6.00 al contado. ✘ Pendiente de implementar
│     └─ RF21-CP03 Rematar a S/ 9.50. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>
╵

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF21RegistrarVentaDeRemateTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF21 Registrar venta de remate ✔
│     ├─ RF21-CP02 Rematar 4 kg de bonito a S/ 6.00 al crédito del cliente 1. ✔
│     ├─ RF21-CP04 Rematar perico sin escribir precio. ✔
│     ├─ RF21-CP01 Rematar 4 kg a S/ 6.00 al contado. ✔
│     └─ RF21-CP03 Rematar a S/ 9.50. ✔
╵

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 20. *Ejecución de RF21RegistrarVentaDeRemateTest en rojo y en verde*

RF22 Registrar autoconsumo · RF22RegistrarAutoconsumoTest

```
@DisplayName("RF22 Registrar autoconsumo")
class RF22RegistrarAutoconsumoTest extends TddBase {

    /** RF22-CP01 (E1). Se descuentan 1.50 kg del lote 2 y el valor es S/ 7.50. */
    @Test
    @DisplayName("RF22-CP01 Registrar 1.5 kg de autoconsumo.")
    void rf22_cp01_autoconsumoValorizado() throws Exception {
        // ARRANGE
        when(productoDao.buscar(5)).thenReturn(Optional.of(producto(5, "Jurel")));
        when(loteDao.kilosDisponibles(5)).thenReturn(d("6.00"));
        when(loteDao.disponiblesFifo(5)).thenReturn(List.of(new LoteKg(2, d("6.00"),
d("5.00"))));
        // ACT
        BigDecimal valor = saldoService().registrarAutoconsumo(5, d("1.5"), 2);
        // ASSERT
        assertEquals(d("7.50"), valor);
        verify(loteDao).descontar(2, d("1.50"));
    }

    /** RF22-CP02 (E2). Se toman 2 kg del lote 1 y 1 kg del lote 7; valor S/ 20.00. */
    @Test
    @DisplayName("RF22-CP02 Registrar 3 kg de autoconsumo.")
    void rf22_cp02_fifoPrimeroElAntiguo() throws Exception {
        // ARRANGE
        when(productoDao.buscar(1)).thenReturn(Optional.of(producto(1, "Bonito")));
        when(loteDao.kilosDisponibles(1)).thenReturn(d("12.00"));
        when(loteDao.disponiblesFifo(1)).thenReturn(List.of(new LoteKg(1, d("2.00"),
d("6.50")), new LoteKg(7, d("10.00"), d("7.00"))));
        // ACT
        BigDecimal valor = saldoService().registrarAutoconsumo(1, d("3"), 2);
        // ASSERT
        assertEquals(d("20.00"), valor);
        InOrder orden = inOrder(loteDao);
        orden.verify(loteDao).descontar(1, d("2.00"));
        orden.verify(loteDao).descontar(7, d("1.00"));
    }

    /** RF22-CP03 (E3). Mensaje «Solo hay 6.00 kg de jurel: el saldo no puede quedar
negativo». */
    @Test
    @DisplayName("RF22-CP03 Registrar 8 kg de autoconsumo.")
    void rf22_cp03_superaSaldo() throws Exception {
        // ARRANGE
```

```
        when(productoDao.buscar(5)).thenReturn(Optional.of(producto(5, "Jurel")));
        when(loteDao.kilosDisponibles(5)).thenReturn(d("6.00"));
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
saldoService().registrarAutoconsumo(5, d("8"), 2));
        // ASSERT
        assertEquals("Solo hay 6.00 kg de jurel: el saldo no puede quedar negativo",
e.getMessage());
        verify(loteDao, never()).descontar(anyInt(), any());
    }

    /** RF22-CP04 (E4). Mensaje «Ingrese los kilos». */
    @Test
    @DisplayName("RF22-CP04 Registrar 0 kg.")
    void rf22_cp04_kilosCero() throws Exception {
        // ARRANGE
        when(productoDao.buscar(5)).thenReturn(Optional.of(producto(5, "Jurel")));
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
saldoService().registrarAutoconsumo(5, d("0"), 2));
        // ASSERT
        assertEquals("Ingrese los kilos", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF22RegistrarAutoconsumoTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF22 Registrar autoconsumo ✔
│     ├─ RF22-CP01 Registrar 1.5 kg de autoconsumo. ✘ Pendiente de implementar
│     ├─ RF22-CP04 Registrar 0 kg. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>
│     ├─ RF22-CP02 Registrar 3 kg de autoconsumo. ✘ Pendiente de implementar
│     └─ RF22-CP03 Registrar 8 kg de autoconsumo. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExce…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF22RegistrarAutoconsumoTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF22 Registrar autoconsumo ✔
│     ├─ RF22-CP01 Registrar 1.5 kg de autoconsumo. ✔
│     ├─ RF22-CP04 Registrar 0 kg. ✔
│     ├─ RF22-CP02 Registrar 3 kg de autoconsumo. ✔
│     └─ RF22-CP03 Registrar 8 kg de autoconsumo. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 21. *Ejecución de RF22RegistrarAutoconsumoTest en rojo y en verde*

RF25 Sincronizar ventas pendientes · RF25SincronizarVentasPendientesTest

```
@DisplayName("RF25 Sincronizar ventas pendientes")
class RF25SincronizarVentasPendientesTest extends TddBase {

    /** RF25-CP01 (E1). Resultado SINCRONIZADA «Venta 0007»; la venta se guarda con fecha de
sincronización. */
    @Test
    @DisplayName("RF25-CP01 Sincronizar.")
    void rf25_cp01_sincronizaVenta() throws Exception {
        // ARRANGE
        prepararCorvina();
        NuevaVenta enCola = new NuevaVenta("cel-0001", Fechas.hoy().atTime(6, 40),
"CONTADO", "EFECTIVO", null, false, List.of(item(2, "1")));
        // ACT
        List<ResultadoSync> r = ventaService().sincronizar(List.of(enCola), 2);
        // ASSERT
        assertEquals("SINCRONIZADA", r.get(0).estado());
        assertEquals("Venta 0007", r.get(0).mensaje());
```

```
        verify(ventaDao).insertarVenta(argThat(v -> v.fechaHora().getHour() == 6), eq(2),
notNull());
    }

    /** RF25-CP02 (E2). Resultado DUPLICADA «Ya estaba registrada»; no se inserta. */
    @Test
    @DisplayName("RF25-CP02 Sincronizar otra vez la misma venta.")
    void rf25_cp02_noDuplica() throws Exception {
        // ARRANGE
        when(ventaDao.existeUuid("cel-0001")).thenReturn(true);
        NuevaVenta repetida = new NuevaVenta("cel-0001", Fechas.ahora(), "CONTADO",
"EFECTIVO", null, false, List.of(item(2, "1")));
        // ACT
        List<ResultadoSync> r = ventaService().sincronizar(List.of(repetida), 2);
        // ASSERT
        assertEquals("DUPLICADA", r.get(0).estado());
        assertEquals("Ya estaba registrada", r.get(0).mensaje());
        verify(ventaDao, never()).insertarVenta(any(), anyInt(), any());
    }

    /** RF25-CP03 (E3). La válida queda SINCRONIZADA; la otra RECHAZADA «Corvina: ingrese
los kilos de la balanza». */
    @Test
    @DisplayName("RF25-CP03 Sincronizar las dos.")
    void rf25_cp03_rechazaUnaYGuardaOtra() throws Exception {
        // ARRANGE
        prepararCorvina();
        NuevaVenta buena = new NuevaVenta("cel-0002", Fechas.ahora(), "CONTADO", "EFECTIVO",
null, false, List.of(item(2, "1")));
        NuevaVenta mala = new NuevaVenta("cel-0003", Fechas.ahora(), "CONTADO", "EFECTIVO",
null, false, List.of(item(2, "0")));
        // ACT
        List<ResultadoSync> r = ventaService().sincronizar(List.of(buena, mala), 2);
        // ASSERT
        assertEquals("SINCRONIZADA", r.get(0).estado());
        assertEquals("RECHAZADA", r.get(1).estado());
        assertEquals("Corvina: ingrese los kilos de la balanza", r.get(1).mensaje());
    }

    /** RF25-CP04 (E4). Resultado RECHAZADA «La venta no tiene identificador». */
    @Test
    @DisplayName("RF25-CP04 Sincronizar.")
    void rf25_cp04_sinIdentificador() throws Exception {
        // ARRANGE
        NuevaVenta sinId = new NuevaVenta(" ", Fechas.ahora(), "CONTADO", "EFECTIVO", null,
false, List.of(item(2, "1")));
        // ACT
        List<ResultadoSync> r = ventaService().sincronizar(List.of(sinId), 2);
        // ASSERT
        assertEquals("RECHAZADA", r.get(0).estado());
        assertEquals("La venta no tiene identificador", r.get(0).mensaje());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF25SincronizarVentasPendientesTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF25 Sincronizar ventas pendientes ✔
│     ├─ RF25-CP01 Sincronizar. ✘ Pendiente de implementar
│     ├─ RF25-CP03 Sincronizar las dos. ✘ Pendiente de implementar
│     ├─ RF25-CP04 Sincronizar. ✘ Pendiente de implementar
│     └─ RF25-CP02 Sincronizar otra vez la misma venta. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF25SincronizarVentasPendientesTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF25 Sincronizar ventas pendientes ✔
│     ├─ RF25-CP01 Sincronizar. ✔
│     ├─ RF25-CP03 Sincronizar las dos. ✔
│     ├─ RF25-CP04 Sincronizar. ✔
│     └─ RF25-CP02 Sincronizar otra vez la misma venta. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 22. *Ejecución de RF25SincronizarVentasPendientesTest en rojo y en verde*

RF27 Registrar saldo para el día siguiente · RF27RegistrarSaldoParaElDiaSiguienteTest

```
@DisplayName("RF27 Registrar saldo para el día siguiente")
class RF27RegistrarSaldoParaElDiaSiguienteTest extends TddBase {

    /** RF27-CP01 (E1). Se descuentan 0.20 kg por «Diferencia al cierre del puesto» y el
saldo para mañana es 5.80 kg. */
    @Test
    @DisplayName("RF27-CP01 Pesar 5.80 kg (3.3 % menos).")
    void rf27_cp01_cierreDescuentaDiferencia() throws Exception {
        // ARRANGE
        prepararCierre(5, "Jurel", "6.00");
        // ACT
        BigDecimal total = saldoService().registrarCierre(Map.of(5, d("5.80")), Map.of(),
2);
        // ASSERT
        assertEquals(d("5.80"), total);
        verify(loteDao).descontar(2, d("0.20"));
    }

    /** RF27-CP02 (E2). Se suman 0.20 kg al último lote. */
    @Test
    @DisplayName("RF27-CP02 Pesar 6.20 kg.")
    void rf27_cp02_cierreSumaDiferencia() throws Exception {
        // ARRANGE
        prepararCierre(5, "Jurel", "6.00");
        when(loteDao.ultimoLote(5)).thenReturn(Optional.of(2));
        // ACT
        saldoService().registrarCierre(Map.of(5, d("6.20")), Map.of(), 2);
        // ASSERT
        verify(loteDao).sumar(2, d("0.20"));
    }

    /** RF27-CP03 (E3). Mensaje «Jurel: diferencia de 16.67%. Indique el motivo». */
    @Test
    @DisplayName("RF27-CP03 Pesar 5.00 kg sin motivo.")
    void rf27_cp03_diferenciaPideMotivo() throws Exception {
        // ARRANGE
        prepararCierre(5, "Jurel", "6.00");
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
```

```
                () -> saldoService().registrarCierre(Map.of(5, d("5.00")), Map.of(), 2));
        // ASSERT
        assertEquals("Jurel: diferencia de 16.67%. Indique el motivo", e.getMessage());
    }

    /** RF27-CP04 (E4). Mensaje «Los kilos reales no pueden ser negativos». */
    @Test
    @DisplayName("RF27-CP04 Ingresar -1 kg.")
    void rf27_cp04_kilosNegativos() throws Exception {
        // ARRANGE
        prepararCierre(5, "Jurel", "6.00");
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> saldoService().registrarCierre(Map.of(5, d("-1")), Map.of(), 2));
        // ASSERT
        assertEquals("Los kilos reales no pueden ser negativos", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF27RegistrarSaldoParaElDiaSiguienteTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF27 Registrar saldo para el día siguiente ✔
│     ├─ RF27-CP01 Pesar 5.80 kg (3.3 % menos). ✘ Pendiente de implementar
│     ├─ RF27-CP02 Pesar 6.20 kg. ✘ Pendiente de implementar
│     ├─ RF27-CP04 Ingresar -1 kg. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>
│     └─ RF27-CP03 Pesar 5.00 kg sin motivo. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF27RegistrarSaldoParaElDiaSiguienteTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF27 Registrar saldo para el día siguiente ✔
│     ├─ RF27-CP01 Pesar 5.80 kg (3.3 % menos). ✔
│     ├─ RF27-CP02 Pesar 6.20 kg. ✔
│     ├─ RF27-CP04 Ingresar -1 kg. ✔
│     └─ RF27-CP03 Pesar 5.00 kg sin motivo. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 23*. Ejecución de RF27RegistrarSaldoParaElDiaSiguienteTest en rojo y en verde*

RF28 Consultar saldo por antigüedad · RF28ConsultarSaldoPorAntiguedadTest

```
@DisplayName("RF28 Consultar saldo por antigüedad")
class RF28ConsultarSaldoPorAntiguedadTest extends TddBase {

    /** RF28-CP01 (E1). Color «verde». */
    @Test
    @DisplayName("RF28-CP01 Clasificar un lote de hoy (0 días).")
    void rf28_cp01_loteDeHoyVerde() throws Exception {
        // ARRANGE
        int diasAlerta = 3;
        // ACT
        String color = Calculos.colorSaldo(0, diasAlerta);
        // ASSERT
        assertEquals("verde", color);
    }

    /** RF28-CP02 (E2). Color «naranja». */
    @Test
    @DisplayName("RF28-CP02 Clasificar un lote de 2 días.")
    void rf28_cp02_unDiaAntesNaranja() throws Exception {
        // ARRANGE
        int diasAlerta = 3;
        // ACT
        String color = Calculos.colorSaldo(2, diasAlerta);
        // ASSERT
```

```
        assertEquals("naranja", color);
    }

    /** RF28-CP03 (E3). Color «rojo». */
    @Test
    @DisplayName("RF28-CP03 Clasificar un lote de 3 días.")
    void rf28_cp03_enAlertaRojo() throws Exception {
        // ARRANGE
        int diasAlerta = 3;
        // ACT
        String color = Calculos.colorSaldo(3, diasAlerta);
        // ASSERT
        assertEquals("rojo", color);
    }

    /** RF28-CP04 (E4). La línea trae 3 días y color «rojo». */
    @Test
    @DisplayName("RF28-CP04 Consultar el saldo.")
    void rf28_cp04_saldoConDias() throws Exception {
        // ARRANGE
        when(configDao.obtener()).thenReturn(config());
        when(loteDao.lotesConKilos()).thenReturn(List.of(lote(1, 1, "Bonito", 3, "8.00",
"6.50")));
        // ACT
        List<SaldoLinea> saldo = saldoService().saldo();
        // ASSERT
        assertEquals(3, saldo.get(0).dias());
        assertEquals("rojo", saldo.get(0).color());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF28ConsultarSaldoPorAntiguedadTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF28 Consultar saldo por antigüedad ✔
│     ├─ RF28-CP04 Consultar el saldo. ✘ Pendiente de implementar
│     ├─ RF28-CP02 Clasificar un lote de 2 días. ✘ Pendiente de implementar
│     ├─ RF28-CP01 Clasificar un lote de hoy (0 días). ✘ Pendiente de implementar
│     └─ RF28-CP03 Clasificar un lote de 3 días. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF28ConsultarSaldoPorAntiguedadTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF28 Consultar saldo por antigüedad ✔
│     ├─ RF28-CP04 Consultar el saldo. ✔
│     ├─ RF28-CP02 Clasificar un lote de 2 días. ✔
│     ├─ RF28-CP01 Clasificar un lote de hoy (0 días). ✔
│     └─ RF28-CP03 Clasificar un lote de 3 días. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 24. *Ejecución de RF28ConsultarSaldoPorAntiguedadTest en rojo y en verde*

RF29 Alertar mercadería antigua · RF29AlertarMercaderiaAntiguaTest

```
@DisplayName("RF29 Alertar mercadería antigua")
class RF29AlertarMercaderiaAntiguaTest extends TddBase {

    /** RF29-CP01 (E1). Solo aparece el bonito. */
    @Test
    @DisplayName("RF29-CP01 Consultar la mercadería antigua.")
    void rf29_cp01_alertaSoloAntiguos() throws Exception {
        // ARRANGE
        when(configDao.obtener()).thenReturn(config());
```

```
        when(loteDao.lotesConKilos()).thenReturn(List.of(lote(1, 1, "Bonito", 3, "8.00",
"6.50"), lote(2, 5, "Jurel", 2, "6.00", "5.00")));
        // ACT
        List<SaldoLinea> antiguos = saldoService().mercaderiaAntigua();
        // ASSERT
        assertEquals(1, antiguos.size());
        assertEquals("Bonito", antiguos.get(0).lote().producto());
    }

    /** RF29-CP02 (E2). Lista vacía: no hay alerta. */
    @Test
    @DisplayName("RF29-CP02 Consultar la mercadería antigua.")
    void rf29_cp02_sinAntiguos() throws Exception {
        // ARRANGE
        when(configDao.obtener()).thenReturn(config());
        when(loteDao.lotesConKilos()).thenReturn(List.of(lote(7, 2, "Corvina", 0, "40.00",
"26.00")));
        // ACT
        List<SaldoLinea> antiguos = saldoService().mercaderiaAntigua();
        // ASSERT
        assertTrue(antiguos.isEmpty());
    }

    /** RF29-CP03 (E3). Aparece el jurel. */
    @Test
    @DisplayName("RF29-CP03 Consultar la mercadería antigua.")
    void rf29_cp03_usaDiasConfigurados() throws Exception {
        // ARRANGE
        when(configDao.obtener()).thenReturn(new Configuracion(2, d("5.00"), d("5.00")));
        when(loteDao.lotesConKilos()).thenReturn(List.of(lote(2, 5, "Jurel", 2, "6.00",
"5.00")));
        // ACT
        List<SaldoLinea> antiguos = saldoService().mercaderiaAntigua();
        // ASSERT
        assertEquals(1, antiguos.size());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF29AlertarMercaderiaAntiguaTest
FASE ROJA: clases Service sin implementar

.
├─ JUnit Jupiter ✔
│  └─ RF29 Alertar mercadería antigua ✔
│     ├─ RF29-CP02 Consultar la mercadería antigua. ✘ Pendiente de implementar
│     ├─ RF29-CP03 Consultar la mercadería antigua. ✘ Pendiente de implementar
│     └─ RF29-CP01 Consultar la mercadería antigua. ✘ Pendiente de implementar

[   3 tests found ]  [   0 tests successful ]  [   3 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF29AlertarMercaderiaAntiguaTest
FASE VERDE: código implementado

.
├─ JUnit Jupiter ✔
│  └─ RF29 Alertar mercadería antigua ✔
│     ├─ RF29-CP02 Consultar la mercadería antigua. ✔
│     ├─ RF29-CP03 Consultar la mercadería antigua. ✔
│     └─ RF29-CP01 Consultar la mercadería antigua. ✔

[   3 tests found ]  [   3 tests successful ]  [   0 tests failed ]
```

Figura 25*. Ejecución de RF29AlertarMercaderiaAntiguaTest en rojo y en verde*

RF30 Registrar ajuste de saldo · RF30RegistrarAjusteDeSaldoTest

```
@DisplayName("RF30 Registrar ajuste de saldo")
class RF30RegistrarAjusteDeSaldoTest extends TddBase {
```

```
    /** RF30-CP01 (E1). Se descuentan 0.50 kg y se registra el ajuste con valor S/ 2.50. */
    @Test
    @DisplayName("RF30-CP01 Ajuste MERMA de -0.5 kg «Pérdida de agua».")
    void rf30_cp01_ajusteMerma() throws Exception {
        // ARRANGE
        when(productoDao.buscar(5)).thenReturn(Optional.of(producto(5, "Jurel")));
        when(loteDao.kilosDisponibles(5)).thenReturn(d("6.00"));
        when(loteDao.disponiblesFifo(5)).thenReturn(List.of(new LoteKg(2, d("6.00"),
d("5.00"))));
        // ACT
        BigDecimal valor = saldoService().registrarAjuste(5, d("-0.5"), "MERMA", "Pérdida de
agua", 1);
        // ASSERT
        assertEquals(d("2.50"), valor);
        verify(loteDao).insertarAjuste(eq(2), eq(1), any(), eq("MERMA"), eq(d("-0.50")),
eq(d("2.50")), eq("Pérdida de agua"));
    }

    /** RF30-CP02 (E2). Se suma 1.00 kg al lote 4 y el valor es S/ -5.50 (no es pérdida). */
    @Test
    @DisplayName("RF30-CP02 Ajuste CORRECCION de +1 kg «Error de conteo».")
    void rf30_cp02_ajustePositivo() throws Exception {
        // ARRANGE
        when(productoDao.buscar(3)).thenReturn(Optional.of(producto(3, "Pota")));
        when(precioDao.ultimoCosto(eq(3), any())).thenReturn(Optional.of(d("5.50")));
        when(loteDao.ultimoLote(3)).thenReturn(Optional.of(4));
        // ACT
        BigDecimal valor = saldoService().registrarAjuste(3, d("1"), "CORRECCION", "Error de
conteo", 1);
        // ASSERT
        assertEquals(d("-5.50"), valor);
        verify(loteDao).sumar(4, d("1.00"));
    }

    /** RF30-CP03 (E3). Mensaje «Indique el motivo del ajuste». */
    @Test
    @DisplayName("RF30-CP03 Ajuste MERMA sin motivo.")
    void rf30_cp03_faltaMotivo() throws Exception {
        // ARRANGE
        when(productoDao.buscar(5)).thenReturn(Optional.of(producto(5, "Jurel")));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> saldoService().registrarAjuste(5, d("-0.5"), "MERMA", "", 1));
        // ASSERT
        assertEquals("Indique el motivo del ajuste", e.getMessage());
    }

    /** RF30-CP04 (E4). Mensaje «Solo hay 6.00 kg de jurel: el saldo no puede quedar
negativo». */
    @Test
    @DisplayName("RF30-CP04 Ajuste DESECHO de -7 kg «Malogrado».")
    void rf30_cp04_saldoNegativo() throws Exception {
        // ARRANGE
        when(productoDao.buscar(5)).thenReturn(Optional.of(producto(5, "Jurel")));
        when(loteDao.kilosDisponibles(5)).thenReturn(d("6.00"));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> saldoService().registrarAjuste(5, d("-7"), "DESECHO", "Malogrado",
1));
        // ASSERT
        assertEquals("Solo hay 6.00 kg de jurel: el saldo no puede quedar negativo",
e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF30RegistrarAjusteDeSaldoTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF30 Registrar ajuste de saldo ✔
│     ├─ RF30-CP01 Ajuste MERMA de -0.5 kg «Pérdida de agua». ✘ Pendiente de implementar
│     ├─ RF30-CP03 Ajuste MERMA sin motivo. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>
│     ├─ RF30-CP02 Ajuste CORRECCION de +1 kg «Error de conteo». ✘ Pendiente de implementar
│     └─ RF30-CP04 Ajuste DESECHO de -7 kg «Malogrado». ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperati…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF30RegistrarAjusteDeSaldoTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF30 Registrar ajuste de saldo ✔
│     ├─ RF30-CP01 Ajuste MERMA de -0.5 kg «Pérdida de agua». ✔
│     ├─ RF30-CP03 Ajuste MERMA sin motivo. ✔
│     ├─ RF30-CP02 Ajuste CORRECCION de +1 kg «Error de conteo». ✔
│     └─ RF30-CP04 Ajuste DESECHO de -7 kg «Malogrado». ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 26. *Ejecución de RF30RegistrarAjusteDeSaldoTest en rojo y en verde*

RF31 Registrar abono de cliente · RF31RegistrarAbonoDeClienteTest

```
@DisplayName("RF31 Registrar abono de cliente")
class RF31RegistrarAbonoDeClienteTest extends TddBase {

    /** RF31-CP01 (E1). Se aplican S/ 640 a la venta 1 y S/ 160 a la venta 2; nuevo saldo S/
572.00. */
    @Test
    @DisplayName("RF31-CP01 Abonar S/ 800 en efectivo.")
    void rf31_cp01_aplicaALaMasAntigua() throws Exception {
        // ARRANGE
        prepararDeudaSaborNorteno();
        when(abonoDao.insertar(anyInt(), anyInt(), any(), any(),
anyString())).thenReturn(5);
        // ACT
        BigDecimal saldo = creditoService().registrarAbono(2, d("800"), "EFECTIVO", 1);
        // ASSERT
        assertEquals(d("572.00"), saldo);
        verify(abonoDao).aplicar(5, 1L, d("640.00"));
        verify(abonoDao).aplicar(5, 2L, d("160.00"));
    }

    /** RF31-CP02 (E2). Saldo S/ 0.00 y se marcan sus compromisos como cumplidos. */
    @Test
    @DisplayName("RF31-CP02 Abonar S/ 1,372 por Yape.")
    void rf31_cp02_abonoTotalCumpleCompromisos() throws Exception {
        // ARRANGE
        prepararDeudaSaborNorteno();
        // ACT
        BigDecimal saldo = creditoService().registrarAbono(2, d("1372"), "YAPE", 1);
        // ASSERT
        assertEquals(d("0.00"), saldo);
        verify(compromisoDao).marcarCumplidos(2);
    }

    /** RF31-CP03 (E3). Mensaje «El abono supera el saldo (S/ 1372.00)». */
    @Test
    @DisplayName("RF31-CP03 Abonar S/ 2,000.")
    void rf31_cp03_superaSaldo() throws Exception {
        // ARRANGE
        prepararDeudaSaborNorteno();
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
creditoService().registrarAbono(2, d("2000"), "EFECTIVO", 1));
        // ASSERT
```

```
        assertEquals("El abono supera el saldo (S/ 1372.00)", e.getMessage());
        verify(abonoDao, never()).insertar(anyInt(), anyInt(), any(), any(), anyString());
    }

    /** RF31-CP04 (E4). Mensaje «Elija el medio de pago». */
    @Test
    @DisplayName("RF31-CP04 Abonar S/ 100 sin medio de pago.")
    void rf31_cp04_faltaMedio() throws Exception {
        // ARRANGE
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
creditoService().registrarAbono(2, d("100"), null, 1));
        // ASSERT
        assertEquals("Elija el medio de pago", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF31RegistrarAbonoDeClienteTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF31 Registrar abono de cliente ✔
│     ├─ RF31-CP02 Abonar S/ 1,372 por Yape. ✘ Pendiente de implementar
│     ├─ RF31-CP01 Abonar S/ 800 en efectivo. ✘ Pendiente de implementar
│     ├─ RF31-CP03 Abonar S/ 2,000. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>
│     └─ RF31-CP04 Abonar S/ 100 sin medio de pago. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationEx…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF31RegistrarAbonoDeClienteTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF31 Registrar abono de cliente ✔
│     ├─ RF31-CP02 Abonar S/ 1,372 por Yape. ✔
│     ├─ RF31-CP01 Abonar S/ 800 en efectivo. ✔
│     ├─ RF31-CP03 Abonar S/ 2,000. ✔
│     └─ RF31-CP04 Abonar S/ 100 sin medio de pago. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 27. *Ejecución de RF31RegistrarAbonoDeClienteTest en rojo y en verde*

RF33 Calcular antigüedad de la deuda · RF33CalcularAntiguedadDeLaDeudaTest

```
@DisplayName("RF33 Calcular antigüedad de la deuda")
class RF33CalcularAntiguedadDeLaDeudaTest extends TddBase {

    /** RF33-CP01 (E1). S/ 732 en «8–15 días» y S/ 640 en «16–30 días». */
    @Test
    @DisplayName("RF33-CP01 Calcular la antigüedad del cliente 2.")
    void rf33_cp01_tramosPorDias() throws Exception {
        // ARRANGE
        prepararDeudaSaborNorteno();
        // ACT
        List<Antiguedad> a = creditoService().antiguedad(2);
        // ASSERT
        assertEquals(d("0.00"), a.get(0).monto());
        assertEquals(d("732.00"), a.get(1).monto());
        assertEquals(d("640.00"), a.get(2).monto());
    }

    /** RF33-CP02 (E2). «0–7 días», «8–15 días», «16–30 días» y «Más de 30 días». */
    @Test
    @DisplayName("RF33-CP02 Clasificar 7, 8, 30 y 31 días.")
    void rf33_cp02_limitesDeTramos() throws Exception {
        // ARRANGE
        long[] dias = {7, 8, 30, 31};
```

```
        // ACT
        String[] tramos = {Calculos.tramo(dias[0]), Calculos.tramo(dias[1]),
Calculos.tramo(dias[2]), Calculos.tramo(dias[3])};
        // ASSERT
        assertArrayEquals(new String[]{"0–7 días", "8–15 días", "16–30 días", "Más de 30
días"}, tramos);
    }

    /** RF33-CP03 (E3). Solo «8–15 días» tiene S/ 732; «16–30 días» queda en S/ 0.00. */
    @Test
    @DisplayName("RF33-CP03 Calcular la antigüedad.")
    void rf33_cp03_ventaPagadaNoCuenta() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(2)).thenReturn(Optional.of(cliente(2, "Restaurante Sabor
Norteño", "1500.00", 7)));
        when(ventaDao.pendientesCliente(2)).thenReturn(List.of(
                new PendienteVenta(1L, Fechas.ahora().minusDays(20), d("640.00"),
d("640.00")),
                new PendienteVenta(2L, Fechas.ahora().minusDays(10), d("732.00"),
d("0.00"))));
        // ACT
        List<Antiguedad> a = creditoService().antiguedad(2);
        // ASSERT
        assertEquals(d("732.00"), a.get(1).monto());
        assertEquals(d("0.00"), a.get(2).monto());
    }

    /** RF33-CP04 (E4). Mensaje «El cliente no existe». */
    @Test
    @DisplayName("RF33-CP04 Calcular la antigüedad del cliente 99.")
    void rf33_cp04_clienteNoExiste() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(99)).thenReturn(Optional.empty());
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
creditoService().antiguedad(99));
        // ASSERT
        assertEquals("El cliente no existe", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF33CalcularAntiguedadDeLaDeudaTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF33 Calcular antigüedad de la deuda ✔
│     ├─ RF33-CP02 Clasificar 7, 8, 30 y 31 días. ✘ Pendiente de implementar
│     ├─ RF33-CP04 Calcular la antigüedad del cliente 99. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOpera…
│     ├─ RF33-CP03 Calcular la antigüedad. ✘ Pendiente de implementar
│     └─ RF33-CP01 Calcular la antigüedad del cliente 2. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF33CalcularAntiguedadDeLaDeudaTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF33 Calcular antigüedad de la deuda ✔
│     ├─ RF33-CP02 Clasificar 7, 8, 30 y 31 días. ✔
│     ├─ RF33-CP04 Calcular la antigüedad del cliente 99. ✔
│     ├─ RF33-CP03 Calcular la antigüedad. ✔
│     └─ RF33-CP01 Calcular la antigüedad del cliente 2. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 28. *Ejecución de RF33CalcularAntiguedadDeLaDeudaTest en rojo y en verde*

RF34 Alertar deudas vencidas · RF34AlertarDeudasVencidasTest

```
@DisplayName("RF34 Alertar deudas vencidas")
```

```
class RF34AlertarDeudasVencidasTest extends TddBase {

    /** RF34-CP01 (E1). Solo aparece el cliente 2. */
    @Test
    @DisplayName("RF34-CP01 Consultar vencidos.")
    void rf34_cp01_soloVencidos() throws Exception {
        // ARRANGE
        when(ventaDao.saldosClientes(any())).thenReturn(List.of(saldoCliente(1, "228.00",
"1500.00", "0.00"), saldoCliente(2, "1372.00", "1500.00", "1372.00")));
        // ACT
        List<SaldoCliente> v = creditoService().vencidos();
        // ASSERT
        assertEquals(1, v.size());
        assertEquals(2, v.get(0).idCliente());
    }

    /** RF34-CP02 (E2). Lista vacía. */
    @Test
    @DisplayName("RF34-CP02 Consultar vencidos.")
    void rf34_cp02_ningunoVencido() throws Exception {
        // ARRANGE
        when(ventaDao.saldosClientes(any())).thenReturn(List.of(saldoCliente(1, "228.00",
"1500.00", "0.00")));
        // ACT
        List<SaldoCliente> v = creditoService().vencidos();
        // ASSERT
        assertTrue(v.isEmpty());
    }

    /** RF34-CP03 (E3). Aparece el cliente 3 por estar sobre su límite. */
    @Test
    @DisplayName("RF34-CP03 Consultar vencidos.")
    void rf34_cp03_sobreLimite() throws Exception {
        // ARRANGE
        when(ventaDao.saldosClientes(any())).thenReturn(List.of(saldoCliente(3, "900.00",
"800.00", "0.00")));
        // ACT
        List<SaldoCliente> v = creditoService().vencidos();
        // ASSERT
        assertEquals(1, v.size());
        assertTrue(v.get(0).sobreLimite());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF34AlertarDeudasVencidasTest
FASE ROJA: clases Service sin implementar

+
├─ JUnit Jupiter ✔
│  └─ RF34 Alertar deudas vencidas ✔
│     ├─ RF34-CP02 Consultar vencidos. ✘ Pendiente de implementar
│     ├─ RF34-CP01 Consultar vencidos. ✘ Pendiente de implementar
│     └─ RF34-CP03 Consultar vencidos. ✘ Pendiente de implementar

[   3 tests found ]  [   0 tests successful ]  [   3 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF34AlertarDeudasVencidasTest
FASE VERDE: código implementado

+
├─ JUnit Jupiter ✔
│  └─ RF34 Alertar deudas vencidas ✔
│     ├─ RF34-CP02 Consultar vencidos. ✔
│     ├─ RF34-CP01 Consultar vencidos. ✔
│     └─ RF34-CP03 Consultar vencidos. ✔

[   3 tests found ]  [   3 tests successful ]  [   0 tests failed ]
```

Figura 29. *Ejecución de RF34AlertarDeudasVencidasTest en rojo y en verde*

RF35 Registrar compromiso de pago · RF35RegistrarCompromisoDePagoTest

```
@DisplayName("RF35 Registrar compromiso de pago")
class RF35RegistrarCompromisoDePagoTest extends TddBase {

    /** RF35-CP01 (E1). Se guarda el compromiso y la bitácora COMPROMISO_PAGO. */
    @Test
    @DisplayName("RF35-CP01 Registrar compromiso de S/ 500 para dentro de 3 días.")
    void rf35_cp01_registraCompromiso() throws Exception {
        // ARRANGE
        prepararDeudaSaborNorteno();
        LocalDate fecha = Fechas.hoy().plusDays(3);
        // ACT
        creditoService().registrarCompromiso(2, fecha, d("500"), 1);
        // ASSERT
        verify(compromisoDao).insertar(2, fecha, d("500.00"));
        verify(auditoria).registrar(eq(1), eq("COMPROMISO_PAGO"), startsWith("Restaurante
Sabor Norteño · S/ 500.00"));
    }

    /** RF35-CP02 (E2). Antes de listar se marcan como incumplidos los vencidos al día de
hoy. */
    @Test
    @DisplayName("RF35-CP02 Abrir la lista de compromisos.")
    void rf35_cp02_marcaIncumplidos() throws Exception {
        // ARRANGE
        when(compromisoDao.listarVigentes()).thenReturn(List.of());
        // ACT
        creditoService().compromisos();
        // ASSERT
        InOrder orden = inOrder(compromisoDao);
        orden.verify(compromisoDao).marcarIncumplidos(Fechas.hoy());
        orden.verify(compromisoDao).listarVigentes();
    }

    /** RF35-CP03 (E3). Mensaje «La fecha debe ser hoy o posterior». */
    @Test
    @DisplayName("RF35-CP03 Registrar compromiso con fecha de ayer.")
    void rf35_cp03_fechaPasada() throws Exception {
        // ARRANGE
        LocalDate ayer = Fechas.hoy().minusDays(1);
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
creditoService().registrarCompromiso(2, ayer, d("500"), 1));
        // ASSERT
        assertEquals("La fecha debe ser hoy o posterior", e.getMessage());
    }

    /** RF35-CP04 (E4). Mensaje «El cliente no tiene deudas». */
    @Test
    @DisplayName("RF35-CP04 Registrar compromiso de S/ 100.")
    void rf35_cp04_sinDeudas() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(3)).thenReturn(Optional.of(cliente(3, "Pollería Don Lucho",
"800.00", 7)));
        when(ventaDao.pendientesCliente(3)).thenReturn(List.of());
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> creditoService().registrarCompromiso(3, Fechas.hoy(), d("100"), 1));
        // ASSERT
        assertEquals("El cliente no tiene deudas", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF35RegistrarCompromisoDePagoTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF35 Registrar compromiso de pago ✔
│     ├─ RF35-CP03 Registrar compromiso con fecha de ayer. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOper…
│     ├─ RF35-CP01 Registrar compromiso de S/ 500 para dentro de 3 días. ✘ Pendiente de implementar
│     ├─ RF35-CP02 Abrir la lista de compromisos. ✘ Pendiente de implementar
│     └─ RF35-CP04 Registrar compromiso de S/ 100. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExc…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF35RegistrarCompromisoDePagoTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF35 Registrar compromiso de pago ✔
│     ├─ RF35-CP03 Registrar compromiso con fecha de ayer. ✔
│     ├─ RF35-CP01 Registrar compromiso de S/ 500 para dentro de 3 días. ✔
│     ├─ RF35-CP02 Abrir la lista de compromisos. ✔
│     └─ RF35-CP04 Registrar compromiso de S/ 100. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 30*. Ejecución de RF35RegistrarCompromisoDePagoTest en rojo y en verde*

RF36 Generar estado de cuenta · RF36GenerarEstadoDeCuentaTest

```
@DisplayName("RF36 Generar estado de cuenta")
class RF36GenerarEstadoDeCuentaTest extends TddBase {

    /** RF36-CP01 (E1). Saldo inicial S/ 100.00, movimientos con saldo 328.00 y 278.00;
saldo final S/ 278.00. */
    @Test
    @DisplayName("RF36-CP01 Generar del 01/10 al 31/10.")
    void rf36_cp01_saldoAcumulado() throws Exception {
        // ARRANGE
        LocalDate desde = LocalDate.of(2026, 10, 1), hasta = LocalDate.of(2026, 10, 31);
        when(clienteDao.buscar(1)).thenReturn(Optional.of(cliente(1, "Cevichería El Muelle",
"1500.00", 15)));
        when(ventaDao.ventasCreditoAntesDe(eq(1), any())).thenReturn(d("100.00"));
        when(abonoDao.abonosAntesDe(eq(1), any())).thenReturn(d("0.00"));
        when(ventaDao.ventasCreditoCliente(eq(1), any(), any())).thenReturn(List.of(
                new VentaVista(3, "u3", LocalDateTime.of(2026, 10, 21, 8, 0), 1, "Cevichería
El Muelle", "CREDITO", null, false, "VIGENTE", null, d("228.00"), "vendedor", null,
false)));
        when(abonoDao.abonosCliente(eq(1), any(), any())).thenReturn(List.of(
                new AbonoVista(1, LocalDateTime.of(2026, 10, 25, 9, 0), "Cevichería El
Muelle", d("50.00"), "YAPE", "VIGENTE")));
        // ACT
        EstadoCuenta ec = creditoService().estadoCuenta(1, desde, hasta);
        // ASSERT
        assertEquals(d("100.00"), ec.saldoInicial());
        assertEquals(d("328.00"), ec.movimientos().get(0).saldo());
        assertEquals(d("278.00"), ec.movimientos().get(1).saldo());
        assertEquals(d("278.00"), ec.saldoFinal());
    }

    /** RF36-CP02 (E2). Sin movimientos; saldo final = saldo inicial = S/ 0.00. */
    @Test
    @DisplayName("RF36-CP02 Generar octubre.")
    void rf36_cp02_sinMovimientos() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(3)).thenReturn(Optional.of(cliente(3, "Pollería Don Lucho",
"800.00", 7)));
        when(ventaDao.ventasCreditoAntesDe(eq(3), any())).thenReturn(d("0.00"));
        when(abonoDao.abonosAntesDe(eq(3), any())).thenReturn(d("0.00"));
        // ACT
        EstadoCuenta ec = creditoService().estadoCuenta(3, LocalDate.of(2026, 10, 1),
LocalDate.of(2026, 10, 31));
        // ASSERT
```

```
        assertTrue(ec.movimientos().isEmpty());
        assertEquals(ec.saldoInicial(), ec.saldoFinal());
    }

    /** RF36-CP03 (E3). Mensaje «Revise el rango de fechas». */
    @Test
    @DisplayName("RF36-CP03 Generar del 31/10 al 01/10.")
    void rf36_cp03_rangoInvertido() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(1)).thenReturn(Optional.of(cliente(1, "Cevichería El Muelle",
"1500.00", 15)));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> creditoService().estadoCuenta(1, LocalDate.of(2026, 10, 31),
LocalDate.of(2026, 10, 1)));
        // ASSERT
        assertEquals("Revise el rango de fechas", e.getMessage());
    }

    /** RF36-CP04 (E4). Mensaje «Elija un periodo de hasta un año». */
    @Test
    @DisplayName("RF36-CP04 Generar del 01/01/2025 al 31/10/2026.")
    void rf36_cp04_periodoMayorAUnAnio() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(1)).thenReturn(Optional.of(cliente(1, "Cevichería El Muelle",
"1500.00", 15)));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> creditoService().estadoCuenta(1, LocalDate.of(2025, 1, 1),
LocalDate.of(2026, 10, 31)));
        // ASSERT
        assertEquals("Elija un periodo de hasta un año", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF36GenerarEstadoDeCuentaTest
FASE ROJA: clases Service sin implementar

.
├─ JUnit Jupiter ✔
│  └─ RF36 Generar estado de cuenta ✔
│     ├─ RF36-CP02 Generar octubre. ✘ Pendiente de implementar
│     ├─ RF36-CP04 Generar del 01/01/2025 al 31/10/2026. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperat…
│     ├─ RF36-CP03 Generar del 31/10 al 01/10. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExcepti…
│     └─ RF36-CP01 Generar del 01/10 al 31/10. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF36GenerarEstadoDeCuentaTest
FASE VERDE: código implementado

.
├─ JUnit Jupiter ✔
│  └─ RF36 Generar estado de cuenta ✔
│     ├─ RF36-CP02 Generar octubre. ✔
│     ├─ RF36-CP04 Generar del 01/01/2025 al 31/10/2026. ✔
│     ├─ RF36-CP03 Generar del 31/10 al 01/10. ✔
│     └─ RF36-CP01 Generar del 01/10 al 31/10. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 31. *Ejecución de RF36GenerarEstadoDeCuentaTest en rojo y en verde*

RF38 Registrar gasto menor · RF38RegistrarGastoMenorTest

```
@DisplayName("RF38 Registrar gasto menor")
class RF38RegistrarGastoMenorTest extends TddBase {

    /** RF38-CP01 (E1). Se guarda el gasto de hoy. */
    @Test
    @DisplayName("RF38-CP01 Registrar HIELO S/ 15 en efectivo.")
    void rf38_cp01_registraGasto() throws Exception {
```

```
        // ARRANGE
        // ACT
        cajaService().registrarGasto("HIELO", null, d("15"), "EFECTIVO", 2);
        // ASSERT
        verify(gastoDao).insertar(2, Fechas.hoy(), "HIELO", null, d("15.00"), "EFECTIVO");
        verify(auditoria).registrar(2, "REGISTRAR_GASTO", "HIELO · S/ 15.00 · EFECTIVO");
    }

    /** RF38-CP02 (E2). Se guarda con su descripción. */
    @Test
    @DisplayName("RF38-CP02 Registrar OTRO «Cuchillo nuevo» S/ 25 por Yape.")
    void rf38_cp02_otroConDescripcion() throws Exception {
        // ARRANGE
        // ACT
        cajaService().registrarGasto("OTRO", "Cuchillo nuevo", d("25"), "YAPE", 2);
        // ASSERT
        verify(gastoDao).insertar(2, Fechas.hoy(), "OTRO", "Cuchillo nuevo", d("25.00"),
"YAPE");
    }

    /** RF38-CP03 (E3). Mensaje «Ingrese un monto mayor que cero». */
    @Test
    @DisplayName("RF38-CP03 Registrar BOLSAS S/ 0.")
    void rf38_cp03_montoCero() throws Exception {
        // ARRANGE
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
cajaService().registrarGasto("BOLSAS", null, d("0"), "EFECTIVO", 2));
        // ASSERT
        assertEquals("Ingrese un monto mayor que cero", e.getMessage());
    }

    /** RF38-CP04 (E4). Mensaje «Describa el gasto». */
    @Test
    @DisplayName("RF38-CP04 Registrar OTRO sin descripción.")
    void rf38_cp04_otroSinDescripcion() throws Exception {
        // ARRANGE
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
cajaService().registrarGasto("OTRO", " ", d("10"), "EFECTIVO", 2));
        // ASSERT
        assertEquals("Describa el gasto", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF38RegistrarGastoMenorTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF38 Registrar gasto menor ✔
│     ├─ RF38-CP04 Registrar OTRO sin descripción. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExc…
│     ├─ RF38-CP03 Registrar BOLSAS S/ 0. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationException>
│     ├─ RF38-CP01 Registrar HIELO S/ 15 en efectivo. ✘ Pendiente de implementar
│     └─ RF38-CP02 Registrar OTRO «Cuchillo nuevo» S/ 25 por Yape. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF38RegistrarGastoMenorTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF38 Registrar gasto menor ✔
│     ├─ RF38-CP04 Registrar OTRO sin descripción. ✔
│     ├─ RF38-CP03 Registrar BOLSAS S/ 0. ✔
│     ├─ RF38-CP01 Registrar HIELO S/ 15 en efectivo. ✔
│     └─ RF38-CP02 Registrar OTRO «Cuchillo nuevo» S/ 25 por Yape. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 32. *Ejecución de RF38RegistrarGastoMenorTest en rojo y en verde*

RF39 Realizar cuadre de caja · RF39RealizarCuadreDeCajaTest

```
@DisplayName("RF39 Realizar cuadre de caja")
class RF39RealizarCuadreDeCajaTest extends TddBase {

    /** RF39-CP01 (E1). Efectivo esperado S/ 520.00, diferencia S/ 0.00; se guarda el
cuadre. */
    @Test
    @DisplayName("RF39-CP01 Cuadrar con S/ 520 contados y S/ 200 revisados.")
    void rf39_cp01_cuadreCoincide() throws Exception {
        // ARRANGE
        LocalDate hoy = Fechas.hoy();
        prepararMovimientos(hoy, "450.00", "200.00", "100.00", "0.00", "30.00", "0.00");
        when(cuadreDao.buscar(hoy)).thenReturn(Optional.empty(), Optional.empty(),
Optional.of(cuadreVista(hoy)));
        // ACT
        cajaService().realizarCuadre(hoy, d("520"), d("200"), null, 1);
        // ASSERT
        verify(cuadreDao).insertar(eq(1), eq(hoy), eq(d("520.00")), eq(d("520.00")),
eq(d("200.00")), eq(d("200.00")), isNull(), any());
    }

    /** RF39-CP02 (E2). Se guarda: S/ -3.00 está dentro de la tolerancia. */
    @Test
    @DisplayName("RF39-CP02 Cuadrar con S/ 517 sin motivo.")
    void rf39_cp02_diferenciaMenorTolerancia() throws Exception {
        // ARRANGE
        LocalDate hoy = Fechas.hoy();
        prepararMovimientos(hoy, "450.00", "200.00", "100.00", "0.00", "30.00", "0.00");
        when(cuadreDao.buscar(hoy)).thenReturn(Optional.empty(), Optional.empty(),
Optional.of(cuadreVista(hoy)));
        // ACT
        cajaService().realizarCuadre(hoy, d("517"), d("200"), "", 1);
        // ASSERT
        verify(cuadreDao).insertar(eq(1), eq(hoy), eq(d("520.00")), eq(d("517.00")), any(),
any(), isNull(), any());
    }

    /** RF39-CP03 (E3). Mensaje «Diferencia de S/ -20.00 en efectivo y S/ 0.00 en digital:
indique el motivo». */
    @Test
    @DisplayName("RF39-CP03 Cuadrar con S/ 500 sin motivo.")
    void rf39_cp03_diferenciaExigeMotivo() throws Exception {
        // ARRANGE
        LocalDate hoy = Fechas.hoy();
        prepararMovimientos(hoy, "450.00", "200.00", "100.00", "0.00", "30.00", "0.00");
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
cajaService().realizarCuadre(hoy, d("500"), d("200"), null, 1));
        // ASSERT
        assertEquals("Diferencia de S/ -20.00 en efectivo y S/ 0.00 en digital: indique el
motivo", e.getMessage());
    }

    /** RF39-CP04 (E4). Mensaje «El día 27/10/2026 ya tiene cuadre». */
    @Test
    @DisplayName("RF39-CP04 Cuadrar otra vez el 27/10/2026.")
    void rf39_cp04_diaYaCuadrado() throws Exception {
        // ARRANGE
        LocalDate dia = LocalDate.of(2026, 10, 27);
        when(cuadreDao.buscar(dia)).thenReturn(Optional.of(cuadreVista(dia)));
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
cajaService().realizarCuadre(dia, d("100"), d("0"), null, 1));
        // ASSERT
        assertEquals("El día 27/10/2026 ya tiene cuadre", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF39RealizarCuadreDeCajaTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF39 Realizar cuadre de caja ✔
│     ├─ RF39-CP04 Cuadrar otra vez el 27/10/2026. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExc…
│     ├─ RF39-CP03 Cuadrar con S/ 500 sin motivo. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExce…
│     ├─ RF39-CP01 Cuadrar con S/ 520 contados y S/ 200 revisados. ✘ Pendiente de implementar
│     └─ RF39-CP02 Cuadrar con S/ 517 sin motivo. ✘ Pendiente de implementar
╵

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF39RealizarCuadreDeCajaTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF39 Realizar cuadre de caja ✔
│     ├─ RF39-CP04 Cuadrar otra vez el 27/10/2026. ✔
│     ├─ RF39-CP03 Cuadrar con S/ 500 sin motivo. ✔
│     ├─ RF39-CP01 Cuadrar con S/ 520 contados y S/ 200 revisados. ✔
│     └─ RF39-CP02 Cuadrar con S/ 517 sin motivo. ✔
╵

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 33. *Ejecución de RF39RealizarCuadreDeCajaTest en rojo y en verde*

RF40 Calcular ganancia del día · RF40CalcularGananciaDelDiaTest

```
@DisplayName("RF40 Calcular ganancia del día")
class RF40CalcularGananciaDelDiaTest extends TddBase {

    /** RF40-CP01 (E1). Ganancia S/ 305.00. */
    @Test
    @DisplayName("RF40-CP01 Calcular la ganancia.")
    void rf40_cp01_formulaGanancia() throws Exception {
        // ARRANGE
        BigDecimal ventas = d("1200.00"), costo = d("850.00"), perdidas = d("0.00"), gastos
= d("45.00");
        // ACT
        BigDecimal g = Calculos.ganancia(ventas, costo, perdidas, gastos);
        // ASSERT
        assertEquals(d("305.00"), g);
    }

    /** RF40-CP02 (E2). Ganancia S/ 279.00 y margen 23.3 %. */
    @Test
    @DisplayName("RF40-CP02 Calcular la ganancia del día.")
    void rf40_cp02_perdidasRestan() throws Exception {
        // ARRANGE
        LocalDate hoy = Fechas.hoy();
        when(ventaDao.cantidadVentas(hoy, hoy)).thenReturn(18);
        when(ventaDao.totalVentas(hoy, hoy)).thenReturn(d("1200.00"));
        when(ventaDao.costoVendido(hoy, hoy)).thenReturn(d("850.00"));
        when(loteDao.perdidas(hoy, hoy)).thenReturn(d("26.00"));
        when(gastoDao.total(hoy, hoy)).thenReturn(d("45.00"));
        // ACT
        Ganancia g = cajaService().ganancia(hoy);
        // ASSERT
        assertEquals(d("279.00"), g.ganancia());
        assertEquals(new BigDecimal("23.3"), g.margen());
    }

    /** RF40-CP03 (E3). Resultado marcado como provisional. */
    @Test
    @DisplayName("RF40-CP03 Calcular la ganancia de hoy.")
    void rf40_cp03_provisionalSinCuadre() throws Exception {
        // ARRANGE
        LocalDate hoy = Fechas.hoy();
        when(cuadreDao.buscar(hoy)).thenReturn(Optional.empty());
        // ACT
        Ganancia g = cajaService().ganancia(hoy);
```

```
        // ASSERT
        assertTrue(g.provisional());
    }

    /** RF40-CP04 (E4). Margen 0.00 %. */
    @Test
    @DisplayName("RF40-CP04 Calcular el margen.")
    void rf40_cp04_margenSinVentas() throws Exception {
        // ARRANGE
        BigDecimal ganancia = d("-30.00"), ventas = d("0.00");
        // ACT
        BigDecimal margen = Calculos.margen(ganancia, ventas);
        // ASSERT
        assertEquals(d("0.00"), margen);
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF40CalcularGananciaDelDiaTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF40 Calcular ganancia del día ✔
│     ├─ RF40-CP04 Calcular el margen. ✘ Pendiente de implementar
│     ├─ RF40-CP02 Calcular la ganancia del día. ✘ Pendiente de implementar
│     ├─ RF40-CP03 Calcular la ganancia de hoy. ✘ Pendiente de implementar
│     └─ RF40-CP01 Calcular la ganancia. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF40CalcularGananciaDelDiaTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF40 Calcular ganancia del día ✔
│     ├─ RF40-CP04 Calcular el margen. ✔
│     ├─ RF40-CP02 Calcular la ganancia del día. ✔
│     ├─ RF40-CP03 Calcular la ganancia de hoy. ✔
│     └─ RF40-CP01 Calcular la ganancia. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 34. *Ejecución de RF40CalcularGananciaDelDiaTest en rojo y en verde*

RF42 Registrar factura emitida · RF42RegistrarFacturaEmitidaTest

```
@DisplayName("RF42 Registrar factura emitida")
class RF42RegistrarFacturaEmitidaTest extends TddBase {

    /** RF42-CP01 (E1). Se guarda la factura y la bitácora «E001-000154 · Cevichería El
Muelle · S/ 228.00». */
    @Test
    @DisplayName("RF42-CP01 Registrar E001-000154 del 27/10/2026 por S/ 228.00.")
    void rf42_cp01_registraFactura() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(1)).thenReturn(Optional.of(cliente(1, "Cevichería El Muelle",
"1500.00", 15)));
        when(facturaDao.insertar(anyInt(), anyString(), anyInt(), any(), any(),
anyBoolean())).thenReturn(1);
        // ACT
        facturaService().registrar(1, "e001", 154, LocalDate.of(2026, 10, 27), d("228"),
false, 3);
        // ASSERT
        verify(facturaDao).insertar(1, "E001", 154, LocalDate.of(2026, 10, 27), d("228.00"),
false);
        verify(auditoria).registrar(3, "REGISTRAR_FACTURA", "E001-000154 · Cevichería El
Muelle · S/ 228.00");
    }
```

```
    /** RF42-CP02 (E2). Se guarda con detracción = sí. */
    @Test
    @DisplayName("RF42-CP02 Registrar F001-000020 por S/ 732 con detracción.")
    void rf42_cp02_conDetraccion() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(2)).thenReturn(Optional.of(cliente(2, "Restaurante Sabor
Norteño", "1500.00", 7)));
        // ACT
        facturaService().registrar(2, "F001", 20, Fechas.hoy(), d("732"), true, 3);
        // ASSERT
        verify(facturaDao).insertar(2, "F001", 20, Fechas.hoy(), d("732.00"), true);
    }

    /** RF42-CP03 (E3). Mensaje «La factura E001-000154 ya está registrada». */
    @Test
    @DisplayName("RF42-CP03 Registrar otra vez E001-000154.")
    void rf42_cp03_facturaDuplicada() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(1)).thenReturn(Optional.of(cliente(1, "Cevichería El Muelle",
"1500.00", 15)));
        when(facturaDao.existe("E001", 154)).thenReturn(true);
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> facturaService().registrar(1, "E001", 154, Fechas.hoy(), d("228"),
false, 3));
        // ASSERT
        assertEquals("La factura E001-000154 ya está registrada", e.getMessage());
    }

    /** RF42-CP04 (E4). Mensaje «El cliente no tiene RUC; no se puede registrar factura». */
    @Test
    @DisplayName("RF42-CP04 Registrar una factura a la pollería.")
    void rf42_cp04_clienteSinRuc() throws Exception {
        // ARRANGE
        when(clienteDao.buscar(3)).thenReturn(Optional.of(new Cliente(3, "Pollería Don
Lucho", null, null, d("800.00"), 7, "ACTIVO")));
        // ACT
        NegocioException e = assertThrows(NegocioException.class,
                () -> facturaService().registrar(3, "E001", 155, Fechas.hoy(), d("50"),
false, 3));
        // ASSERT
        assertEquals("El cliente no tiene RUC; no se puede registrar factura",
e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF42RegistrarFacturaEmitidaTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF42 Registrar factura emitida ✔
│     ├─ RF42-CP04 Registrar una factura a la pollería. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperati…
│     ├─ RF42-CP02 Registrar F001-000020 por S/ 732 con detracción. ✘ Pendiente de implementar
│     ├─ RF42-CP01 Registrar E001-000154 del 27/10/2026 por S/ 228.00. ✘ Pendiente de implementar
│     └─ RF42-CP03 Registrar otra vez E001-000154. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExc…
╵

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF42RegistrarFacturaEmitidaTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF42 Registrar factura emitida ✔
│     ├─ RF42-CP04 Registrar una factura a la pollería. ✔
│     ├─ RF42-CP02 Registrar F001-000020 por S/ 732 con detracción. ✔
│     ├─ RF42-CP01 Registrar E001-000154 del 27/10/2026 por S/ 228.00. ✔
│     └─ RF42-CP03 Registrar otra vez E001-000154. ✔
╵

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 35*. Ejecución de RF42RegistrarFacturaEmitidaTest en rojo y en verde*

RF43 Asociar ventas a factura · RF43AsociarVentasAFacturaTest

```
@DisplayName("RF43 Asociar ventas a factura")
class RF43AsociarVentasAFacturaTest extends TddBase {

    /** RF43-CP01 (E1). Se asigna la factura 1 a la venta 3; diferencia S/ 0.00. */
    @Test
    @DisplayName("RF43-CP01 Asociar la venta 3.")
    void rf43_cp01_asociaVentas() throws Exception {
        // ARRANGE
        prepararFactura(1, 1, "228.00");
        when(ventaDao.sinFacturaCliente(1)).thenReturn(List.of(venta(3, 1, "CREDITO",
"228.00", "VIGENTE", null)));
        // ACT
        BigDecimal dif = facturaService().asociar(1, List.of(3L), false, 3);
        // ASSERT
        assertEquals(d("0.00"), dif);
        verify(ventaDao).asignarFactura(List.of(3L), 1);
    }

    /** RF43-CP02 (E2). Pide confirmar: «La suma de las ventas (S/ 200.00) difiere del
importe pendiente de la factura en S/ 28.00. ¿Confirmar?». */
    @Test
    @DisplayName("RF43-CP02 Asociar sin confirmar.")
    void rf43_cp02_diferenciaPideConfirmar() throws Exception {
        // ARRANGE
        prepararFactura(1, 1, "228.00");
        when(ventaDao.sinFacturaCliente(1)).thenReturn(List.of(venta(3, 1, "CREDITO",
"200.00", "VIGENTE", null)));
        // ACT
        ConfirmacionException e = assertThrows(ConfirmacionException.class, () ->
facturaService().asociar(1, List.of(3L), false, 3));
        // ASSERT
        assertEquals("La suma de las ventas (S/ 200.00) difiere del importe pendiente de la
factura en S/ 28.00. ¿Confirmar?", e.getMessage());
        verify(ventaDao, never()).asignarFactura(any(), anyInt());
    }

    /** RF43-CP03 (E3). Mensaje «La venta 0001 ya está asociada a otra factura o no es de
este cliente». */
    @Test
    @DisplayName("RF43-CP03 Asociar la venta 1 a la factura 1.")
    void rf43_cp03_ventaNoDisponible() throws Exception {
        // ARRANGE
        prepararFactura(1, 1, "228.00");
```

```
        when(ventaDao.sinFacturaCliente(1)).thenReturn(List.of());
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
facturaService().asociar(1, List.of(1L), false, 3));
        // ASSERT
        assertEquals("La venta 0001 ya está asociada a otra factura o no es de este
cliente", e.getMessage());
    }

    /** RF43-CP04 (E4). Mensaje «Seleccione las ventas que sustenta la factura». */
    @Test
    @DisplayName("RF43-CP04 Asociar sin marcar ventas.")
    void rf43_cp04_sinVentas() throws Exception {
        // ARRANGE
        prepararFactura(1, 1, "228.00");
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
facturaService().asociar(1, List.of(), false, 3));
        // ASSERT
        assertEquals("Seleccione las ventas que sustenta la factura", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF43AsociarVentasAFacturaTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF43 Asociar ventas a factura ✔
│     ├─ RF43-CP01 Asociar la venta 3. ✘ Pendiente de implementar
│     ├─ RF43-CP04 Asociar sin marcar ventas. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExceptio…
│     ├─ RF43-CP03 Asociar la venta 1 a la factura 1. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperation…
│     └─ RF43-CP02 Asociar sin confirmar. ✘ Unexpected exception type thrown, expected: <ConfirmacionException> but was: <UnsupportedOperationExcepti…

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF43AsociarVentasAFacturaTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF43 Asociar ventas a factura ✔
│     ├─ RF43-CP01 Asociar la venta 3. ✔
│     ├─ RF43-CP04 Asociar sin marcar ventas. ✔
│     ├─ RF43-CP03 Asociar la venta 1 a la factura 1. ✔
│     └─ RF43-CP02 Asociar sin confirmar. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 36. *Ejecución de RF43AsociarVentasAFacturaTest en rojo y en verde*

RF45 Generar cierre mensual · RF45GenerarCierreMensualTest

```
@DisplayName("RF45 Generar cierre mensual")
class RF45GenerarCierreMensualTest extends TddBase {

    /** RF45-CP01 (E1). Ganancia S/ 5,800.00 y ventas sin factura S/ 16,000.00. */
    @Test
    @DisplayName("RF45-CP01 Generar el cierre de septiembre.")
    void rf45_cp01_resumenDelMes() throws Exception {
        // ARRANGE
        YearMonth mes = YearMonth.of(2026, 9);
        LocalDate desde = mes.atDay(1), hasta = mes.atEndOfMonth();
        when(ventaDao.totalVentas(desde, hasta)).thenReturn(d("25000.00"));
        when(ventaDao.costoVendido(desde, hasta)).thenReturn(d("18000.00"));
        when(loteDao.perdidas(desde, hasta)).thenReturn(d("300.00"));
        when(gastoDao.total(desde, hasta)).thenReturn(d("900.00"));
        when(facturaDao.totalEmitido(desde, hasta)).thenReturn(d("9000.00"));
        // ACT
        CierreMensual c = reporteService().cierreMensual(mes);
        // ASSERT
        assertEquals(d("5800.00"), c.ganancia());
```

```
        assertEquals(d("16000.00"), c.noFacturado());
    }

    /** RF45-CP02 (E2). Días con ventas sin cuadre = 1. */
    @Test
    @DisplayName("RF45-CP02 Generar el cierre.")
    void rf45_cp02_diasSinCuadre() throws Exception {
        // ARRANGE
        YearMonth mes = YearMonth.of(2026, 9);
        LocalDate dia = LocalDate.of(2026, 9, 15);
        when(ventaDao.cantidadVentas(dia, dia)).thenReturn(12);
        // ACT
        CierreMensual c = reporteService().cierreMensual(mes);
        // ASSERT
        assertEquals(1, c.diasSinCuadre());
    }

    /** RF45-CP03 (E3). Cierre marcado como parcial. */
    @Test
    @DisplayName("RF45-CP03 Generar el cierre del mes actual.")
    void rf45_cp03_mesEnCursoParcial() throws Exception {
        // ARRANGE
        YearMonth actual = YearMonth.from(Fechas.hoy());
        // ACT
        CierreMensual c = reporteService().cierreMensual(actual);
        // ASSERT
        assertTrue(c.parcial());
    }

    /** RF45-CP04 (E4). Mensaje «El mes aún no empieza». */
    @Test
    @DisplayName("RF45-CP04 Generar el cierre del próximo mes.")
    void rf45_cp04_mesFuturo() throws Exception {
        // ARRANGE
        YearMonth siguiente = YearMonth.from(Fechas.hoy()).plusMonths(1);
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
reporteService().cierreMensual(siguiente));
        // ASSERT
        assertEquals("El mes aún no empieza", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF45GenerarCierreMensualTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF45 Generar cierre mensual ✔
│     ├─ RF45-CP04 Generar el cierre del próximo mes. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperation…
│     ├─ RF45-CP02 Generar el cierre. ✘ Pendiente de implementar
│     ├─ RF45-CP01 Generar el cierre de septiembre. ✘ Pendiente de implementar
│     └─ RF45-CP03 Generar el cierre del mes actual. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF45GenerarCierreMensualTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF45 Generar cierre mensual ✔
│     ├─ RF45-CP04 Generar el cierre del próximo mes. ✔
│     ├─ RF45-CP02 Generar el cierre. ✔
│     ├─ RF45-CP01 Generar el cierre de septiembre. ✔
│     └─ RF45-CP03 Generar el cierre del mes actual. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 37. *Ejecución de RF45GenerarCierreMensualTest en rojo y en verde*

RF47 Consultar ganancia por producto · RF47ConsultarGananciaPorProductoTest

```
@DisplayName("RF47 Consultar ganancia por producto")
class RF47ConsultarGananciaPorProductoTest extends TddBase {

    /** RF47-CP01 (E1). Ganancia S/ 120.00 y margen 18.8 %. */
    @Test
    @DisplayName("RF47-CP01 Calcular ganancia y margen.")
    void rf47_cp01_gananciaYMargen() throws Exception {
        // ARRANGE
        GananciaProducto corvina = new GananciaProducto("Corvina", d("20.00"), d("640.00"),
d("520.00"));
        // ACT
        BigDecimal ganancia = corvina.ganancia();
        BigDecimal margen = corvina.margen();
        // ASSERT
        assertEquals(d("120.00"), ganancia);
        assertEquals(new BigDecimal("18.8"), margen);
    }

    /** RF47-CP02 (E2). Ganancia S/ -2.00. */
    @Test
    @DisplayName("RF47-CP02 Calcular la ganancia.")
    void rf47_cp02_perdidaPorRemate() throws Exception {
        // ARRANGE
        GananciaProducto bonito = new GananciaProducto("Bonito", d("4.00"), d("24.00"),
d("26.00"));
        // ACT
        BigDecimal ganancia = bonito.ganancia();
        // ASSERT
        assertEquals(d("-2.00"), ganancia);
        assertTrue(ganancia.signum() < 0);
    }

    /** RF47-CP03 (E3). Mensaje «Revise el rango de fechas». */
    @Test
    @DisplayName("RF47-CP03 Consultar del 31/10 al 01/10.")
    void rf47_cp03_rangoInvertido() throws Exception {
        // ARRANGE
        LocalDate desde = LocalDate.of(2026, 10, 31), hasta = LocalDate.of(2026, 10, 1);
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
reporteService().gananciaPorProducto(desde, hasta));
        // ASSERT
        assertEquals("Revise el rango de fechas", e.getMessage());
    }

    /** RF47-CP04 (E4). Margen 0.00 %. */
    @Test
    @DisplayName("RF47-CP04 Calcular su margen.")
    void rf47_cp04_sinVentasMargenCero() throws Exception {
        // ARRANGE
        GananciaProducto perico = new GananciaProducto("Perico", d("0.00"), d("0.00"),
d("0.00"));
        // ACT
        BigDecimal margen = perico.margen();
        // ASSERT
        assertEquals(d("0.00"), margen);
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF47ConsultarGananciaPorProductoTest
FASE ROJA: clases Service sin implementar

+
├─ JUnit Jupiter ✔
│  └─ RF47 Consultar ganancia por producto ✔
│     ├─ RF47-CP03 Consultar del 31/10 al 01/10. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperationExcep…
│     ├─ RF47-CP04 Calcular su margen. ✘ Pendiente de implementar
│     ├─ RF47-CP02 Calcular la ganancia. ✘ Pendiente de implementar
│     └─ RF47-CP01 Calcular ganancia y margen. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF47ConsultarGananciaPorProductoTest
FASE VERDE: código implementado

+
├─ JUnit Jupiter ✔
│  └─ RF47 Consultar ganancia por producto ✔
│     ├─ RF47-CP03 Consultar del 31/10 al 01/10. ✔
│     ├─ RF47-CP04 Calcular su margen. ✔
│     ├─ RF47-CP02 Calcular la ganancia. ✔
│     └─ RF47-CP01 Calcular ganancia y margen. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 38*. Ejecución de RF47ConsultarGananciaPorProductoTest en rojo y en verde*

RF48 Exportar reportes a Excel · RF48ExportarReportesAExcelTest

```
@DisplayName("RF48 Exportar reportes a Excel")
class RF48ExportarReportesAExcelTest extends TddBase {

    /** RF48-CP01 (E1). Excel con hoja «Ventas», encabezado en la fila 3 y 2 filas de datos.
*/
    @Test
    @DisplayName("RF48-CP01 Exportar ventas del 27/10.")
    void rf48_cp01_excelVentas() throws Exception {
        // ARRANGE
        LocalDate dia = LocalDate.of(2026, 10, 27);
        when(ventaDao.listarEntre(dia, dia)).thenReturn(List.of(venta(10, null, "CONTADO",
"59.20", "VIGENTE", null), venta(11, 1, "CREDITO", "320.00", "VIGENTE", null)));
        // ACT
        byte[] xlsx = reporteService().excelVentas(dia, dia);
        // ASSERT
        try (Workbook wb = new XSSFWorkbook(new ByteArrayInputStream(xlsx))) {
            Sheet sh = wb.getSheet("Ventas");
            assertNotNull(sh);
            assertEquals("Total (S/)", sh.getRow(2).getCell(8).getStringCellValue());
            assertEquals(320.00, sh.getRow(4).getCell(8).getNumericCellValue(), 0.001);
        }
    }

    /** RF48-CP02 (E2). Excel con 2 hojas: «Cierre 2026-09» y «Por producto». */
    @Test
    @DisplayName("RF48-CP02 Exportar el cierre mensual.")
    void rf48_cp02_excelCierreDosHojas() throws Exception {
        // ARRANGE
        YearMonth mes = YearMonth.of(2026, 9);
        // ACT
        byte[] xlsx = reporteService().excelCierreMensual(mes);
        // ASSERT
        try (Workbook wb = new XSSFWorkbook(new ByteArrayInputStream(xlsx))) {
            assertEquals(2, wb.getNumberOfSheets());
            assertEquals("Cierre 2026-09", wb.getSheetName(0));
            assertEquals("Por producto", wb.getSheetName(1));
        }
    }

    /** RF48-CP03 (E3). La fila de la corvina tiene margen 18.8. */
    @Test
    @DisplayName("RF48-CP03 Exportar ganancia por producto.")
    void rf48_cp03_excelGananciaProducto() throws Exception {
```

```
        // ARRANGE
        LocalDate desde = LocalDate.of(2026, 10, 1), hasta = LocalDate.of(2026, 10, 31);
        when(ventaDao.gananciaPorProducto(desde, hasta)).thenReturn(List.of(new
GananciaProducto("Corvina", d("20.00"), d("640.00"), d("520.00"))));
        // ACT
        byte[] xlsx = reporteService().excelGananciaProducto(desde, hasta);
        // ASSERT
        try (Workbook wb = new XSSFWorkbook(new ByteArrayInputStream(xlsx))) {
            Row fila = wb.getSheetAt(0).getRow(3);
            assertEquals("Corvina", fila.getCell(0).getStringCellValue());
            assertEquals(18.8, fila.getCell(5).getNumericCellValue(), 0.001);
        }
    }

    /** RF48-CP04 (E4). Mensaje «Revise el rango de fechas». */
    @Test
    @DisplayName("RF48-CP04 Exportar ventas del 31/10 al 01/10.")
    void rf48_cp04_rangoInvertido() throws Exception {
        // ARRANGE
        LocalDate desde = LocalDate.of(2026, 10, 31), hasta = LocalDate.of(2026, 10, 1);
        // ACT
        NegocioException e = assertThrows(NegocioException.class, () ->
reporteService().excelVentas(desde, hasta));
        // ASSERT
        assertEquals("Revise el rango de fechas", e.getMessage());
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF48ExportarReportesAExcelTest
FASE ROJA: clases Service sin implementar

.
├─ JUnit Jupiter ✔
│  └─ RF48 Exportar reportes a Excel ✔
│     ├─ RF48-CP04 Exportar ventas del 31/10 al 01/10. ✘ Unexpected exception type thrown, expected: <NegocioException> but was: <UnsupportedOperatio…
│     ├─ RF48-CP03 Exportar ganancia por producto. ✘ Pendiente de implementar
│     ├─ RF48-CP02 Exportar el cierre mensual. ✘ Pendiente de implementar
│     └─ RF48-CP01 Exportar ventas del 27/10. ✘ Pendiente de implementar

[   4 tests found ]  [   0 tests successful ]  [   4 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF48ExportarReportesAExcelTest
FASE VERDE: código implementado

.
├─ JUnit Jupiter ✔
│  └─ RF48 Exportar reportes a Excel ✔
│     ├─ RF48-CP04 Exportar ventas del 31/10 al 01/10. ✔
│     ├─ RF48-CP03 Exportar ganancia por producto. ✔
│     ├─ RF48-CP02 Exportar el cierre mensual. ✔
│     └─ RF48-CP01 Exportar ventas del 27/10. ✔

[   4 tests found ]  [   4 tests successful ]  [   0 tests failed ]
```

Figura 39. *Ejecución de RF48ExportarReportesAExcelTest en rojo y en verde*

RF49 Generar respaldo de la información · RF49GenerarRespaldoDeLaInformacionTest

```
@DisplayName("RF49 Generar respaldo de la información")
class RF49GenerarRespaldoDeLaInformacionTest extends TddBase {

    /** RF49-CP01 (E1). El archivo contiene las sentencias INSERT del DAO. */
    @Test
    @DisplayName("RF49-CP01 Generar el respaldo.")
    void rf49_cp01_generaRespaldo() throws Exception {
        // ARRANGE
        when(respaldoDao.exportarSql()).thenReturn("INSERT INTO producto VALUES
(1,'Bonito');\n");
        // ACT
        byte[] sql = respaldoService().generar(1);
```

```
        // ASSERT
        assertTrue(new String(sql, StandardCharsets.UTF_8).startsWith("INSERT INTO
producto"));
    }

    /** RF49-CP02 (E2). Bitácora RESPALDO «Respaldo generado (2 KB)». */
    @Test
    @DisplayName("RF49-CP02 Generar el respaldo.")
    void rf49_cp02_registraEnBitacora() throws Exception {
        // ARRANGE
        when(respaldoDao.exportarSql()).thenReturn("x".repeat(2048));
        // ACT
        respaldoService().generar(1);
        // ASSERT
        verify(auditoria).registrar(1, "RESPALDO", "Respaldo generado (2 KB)");
    }

    /** RF49-CP03 (E3). El texto conserva «Ñaña» en UTF-8. */
    @Test
    @DisplayName("RF49-CP03 Generar el respaldo.")
    void rf49_cp03_utf8() throws Exception {
        // ARRANGE
        when(respaldoDao.exportarSql()).thenReturn("INSERT INTO cliente VALUES (9,'Pollería
Doña Ñaña');");
        // ACT
        byte[] sql = respaldoService().generar(1);
        // ASSERT
        assertTrue(new String(sql, StandardCharsets.UTF_8).contains("Doña Ñaña"));
    }
}
```

Antes de programar (rojo):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF49GenerarRespaldoDeLaInformacionTest
FASE ROJA: clases Service sin implementar

╷
├─ JUnit Jupiter ✔
│  └─ RF49 Generar respaldo de la información ✔
│     ├─ RF49-CP01 Generar el respaldo. ✘ Pendiente de implementar
│     ├─ RF49-CP03 Generar el respaldo. ✘ Pendiente de implementar
│     └─ RF49-CP02 Generar el respaldo. ✘ Pendiente de implementar

[   3 tests found ]  [   0 tests successful ]  [   3 tests failed ]
```

Con el código implementado (verde):

```
$ java -jar junit-platform-console-standalone.jar --select-class pe.canomar.svc.tdd.RF49GenerarRespaldoDeLaInformacionTest
FASE VERDE: código implementado

╷
├─ JUnit Jupiter ✔
│  └─ RF49 Generar respaldo de la información ✔
│     ├─ RF49-CP01 Generar el respaldo. ✔
│     ├─ RF49-CP03 Generar el respaldo. ✔
│     └─ RF49-CP02 Generar el respaldo. ✔

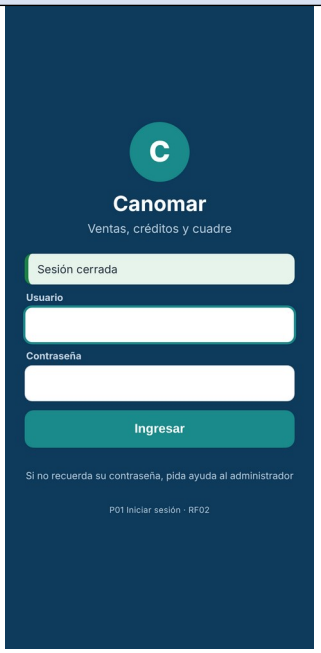
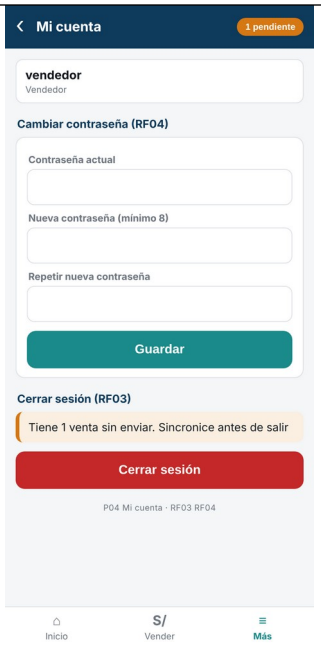
[   3 tests found ]  [   3 tests successful ]  [   0 tests failed ]
```

Figura 40. *Ejecución de RF49GenerarRespaldoDeLaInformacionTest en rojo y en verde*

2.6. Evidencia de pruebas manuales

Cada CP de tipo Pantalla se probó a mano sobre el sistema en ejecución (Spring Boot con MySQL/MariaDB y datos de una jornada: compra, precios, ventas, remate, anulación, abono, compromiso, pago a proveedor, cuadre y factura). Las capturas son del navegador en tamaño de celular (412 × 860).

Tabla 13. *Pruebas manuales de RF03 Cerrar sesión*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF03-CP01	Sesión abierta como «vendedor», sin ventas pendientes. **Acción:** Mi cuenta > Cerrar sesión. **Esperado:** Pantalla de acceso con el aviso «Sesión cerrada».		**Sí**
RF03-CP02	1 venta guardada en el celular que no se pudo enviar. **Acción:** Mi cuenta > Cerrar sesión > Cancelar en el aviso «Tiene 1 venta sin enviar… ¿Cerrar sesión?». **Esperado:** Sigue en Mi cuenta con el aviso «Tiene 1 venta sin enviar. Sincronice antes de salir» y el indicador «1 pendiente».		**Sí**

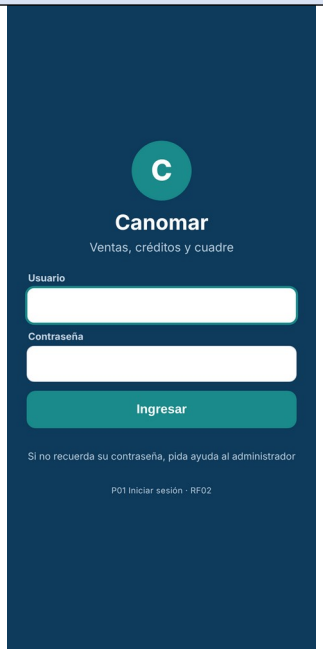
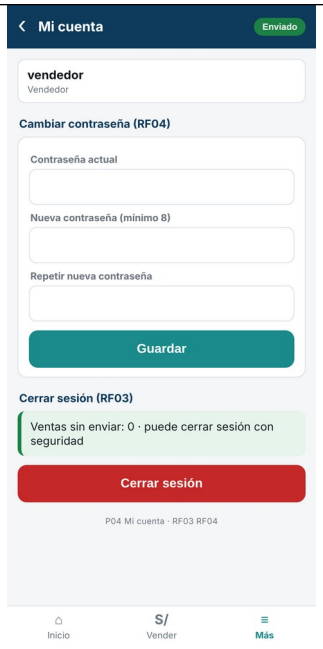
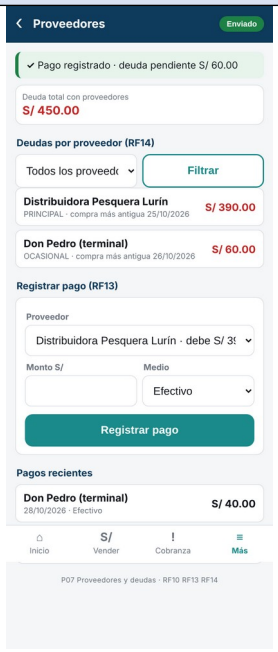
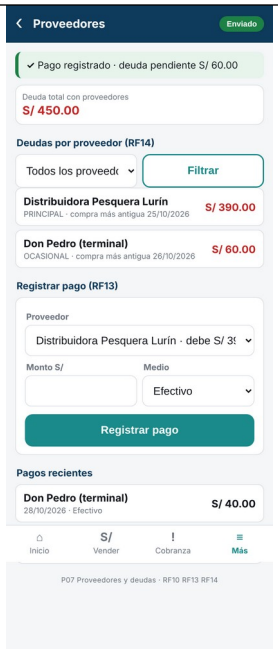
CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF03-CP03	Sesión cerrada. **Acción:** Abrir /ventas desde la barra de direcciones. **Esperado:** El sistema redirige a la pantalla de acceso.		**Sí**
RF03-CP04	Sin ventas pendientes. **Acción:** Abrir Mi cuenta. **Esperado:** Aviso verde «Ventas sin enviar: 0 · puede cerrar sesión con seguridad».		**Sí**

Tabla 14. *Pruebas manuales de RF14 Consultar deuda con proveedores*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF14-CP01	Compras a crédito de días anteriores. **Acción:** Abrir Más > Proveedores y deudas. **Esperado:** Se ve cada proveedor con su deuda y «desde» la fecha más antigua.		**Sí**
RF14-CP02	Deuda con Lurín S/ 390 y Don Pedro S/ 100. **Acción:** Ver la parte superior de la pantalla. **Esperado:** Tarjeta «Deuda total con proveedores S/ 490.00».		**Sí**

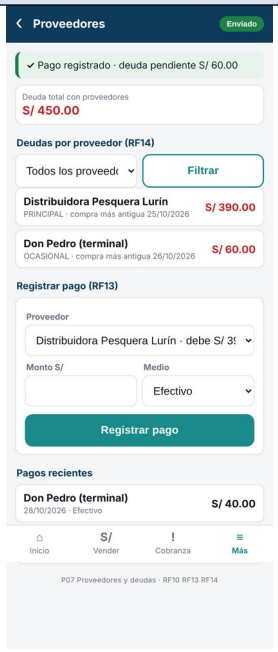
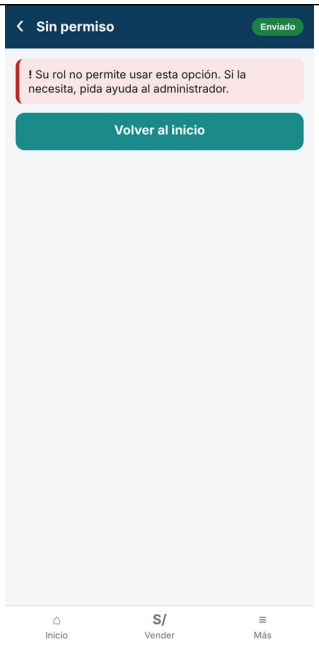
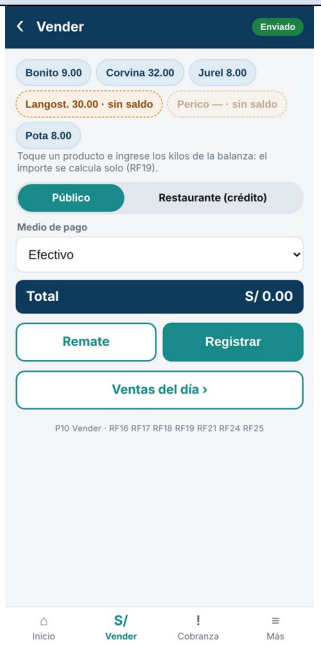
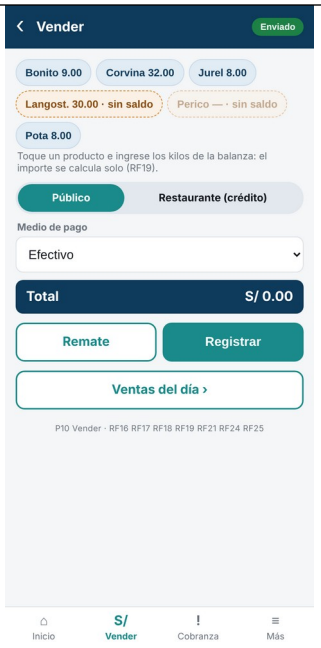
CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF14-CP03	Se registró un pago a proveedor. **Acción:** Bajar a «Pagos recientes». **Esperado:** Se ve el pago con fecha, proveedor, monto y medio.		**Sí**
RF14-CP04	Sesión como «vendedor». **Acción:** Abrir /proveedores. **Esperado:** Pantalla «Sin permiso».		**Sí**

Tabla 15. *Pruebas manuales de RF16 Consultar precios del día*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF16-CP01	Precios guardados hoy. **Acción:** Abrir Vender. **Esperado:** Cada botón muestra el producto y su precio (por ejemplo «Corvina 32.00»).		**Sí**
RF16-CP02	Perico sin precio hoy. **Acción:** Abrir Vender. **Esperado:** El botón «Perico — · sin saldo» se ve desactivado.		**Sí**

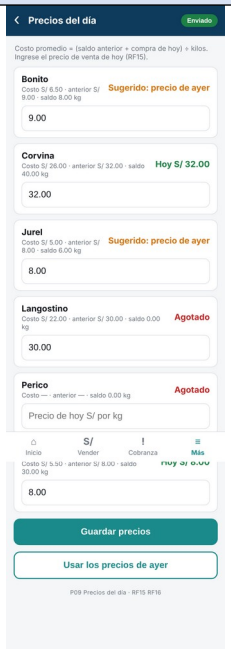
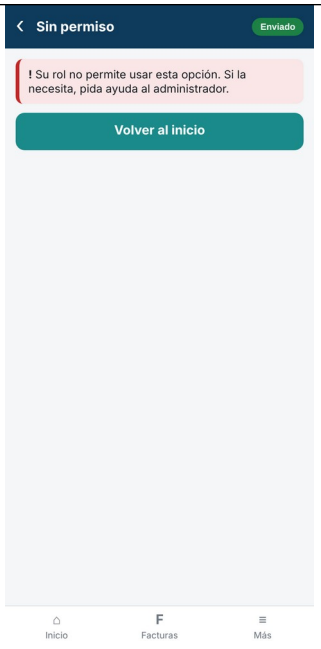
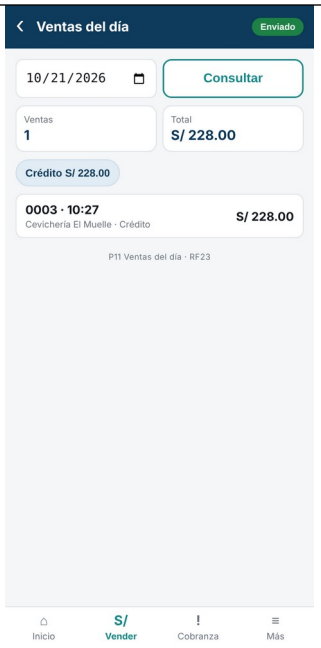
CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF16-CP03	Costo y precio de ayer cargados. **Acción:** Abrir Más > Precios del día. **Esperado:** Cada fila muestra costo S/ por kg, precio de ayer y casilla de precio de hoy.		**Sí**
RF16-CP04	Sesión como «contador». **Acción:** Abrir /vender. **Esperado:** Pantalla «Sin permiso».		**Sí**

Tabla 16. *Pruebas manuales de RF23 Consultar ventas del día*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF23-CP01	Ventas registradas hoy. **Acción:** Abrir Vender > Ventas del día. **Esperado:** Lista de ventas de hoy con «Ventas N» y «Total S/».		**Sí**
RF23-CP02	Ventas al crédito de días anteriores. **Acción:** Elegir la fecha de hace 7 días y tocar Consultar. **Esperado:** Se ven las ventas de esa fecha.		**Sí**

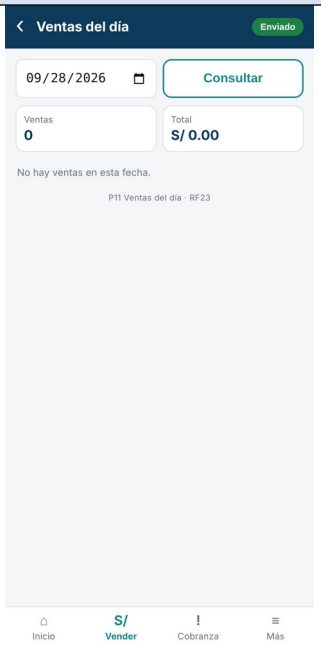
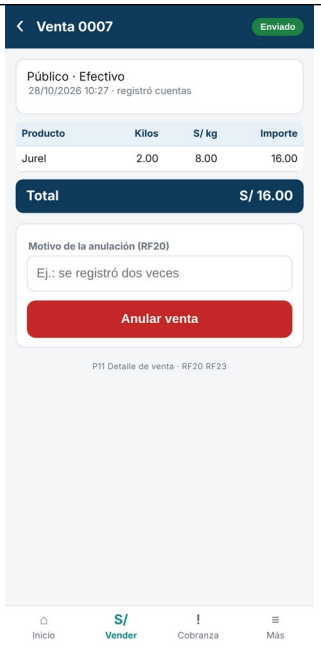
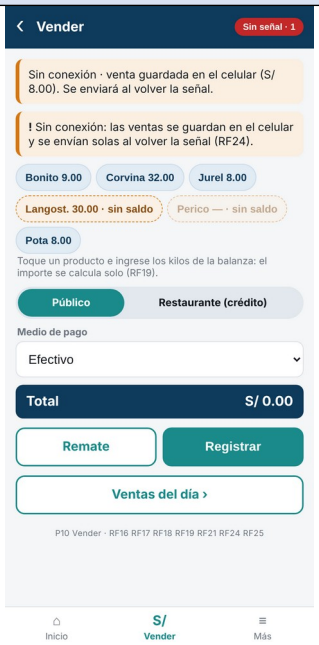
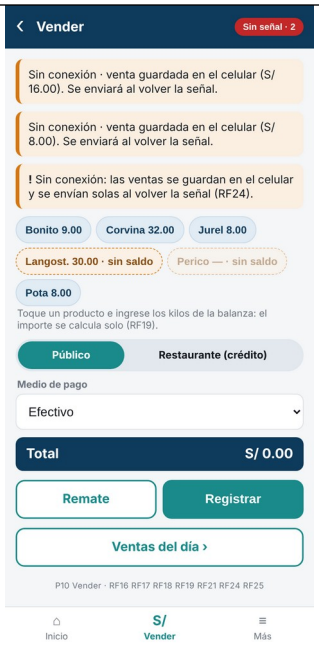
CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF23-CP03	Fecha sin ventas. **Acción:** Elegir una fecha sin ventas. **Esperado:** Mensaje «No hay ventas en esta fecha.».		**Sí**
RF23-CP04	Venta 0001 registrada. **Acción:** Tocar la venta 0001. **Esperado:** Se muestra el detalle: producto, kilos, precio e importe.		**Sí**

Tabla 17. *Pruebas manuales de RF24 Guardar venta sin conexión*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF24-CP01	Navegador en modo sin conexión. **Acción:** Registrar 1 kg de jurel al contado. **Esperado:** Aviso «Sin conexión · venta guardada en el celular (S/ 8.00). Se enviará al volver la señal.» e indicador «Sin señal · 1».		**Sí**
RF24-CP02	Sin conexión, 1 venta en cola. **Acción:** Registrar otra venta. **Esperado:** El indicador muestra «Sin señal · 2».		**Sí**

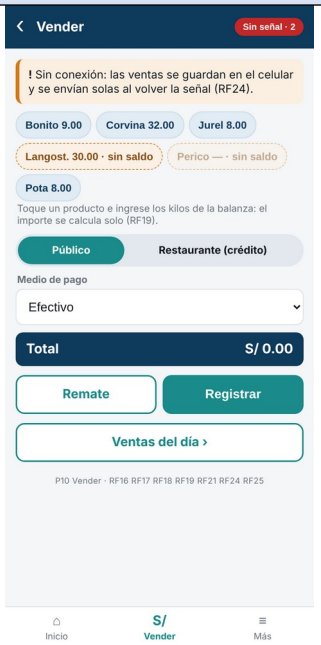
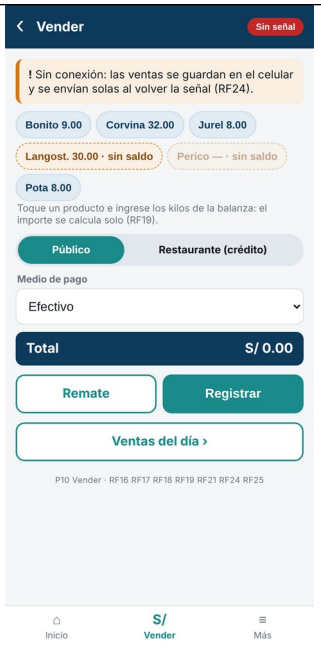
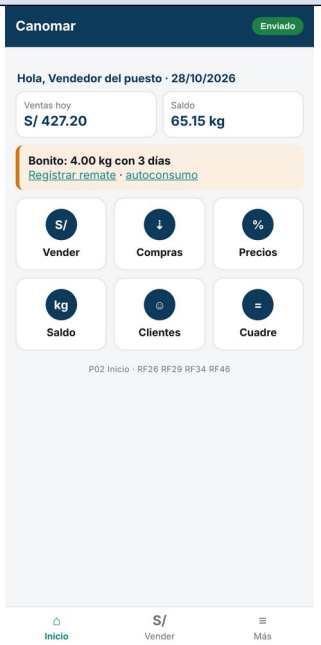
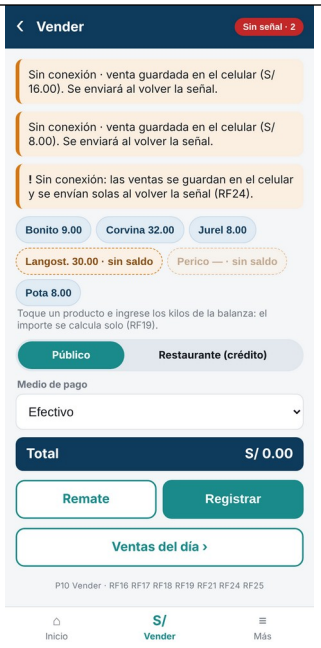
CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF24-CP03	2 ventas en cola. **Acción:** Recargar la página sin conexión. **Esperado:** La página abre desde la copia del celular y el indicador sigue en «Sin señal · 2».		**Sí**
RF24-CP04	Sin conexión. **Acción:** Abrir Vender. **Esperado:** Se ve el aviso «Sin conexión: las ventas se guardan en el celular…».		**Sí**

Tabla 18. *Pruebas manuales de RF26 Mostrar estado de sincronización*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF26-CP01	Sin ventas en cola. **Acción:** Abrir Inicio. **Esperado:** Indicador verde «Enviado».		**Sí**
RF26-CP02	2 ventas en cola sin conexión. **Acción:** Mirar el indicador. **Esperado:** Indicador rojo «Sin señal · 2».		**Sí**

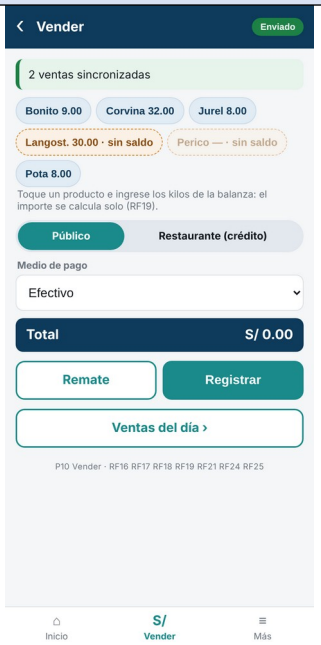
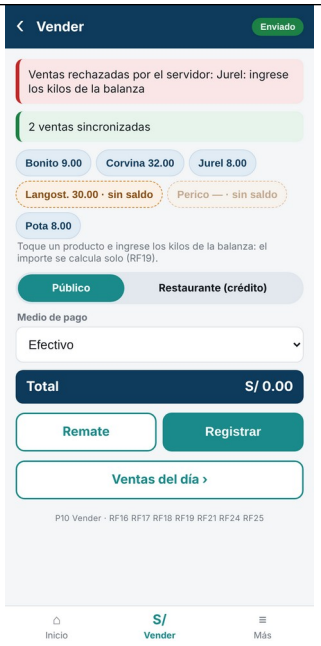
CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF26-CP03	2 ventas en cola. **Acción:** Volver a conectar. **Esperado:** Aviso «2 ventas sincronizadas» e indicador «Enviado».		**Sí**
RF26-CP04	Una venta en cola con 0 kg de jurel. **Acción:** Volver a conectar. **Esperado:** Aviso rojo «Ventas rechazadas por el servidor: Jurel: ingrese los kilos de la balanza».		**Sí**

Tabla 19. *Pruebas manuales de RF29 Alertar mercadería antigua*

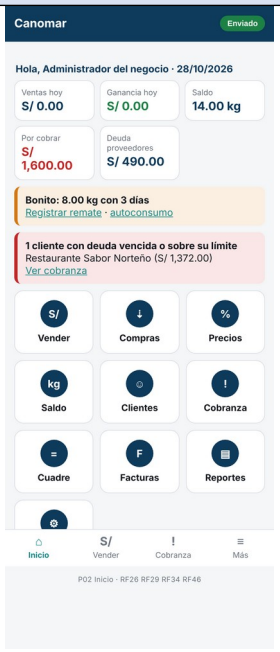
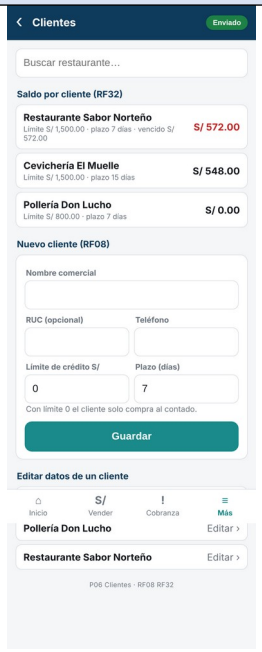
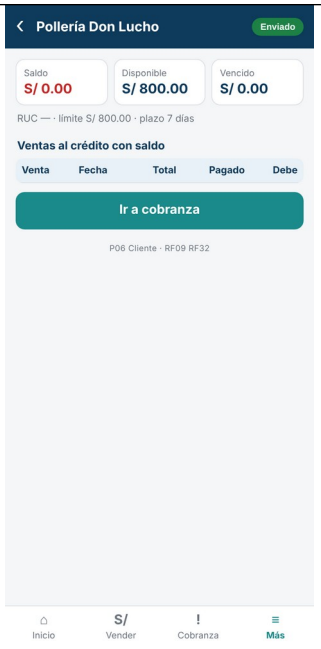
CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF29-CP04	Bonito con 3 días. **Acción:** Abrir Inicio. **Esperado:** Tarjeta naranja «Bonito: 8.00 kg con 3 días» con enlaces «Registrar remate · autoconsumo».		**Sí**

Tabla 20. *Pruebas manuales de RF32 Consultar saldo por cliente*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF32-CP01	Clientes con ventas al crédito. **Acción:** Abrir Clientes. **Esperado:** Cada cliente con saldo y límite.		**Sí**

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF32-CP02	Cevichería El Muelle con saldo. **Acción:** Tocar Cevichería El Muelle. **Esperado:** Tarjetas Saldo, Disponible y Vencido.		**Sí**
RF32-CP03	Pollería Don Lucho sin ventas. **Acción:** Abrir la pollería. **Esperado:** Saldo S/ 0.00.		**Sí**

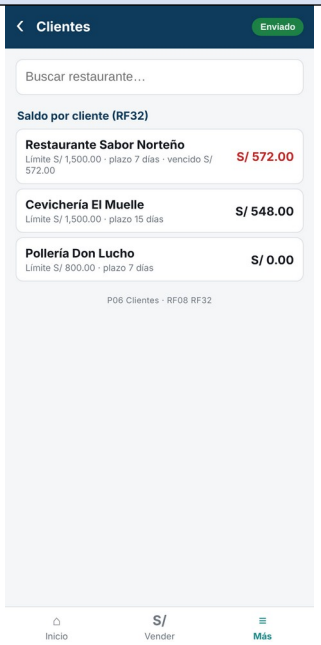
CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF32-CP04	Sesión como vendedor. **Acción:** Abrir Clientes. **Esperado:** Se ven los saldos (solo lectura).		**Sí**

Tabla 21. *Pruebas manuales de RF34 Alertar deudas vencidas*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF34-CP04	Cliente 2 con deuda vencida. **Acción:** Abrir Cobranza. **Esperado:** Tarjeta «Vencido (RF34)» con el monto vencido (S/ 572.00 después del abono) y el cliente en rojo.		**Sí**

Tabla 22. *Pruebas manuales de RF37 Compartir estado de cuenta*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF37-CP01	Estado de cuenta de Cevichería El Muelle. **Acción:** Tocar «Descargar PDF». **Esperado:** Se descarga «EstadoCuenta_Cevicher_a_El_Muelle_2026-10-28.pdf».		**Sí**
RF37-CP02	Celular Android (Chrome), estado de cuenta de Restaurante Sabor Norteño. **Acción:** Tocar «Compartir (RF37)». **Esperado:** Se abre la hoja de compartir del celular con el PDF adjunto.		**Sí**

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF37-CP03	Navegador de escritorio. **Acción:** Tocar «Compartir (RF37)». **Esperado:** Como el navegador no ofrece compartir, se descarga el PDF.		**Sí**
RF37-CP04	PDF descargado. **Acción:** Abrir el PDF. **Esperado:** Encabezado «Canomar — Estado de cuenta» y saldo final.		**Sí**

Tabla 23. *Pruebas manuales de RF41 Consultar historial de cuadres*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF41-CP01	Cuadres anteriores. **Acción:** Abrir Cuadre > Historial. **Esperado:** Tabla con fecha, efectivo, diferencia y ganancia.		**Sí**
RF41-CP02	Cuadres de la semana. **Acción:** Elegir el rango de la semana. **Esperado:** Solo los días del rango.		**Sí**

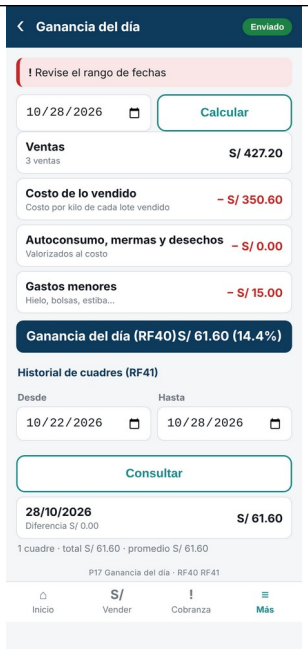
CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF41-CP03	Rango sin cuadres. **Acción:** Elegir enero de 2026. **Esperado:** Resumen «0 cuadres · total S/ 0.00 · promedio S/ 0.00».		**Sí**
RF41-CP04	Rango invertido. **Acción:** Desde mayor que Hasta. **Esperado:** Mensaje «Revise el rango de fechas».		**Sí**

Tabla 24. *Pruebas manuales de RF44 Consultar ventas pendientes de facturar*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF44-CP01	Ventas al crédito de clientes con RUC sin factura. **Acción:** Abrir Facturas. **Esperado:** Sección «Pendientes de facturar» con cliente, RUC, N° de ventas y total.		**Sí**
RF44-CP02	Ventas en el mes actual. **Acción:** Elegir septiembre de 2026. **Esperado:** Solo las ventas de ese mes (ninguna: «No hay ventas por facturar»).		**Sí**

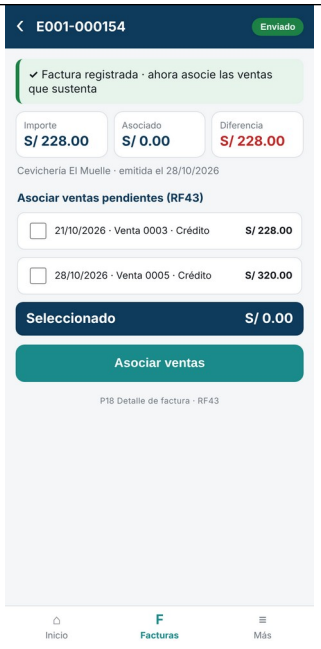
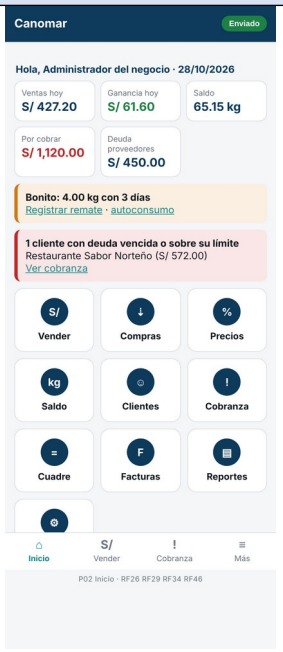
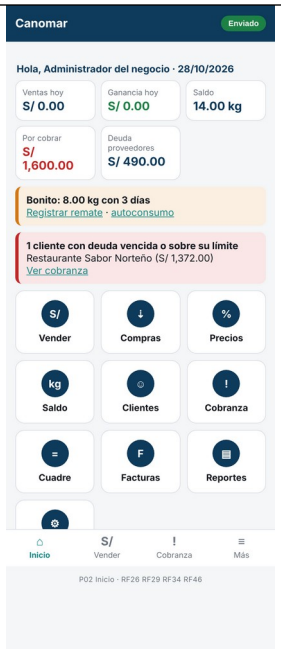
CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF44-CP03	Mes sin pendientes. **Acción:** Elegir enero de 2026. **Esperado:** Mensaje «No hay ventas por facturar».		**Sí**
RF44-CP04	Factura registrada. **Acción:** Abrir la factura. **Esperado:** Lista de ventas sin factura del cliente para marcar.		**Sí**

Tabla 25. *Pruebas manuales de RF46 Consultar panel del negocio*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF46-CP01	Ventas registradas hoy. **Acción:** Abrir Inicio como administrador. **Esperado:** Tarjetas Ventas hoy, Ganancia hoy, Saldo, Por cobrar y Deuda proveedores.		**Sí**
RF46-CP02	Sin ventas hoy. **Acción:** Abrir Inicio temprano. **Esperado:** Ventas hoy S/ 0.00 y el resto de tarjetas con datos.		**Sí**

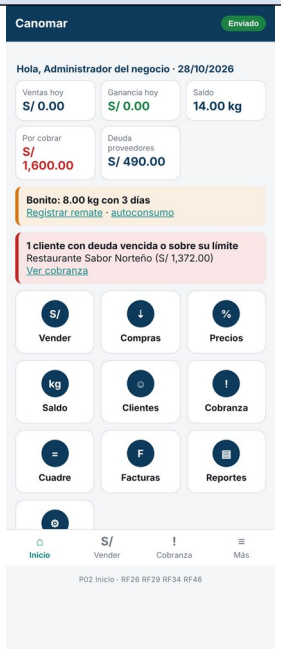
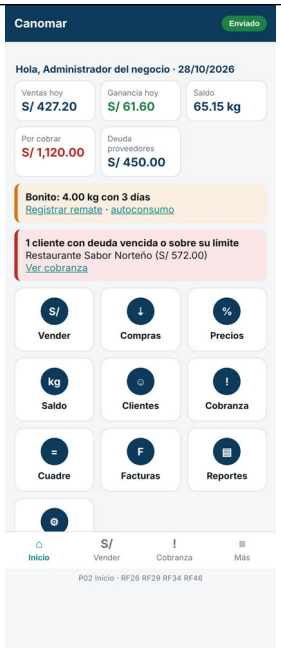
CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF46-CP03	Bonito con 3 días. **Acción:** Abrir Inicio. **Esperado:** Alerta de mercadería antigua.		**Sí**
RF46-CP04	Sabor Norteño vencido. **Acción:** Abrir Inicio. **Esperado:** Alerta roja «1 cliente con deuda vencida o sobre su límite» con «Ver cobranza».		**Sí**

Tabla 26. *Pruebas manuales de RF49 Generar respaldo de la información*

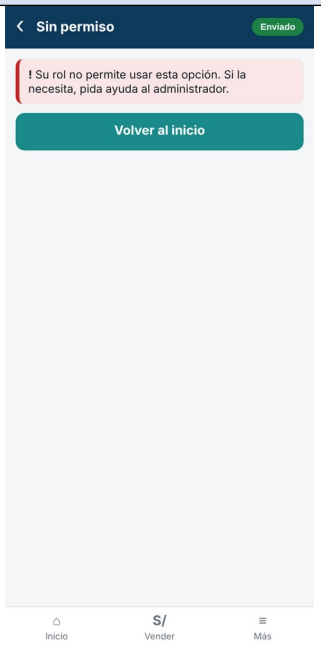
CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF49-CP04	Sesión como vendedor. **Acción:** Abrir /ajustes. **Esperado:** Pantalla «Sin permiso».		**Sí**

Tabla 27. *Pruebas manuales de RF50 Consultar bitácora de auditoría*

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF50-CP01	Operaciones registradas. **Acción:** Abrir Más > Respaldo y bitácora. **Esperado:** Lista de eventos recientes.		**Sí**

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF50-CP02	Una venta anulada. **Acción:** Filtrar por ANULAR_VENTA. **Esperado:** Solo la anulación.		**Sí**
RF50-CP03	Filtro sin resultados. **Acción:** Filtrar ANULAR_VENTA del 01/01 al 02/01/2026. **Esperado:** Mensaje «No hay eventos con ese filtro.».		**Sí**

CP	Qué se hizo	Captura de pantalla	¿Cumple?
RF50-CP04	Rango invertido. **Acción:** Desde mayor que Hasta. **Esperado:** Mensaje «La fecha inicial no puede ser mayor que la final».		**Sí**

2.7. Resumen de resultados

Tabla 28*. Resumen de las pruebas*

Concepto	Cantidad
CP totales (50 fichas)	200
CP automatizados (Regla e Infraestructura)	149
CP manuales (Pantalla)	51
Pasan (automatizados)	149 de 149
Pasan (manuales)	51 de 51 (1 pendiente de probar en el celular)
Fallan	0
RF aptos con todos sus tests en verde	38 de 38 = 100 %

Los números cuadran con las secciones anteriores: 38 RF aptos en 2.4, 149 tests en 2.5 (uno por CP de Regla o Infraestructura) y 51 CP de Pantalla en 2.6. Además, el repositorio conserva 39 pruebas unitarias del avance anterior (CalculosTest y pruebas de servicios), por lo que mvn test ejecuta 188 pruebas en total.

CAPÍTULO 3. CONTROL DE VERSIONES

Este capítulo demuestra qué porcentaje de los entregables del proyecto está en GitHub y quién contribuye.

3.1. Repositorio y ramas

Repositorio: https://github.com/ashino99/svc-canomar-39120

El nombre sigue la convención nombre-proyecto-sección de la guía del curso (svc-canomar + sección 39120). El repositorio es público. La versión de este avance está publicada como release con la etiqueta apf3 sobre la rama main.



Figura 41. *Ramas del repositorio en GitHub (Code > Branches)*

Fuente: Captura de GitHub.

Tabla 29. *Ramas del repositorio*

Rama	Para qué sirve
main	Versión estable que se entrega en cada avance (APF1, APF2, APF3 y final). Solo recibe cambios desde develop.
develop	Integración: aquí se juntan los módulos terminados y GitHub Actions compila y ejecuta las pruebas en cada push.
feature/<módulo>	Una rama por módulo en construcción (por ejemplo feature/ventas, feature/tdd, feature/docs-apf3). Se une a develop con un pull request que solo se acepta con el check verde de GitHub Actions.

Convención de mensajes de commit: **tipo(ámbito): descripción en presente**, con los tipos feat (función nueva), fix (corrección), test (pruebas), docs (documentos), refactor y build. Ejemplos: «feat(ventas): registrar venta al crédito con límite», «test(tdd): CP de RF31 abono a la venta más antigua», «docs(apf3): informe y anexos».

3.2. Entregables frente al repositorio

Tabla 30. *Entregables del proyecto y su carpeta en el repositorio*

Entregable	Carpeta en el repositorio	¿Subido?
APF1 (informe PDF)	docs/entregables/APF1_Equipo7.pdf	Sí
APF2 (informe PDF)	docs/entregables/APF2_Equipo7.pdf	Sí
Anexos del APF2 (Gantt, BPMN, UML, prototipo, normalización)	docs/entregables/Anexos_APF2_Equipo7.zip	Sí

Entregable	Carpeta en el repositorio	¿Subido?
APF3 (este informe, sin el Anexo F confidencial)	docs/entregables/APF3_Equipo7.pdf	Sí
Anexos A a E del APF3 (Excel y script SQL)	docs/anexos/	Sí
Código fuente	src/main/java, src/main/resources	Sí
Carpeta de pruebas (149 TDD + 39 unitarias)	src/test/java	Sí
Script SQL del modelo final	src/main/resources/schema.sql y docs/anexos/Anexo_E_svc_canomar_final.sql	Sí
Diagramas fuente (.drawio y .dot)	docs/diagramas/	Sí
Capturas de las pruebas manuales y de TDD	docs/capturas/	Sí
README e instrucciones	README.md, PASOS_GITHUB.md, docs/GUIA_DEL_CODIGO.md	Sí
Integración continua	.github/workflows/maven.yml	Sí
Porcentaje	12 subidos / 12 entregables × 100	100 %

El Anexo F (propuestas con hardware) no cuenta como entregable del repositorio: se entrega solo por UTP+Class y está excluido en .gitignore.

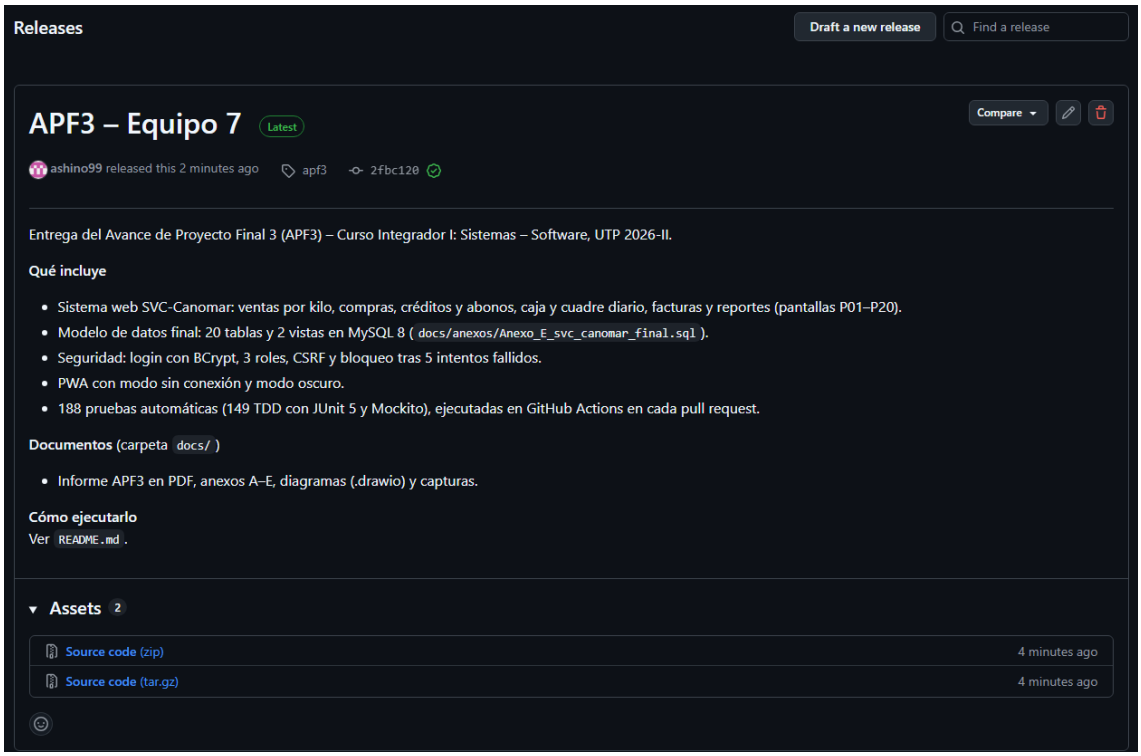


Figura 42. *Release apf3 publicado en GitHub*

Fuente: Captura de GitHub.

3.3. Contribuidores

El historial de main tiene 9 commits con mensaje descriptivo y 5 commits de merge (uno por pull request), todos del usuario ashino99 (Doria Bautista, I.). El integrante Coronel Lopez, F. no registra commits, lo que coincide con la tabla de participación.

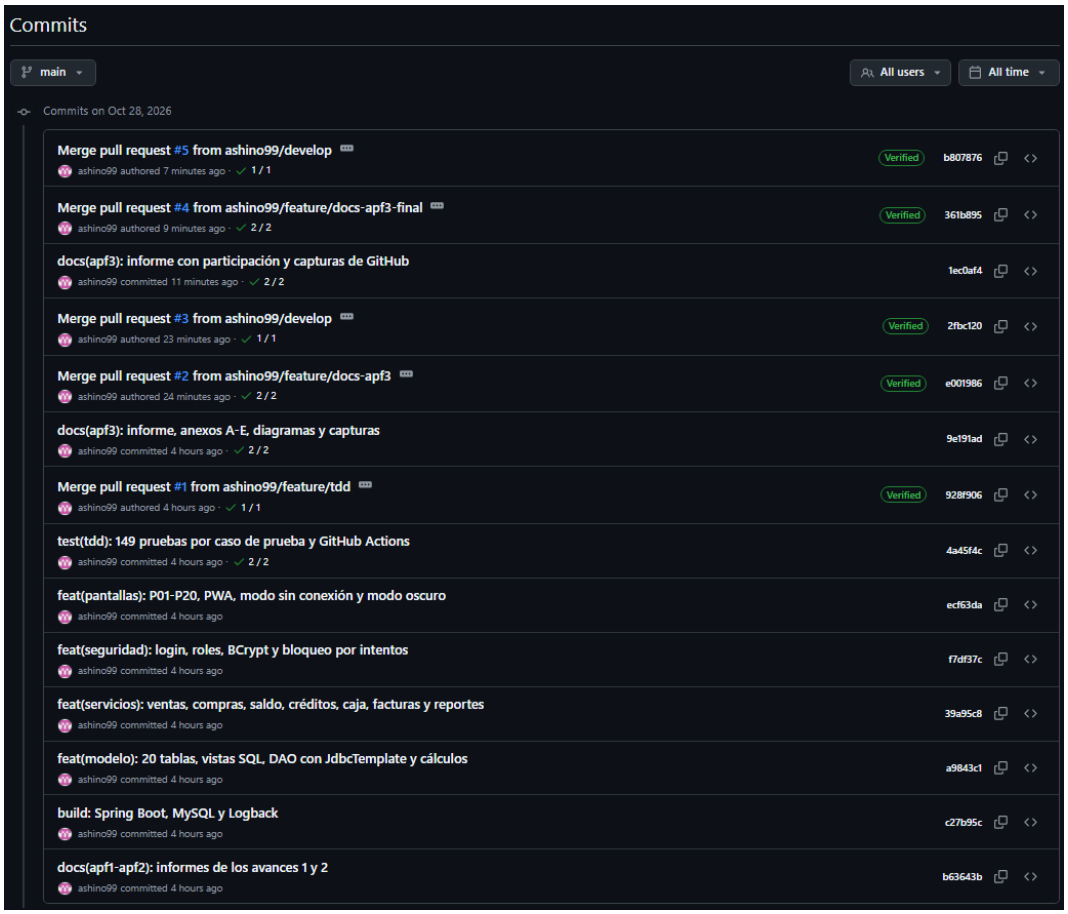


Figura 43. *Historial de commits de la rama main (pestaña Commits)*

Fuente: Captura de GitHub.

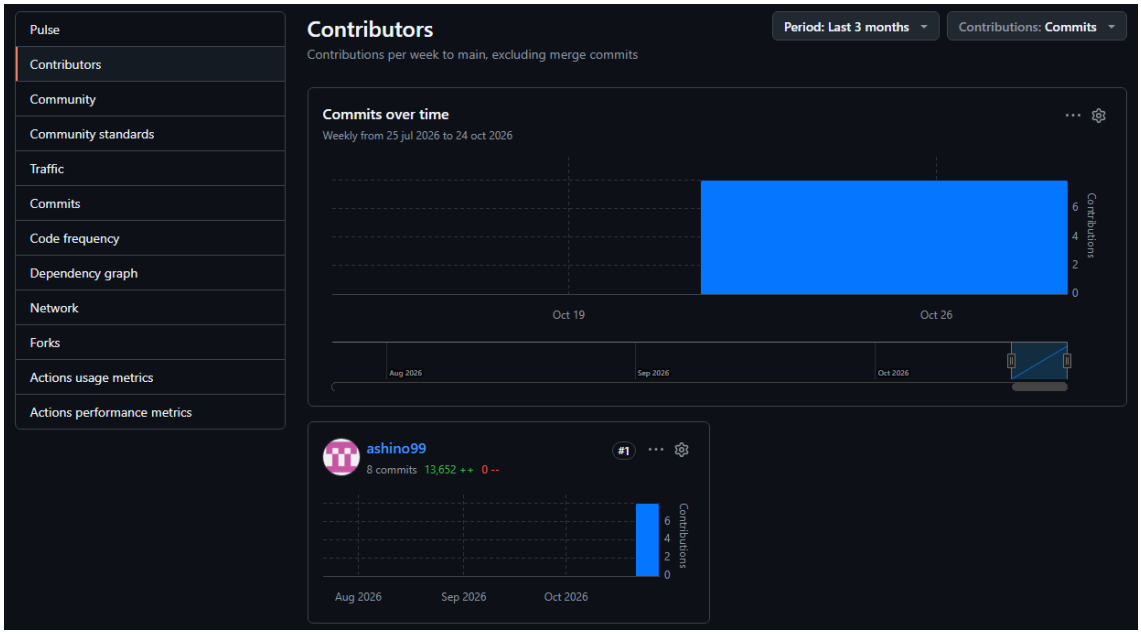


Figura 44. *Contribuidores del repositorio (Insights > Contributors)*

Fuente: Captura de GitHub.

3.4. Archivos excluidos

El archivo .gitignore evita subir compilados, registros, configuraciones del IDE y, sobre todo, contraseñas y claves. La contraseña de MySQL no está en el repositorio: application.properties la lee de la variable de entorno DB_PASSWORD (por defecto «root» solo para la máquina local).

```
target/
logs/
*.log
.idea/
*.iml
.vscode/
.DS_Store
# Credenciales locales: no subir contraseñas
application-local.properties
# Variables de entorno con claves
.env
*.env
# Anexo confidencial de las propuestas con hardware (solo se entrega por
UTP+Class)
docs/confidencial/
*_Confidencial*.pdf
```

```
# ---- Base de datos MySQL 8 ----
# La base se crea sola la primera vez (createDatabaseIfNotExist).
# Cambie la contraseña por la de su usuario root de MySQL, o use la variable de
entorno DB_PASSWORD.
spring.datasource.url=jdbc:mysql://localhost:3306/svc_canomar?
createDatabaseIfNotExist=true&serverTimezone=America/
Lima&useUnicode=true&characterEncoding=UTF-8
spring.datasource.username=${DB_USER:root}
spring.datasource.password=${DB_PASSWORD:root}
```

CAPÍTULO 4. VERIFICACIÓN DE INTERFACES

La verificación UX/UI de la semana 10 se hizo como auto-auditoría (sección 39120) con las 4 capas de la guía: trazabilidad vista/RF, heurísticas, coherencia vista/datos y navegación. Se auditaron las pantallas del sistema construido, en ejecución y conectado a la base de datos, con los tres roles.

4.1. Capa 1: trazabilidad vista / RF

La matriz completa de los 50 RF está en el Anexo C. En el cuerpo se muestran los primeros 15.

Tabla 31*. Matriz de trazabilidad vista / RF (RF01 a RF15)*

RF	Nombre	Pantalla implementada	¿Rotulada con su RF?	¿Pasos del CUS en pantalla?	¿Cada excepción tiene su mensaje?	Estado
RF01	Registrar usuario	P03 Usuarios (usuarios.html)	Sí	Sí	Sí: «El usuario ya existe, ingrese otro» · «La contraseña debe tener al menos 8 caracteres» · «Elija un rol válido»	Funcional
RF02	Iniciar sesión	P01 Iniciar sesión (login.html)	Sí	Sí	Sí: «Usuario o contraseña incorrectos» · «Acceso bloqueado por 5 minutos» · «Su cuenta está desactivada»	Funcional
RF03	Cerrar sesión	P04 Mi cuenta (mi-cuenta.html)	Sí	Sí	Sí: «¿Cerrar sesión?» si hay ventas sin enviar · «Sesión cerrada»	Funcional
RF04	Cambiar contraseña	P04 Mi cuenta (mi-cuenta.html)	Sí	Sí	Sí: «Las contraseñas no coinciden» · «La contraseña actual es incorrecta» · «La contraseña debe tener al menos 8 caracteres»	Funcional
RF05	Desactivar usuario	P03 Usuarios (usuarios.html)	Sí	Sí	Sí: «No puede desactivar su propia cuenta» · «No puede desactivar al único administrador»	Funcional
RF06	Registrar producto	P05 Productos (productos.html)	Sí	Sí	Sí: «El producto ya existe» · «Ingrese el nombre del producto»	Funcional
RF07	Editar producto	P05 Productos (productos.html)	Sí	Sí	Sí: «No se puede inactivar: tiene N kg en saldo» · «Ya existe otro producto con ese nombre»	Funcional
RF08	Registrar cliente	P06 Clientes (clientes.html · cliente-detalle.html)	Sí	Sí	Sí: «El RUC debe tener 11 dígitos» · «El cliente ya está registrado con ese RUC»	Funcional
RF09	Editar cliente	P06 Clientes (clientes.html · cliente-detalle.html)	Sí	Sí	Sí: «El saldo actual (S/ …) supera el nuevo límite…» · «El plazo debe estar entre 0 y 90 días» · «El límite de crédito no puede ser negativo»	Funcional
RF10	Registrar proveedor	P07 Proveedores y deudas (proveedores.html)	Sí	Sí	Sí: «El proveedor ya está registrado» · «El RUC debe tener 11 dígitos»	Funcional
RF11	Registrar compra del día	P08 Nueva compra (compra-nueva.html)	Sí	Sí	Sí: «Los kilos y el costo deben ser mayores que cero» · «X está inactivo» · «Indique si la compra fue al contado o a crédito»	Funcional
RF12	Calcular costo promedio por kilo	P08 Nueva compra (compra-nueva.html)	Sí	Sí	Sí: Muestra el costo promedio recalculado en Precios del día	Funcional
RF13	Registrar pago a proveedor	P07 Proveedores y deudas (proveedores.html)	Sí	Sí	Sí: «El pago supera la deuda (S/ …)» · «Ingrese un monto mayor que cero»	Funcional
RF14	Consultar deuda con proveedores	P07 Proveedores y deudas (proveedores.html)	Sí	Sí	Sí: Deuda total y por proveedor · «Sin permiso» para otros roles	Funcional
RF15	Registrar precios de venta del día	P09 Precios del día (precios.html)	Sí	Sí	Sí: «X: S/ … está por debajo del costo S/ …. ¿Confirmar los precios?» · «X: el precio debe ser mayor que cero»	Funcional

Funcional significa que la pantalla guarda o consulta datos reales en la base de datos. Las 50 filas del Anexo C están en estado Funcional; ninguna es solo visual.

4.2. Matriz de hallazgos

Tabla 32*. Matriz de hallazgos de las 4 capas*

N°	Vista (RF)	Capa	Hallazgo	Severidad	Mejora propuesta	Origen	Estado
H01	P10 Vender (RF16, RF17)	Capa 3 · Coherencia vista/datos	Los botones de productos sin saldo (Corvina, Langostino, Pota con 0 kg) se veían iguales a los demás; el vendedor recién se enteraba al registrar, con el aviso «Solo quedan 0.00 kg».	Alta	Marcar el botón con «· sin saldo» y borde naranja punteado, sin bloquearlo (el saldo puede estar mal contado).	Auto-auditoría	Levantado
H02	P08 Nueva compra (RF11)	Capa 2 · Heurística: prevención de errores	La opción «A crédito» estaba marcada por defecto; una compra pagada al contado podía quedar como deuda con el proveedor y distorsionar RF14.	Alta	Quitar la opción por defecto: el sistema exige elegir Contado o A crédito («Indique si la compra fue al contado o a crédito»).	Auto-auditoría	Levantado
H03	Las 20 pantallas	Capa 1 · Trazabilidad	Las pantallas no mostraban su código ni los RF que implementan; RF02, RF05, RF17, RF23, RF25 y RF46 no aparecían rotulados en ninguna.	Media	Agregar al pie de cada pantalla el rótulo «P## Nombre · RF…» (fragmento pie de base.html).	Auto-auditoría	Levantado
H04	P09 Precios del día (RF15)	Capa 3 · Coherencia vista/datos	Decía «Sin precio hoy» aunque la casilla ya mostraba precargado el precio de ayer, y no quedaba claro si estaba guardado.	Media	Mostrar «Sugerido: precio de ayer» cuando hay precio anterior y aún no se guardó el de hoy.	Auto-auditoría	Levantado
H05	P16 Gastos y cuadre (RF39)	Capa 2 · Heurística: consistencia	Con ventas sin enviar, el cuadre mostraba una ventana nativa del navegador («Sincronice las 1 ventas pendientes…») distinta de los demás avisos y que bloquea la pantalla.	Media	Usar el aviso rojo de la propia página, igual que el resto del sistema.	Auto-auditoría	Levantado
H06	P01 Iniciar sesión (RF03)	Capa 2 · Heurística: visibilidad del estado	El aviso «Sesión cerrada» tenía letras blancas sobre fondo verde claro: casi no se leía.	Media	Texto oscuro en los avisos de la pantalla de acceso.	Auto-auditoría	Levantado
H07	P02, P04, P17, P18	Capa 2 · Heurística: lenguaje del usuario	Plurales fijos: «1 clientes con deuda vencida», «1 ventas», «1 cuadres», «1 ventas sin enviar».	Baja	Singular o plural según la cantidad.	Auto-auditoría	Levantado
H08	P18 Facturas (RF42)	Capa 2 · Heurística: visibilidad	La sección «Facturas registradas» quedaba vacía sin explicación cuando aún no había facturas.	Baja	Mensaje «Aún no hay facturas registradas.».	Auto-auditoría	Levantado
H09	P02 Inicio del contador	Capa 4 · Navegación por rol	El contador solo ve tres accesos (Clientes, Facturas, Reportes) y ningún indicador propio en el Inicio.	Baja	Agregar la tarjeta «Por facturar» con el total pendiente de RF44.	Auto-auditoría	Pendiente (entrega final)
H10	P08 Nueva compra (RF11)	Capa 2 · Heurística: eficiencia	Después de registrar una compra, el proveedor vuelve al primero de la lista y hay que elegirlo de nuevo para la siguiente.	Baja	Recordar el último proveedor elegido en la sesión.	Auto-auditoría	Pendiente (entrega final)

Hay al menos un hallazgo por capa: Capa 1 (H03), Capa 2 (H02, H05, H06, H07, H08, H10), Capa 3 (H01, H04) y Capa 4 (H09). Los dos hallazgos pendientes son de severidad Baja y se levantarán en la entrega final.

4.3. Levantamiento de hallazgos

Los hallazgos de severidad Alta se muestran con la captura antes y después de corregir. «Antes» es la versión del sistema entregada al inicio del APF3; «después», la versión actual. Ambas se ejecutaron con los mismos datos de demostración.

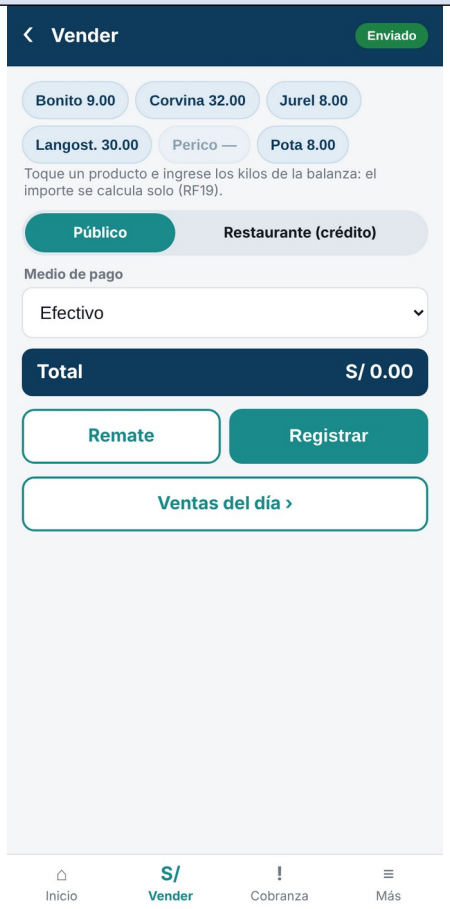
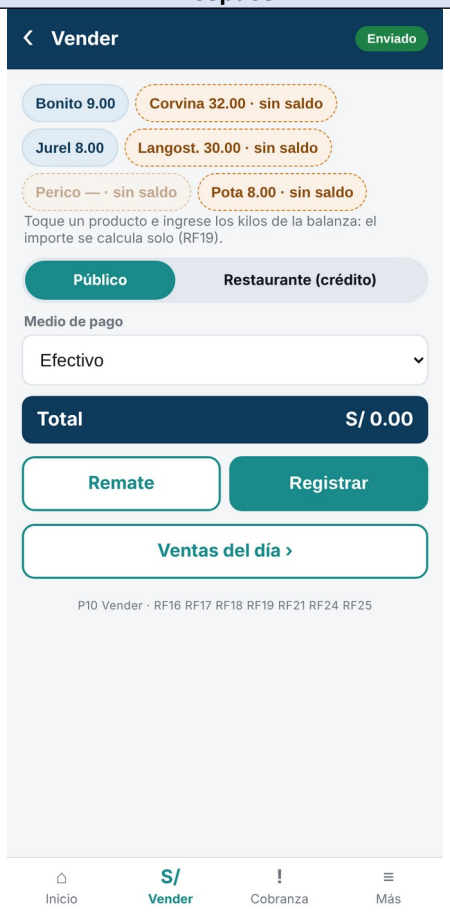
Antes	Después
	

Figura 45*. H01 · Vender: productos sin saldo (antes y después)*

Hallazgo: Los botones de productos sin saldo (Corvina, Langostino, Pota con 0 kg) se veían iguales a los demás; el vendedor recién se enteraba al registrar, con el aviso «Solo quedan 0.00 kg». **Corrección:** Marcar el botón con «· sin saldo» y borde naranja punteado, sin bloquearlo (el saldo puede estar mal contado).

Antes	Después

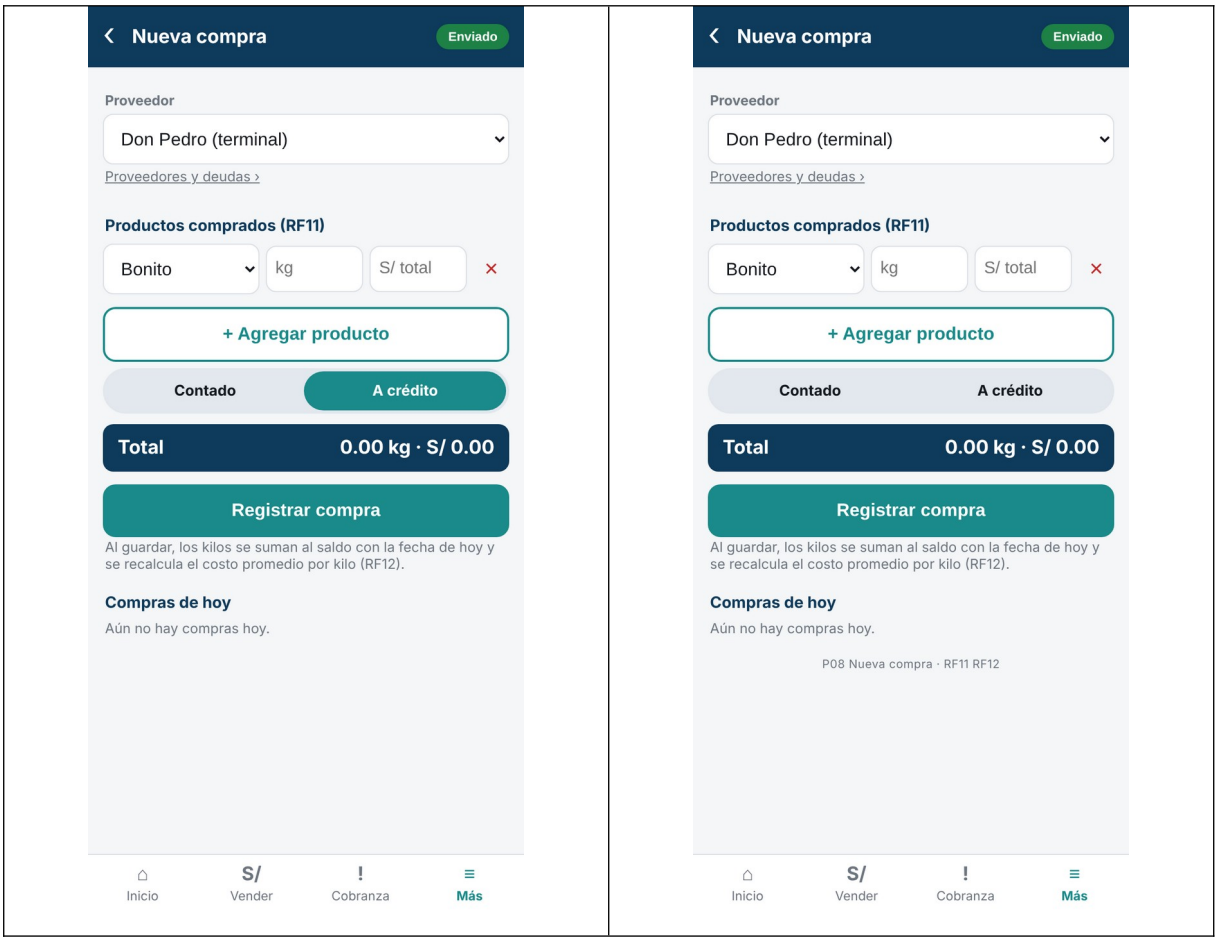


Figura 46. *H02 · Nueva compra: tipo de pago por defecto (antes y después)*

Hallazgo: La opción «A crédito» estaba marcada por defecto; una compra pagada al contado podía quedar como deuda con el proveedor y distorsionar RF14. **Corrección:** Quitar la opción por defecto: el sistema exige elegir Contado o A crédito («Indique si la compra fue al contado o a crédito»).

Los hallazgos de severidad Media y Baja figuran como «Levantado» o «Pendiente» en la columna Estado de la matriz 4.2. En el caso de H05, la ventana nativa del navegador no sale en las capturas; su texto era «Sincronice las 1 ventas pendientes antes de cuadrar.» y ahora se muestra el aviso «Sincronice la venta pendiente antes de cuadrar (RF39).» dentro de la página.

4.4. Capa 4: navegación por rol

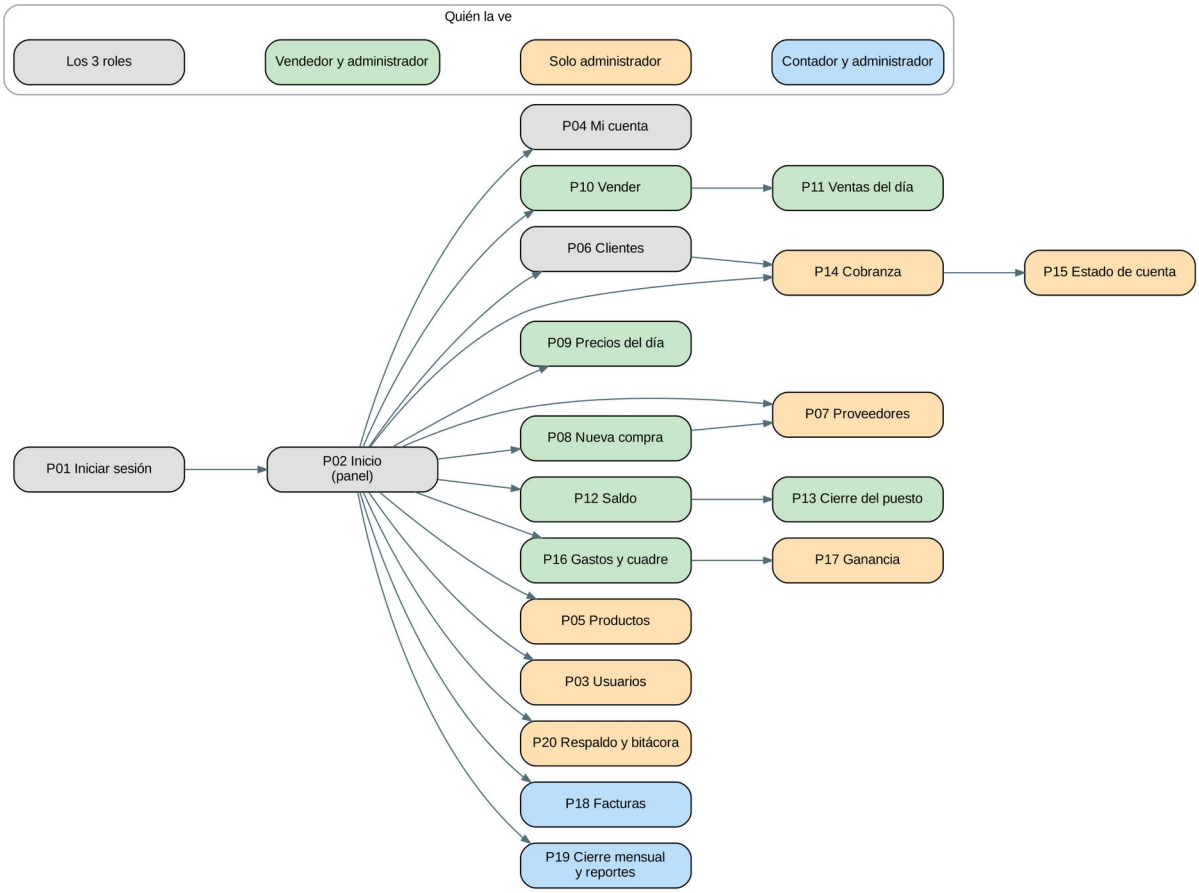


Figura 47. *Mapa de navegación actualizado: pantallas y rol que las ve*

Fuente: Elaboración propia.

Todas las pantallas tienen salida: la barra superior tiene el botón volver y la barra inferior lleva a Inicio y a Más. Cada rol ve solo sus opciones; si escribe la dirección de otra pantalla, el sistema muestra «Sin permiso» (capturas RF14-CP04, RF16-CP04 y RF49-CP04 en la sección 2.6).

Vendedor	Administrador del negocio	Contador

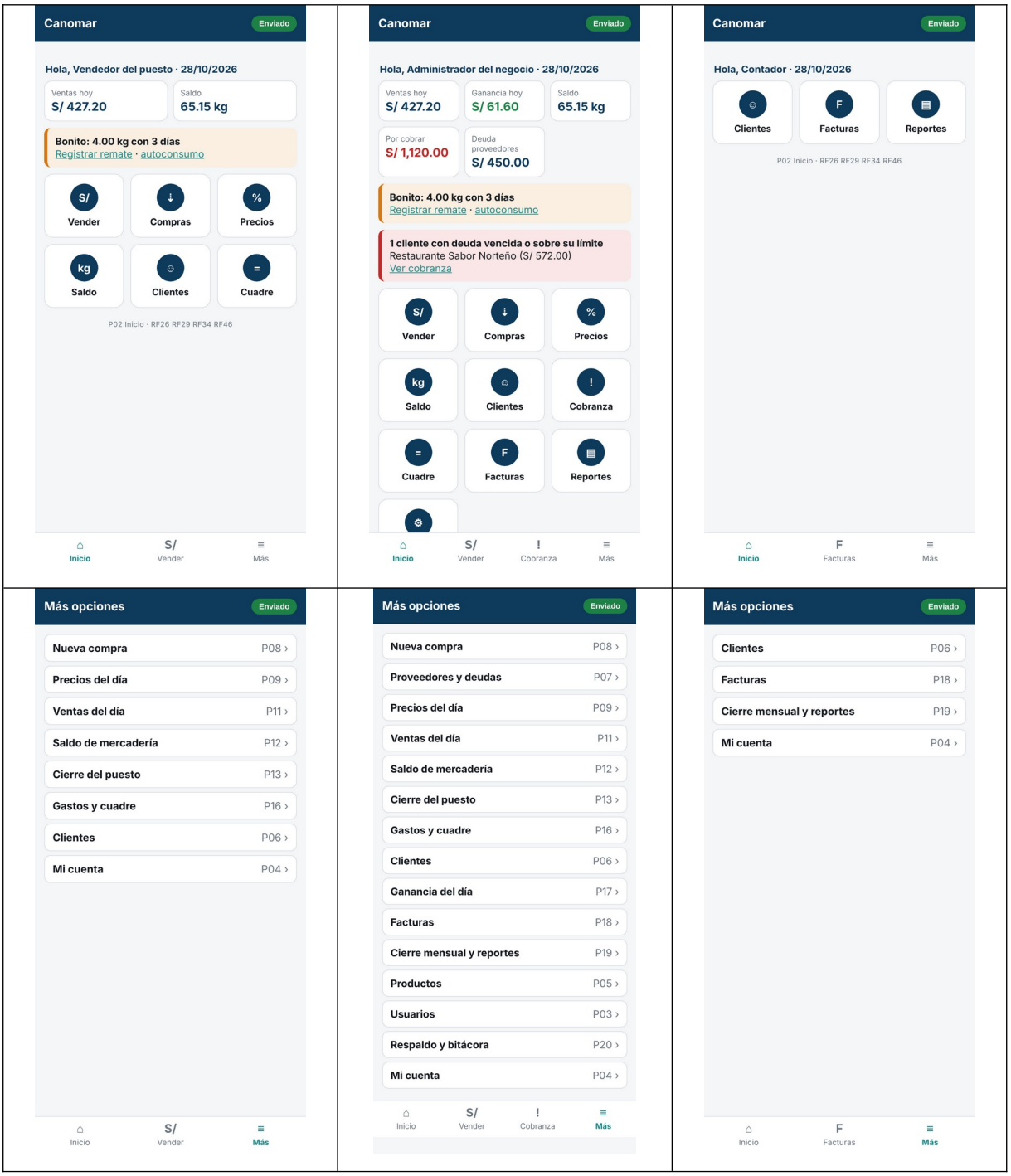


Figura 48. *Menú de cada rol: Inicio (arriba) y Más opciones (abajo)*

4.5. Cobertura

Cobertura = RF con pantalla funcional y sin hallazgos Altos pendientes / 50 × 100 = **50 / 50 × 100 = 100 %**. Los dos hallazgos Altos (H01 y H02) están levantados y los 50 RF tienen su pantalla en estado Funcional (Anexo C).

CAPÍTULO 5. PRODUCTO CONSTRUIDO Y MODELO DE DATOS FINAL

Este capítulo muestra la evidencia de los cuatro criterios de la rúbrica del producto final: completo, coherente, buenas prácticas y autoría.

5.1. Completo

Tabla 33*. Módulos del alcance del APF2 y su estado*

Módulo	RF que agrupa	Estado	Evidencia
Gestión de usuarios y seguridad	RF01 a RF05	Funciona	Tests de la sección 2.5 y capturas de 2.6
Catálogos	RF06 a RF10	Funciona	Tests de la sección 2.5 y capturas de 2.6
Compras y precios del día	RF11 a RF16	Funciona	Tests de la sección 2.5 y capturas de 2.6
Ventas en el puesto	RF17 a RF23	Funciona	Tests de la sección 2.5 y capturas de 2.6
Funcionamiento sin conexión	RF24 a RF26	Funciona	Tests de la sección 2.5 y capturas de 2.6
Saldo de mercadería	RF27 a RF30	Funciona	Tests de la sección 2.5 y capturas de 2.6
Créditos y cobranza	RF31 a RF37	Funciona	Tests de la sección 2.5 y capturas de 2.6
Cuadre diario	RF38 a RF41	Funciona	Tests de la sección 2.5 y capturas de 2.6
Facturas y cierre mensual	RF42 a RF45	Funciona	Tests de la sección 2.5 y capturas de 2.6
Reportes	RF46 a RF48	Funciona	Tests de la sección 2.5 y capturas de 2.6
Administración del sistema	RF49 a RF50	Funciona	Tests de la sección 2.5 y capturas de 2.6
Porcentaje	50 RF	11 de 11 módulos funcionan = 100 %	

5.2. Coherente: documentación y código alineados

5.2.1. Normalización de las 5 vistas físicas

Se normalizaron hasta 3FN cinco pantallas del sistema construido, las mismas que se trabajaron en el APF2, ahora con la captura de la pantalla real. Los datos calculados (totales, importes, saldos y diferencias) se excluyen del modelo.

Vista 1: Nueva compra (P08 · RF11, RF12)



Figura 49. *Pantalla real P08 Nueva compra con datos del día*

Fuente: Captura del sistema SVC-Canomar en ejecución.

Tabla 34. *Vista 1 · 0FN*

Tabla	Atributos
COMPRA_DIA	fecha, nombre_proveedor, ruc_proveedor, tipo_proveedor, tipo_pago, usuario_registra, { nombre_producto, categoria, kilos, costo_total, costo_kg }, total_compra

Qué separó 0FN: Entre llaves { } está el grupo repetitivo: una compra tiene varios productos. costo_kg y total_compra se muestran en pantalla pero se calculan.

Tabla 35. *Vista 1 · 1FN*

Tabla	Atributos
COMPRA	nro_compra (PK), fecha, nombre_proveedor, ruc_proveedor, tipo_proveedor, tipo_pago, usuario_registra, total_compra
DETALLE_COMPRA	nro_compra (PK), nombre_producto (PK), categoria, kilos, costo_total, costo_kg

Qué separó 1FN: Se separa el grupo repetitivo en DETALLE_COMPRA; cada fila es un producto de una compra. La llave es compuesta (nro_compra, nombre_producto).

Tabla 36. *Vista 1 · 2FN*

Tabla	Atributos
COMPRA	nro_compra (PK), fecha, nombre_proveedor, ruc_proveedor, tipo_proveedor, tipo_pago, usuario_registra, total_compra
PRODUCTO	id_producto (PK), nombre, categoria

Tabla	Atributos
DETALLE_COMPRA	nro_compra (PK), id_producto (PK), kilos, costo_total, costo_kg

Qué separó 2FN: categoria depende solo del producto (parte de la llave): es una dependencia parcial y se traslada a PRODUCTO, que recibe una llave subrogada.

Tabla 37. *Vista 1 · 3FN*

Tabla	Atributos
PROVEEDOR	id_proveedor (PK), nombre, ruc, tipo
USUARIO	id_usuario (PK), nombre_completo
COMPRA	id_compra (PK), id_proveedor (FK), id_usuario (FK), fecha, tipo_pago, estado
DETALLE_COMPRA	id_detalle_compra (PK), id_compra (FK), id_producto (FK), kilos, costo_total
PRODUCTO	id_producto (PK), nombre, nombre_corto, categoria, estado

Qué separó 3FN: ruc y tipo dependen del proveedor, no de la compra: dependencia transitiva que se resuelve con PROVEEDOR; lo mismo ocurre con el usuario. costo_kg y total_compra se eliminan porque se calculan (costo_total ÷ kilos y suma de detalles).

Vista 2: Vender (P10 · RF17, RF18, RF19)

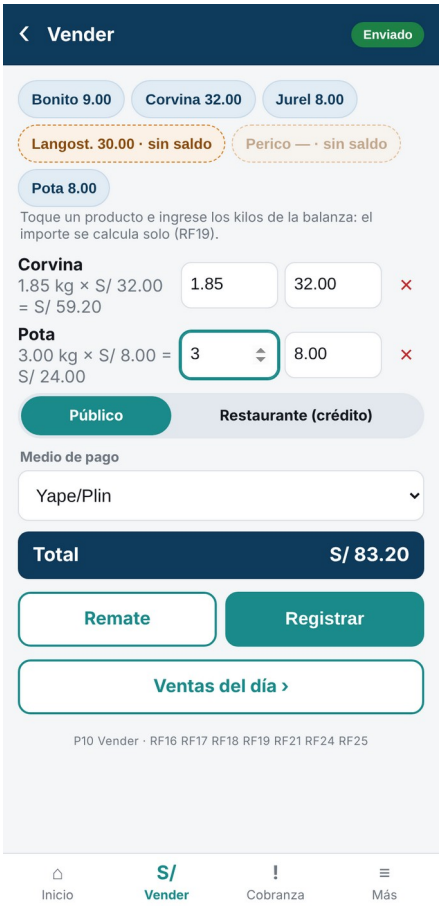


Figura 50. *Pantalla real P10 Vender con datos del día*

Fuente: Captura del sistema SVC-Canomar en ejecución.

Tabla 38. *Vista 2 · 0FN*

Tabla	Atributos
VENTA_PUESTO	nro_venta, fecha_hora, vendedor, tipo_pago, medio_pago, nombre_cliente, ruc_cliente, limite_credito, { nombre_producto, kilos, precio_kg, importe }, total_venta

Qué separó 0FN: El grupo repetitivo son los productos de la venta. importe y total_venta se calculan.

Tabla 39*. Vista 2 · 1FN*

Tabla	Atributos
VENTA	nro_venta (PK), fecha_hora, vendedor, tipo_pago, medio_pago, nombre_cliente, ruc_cliente, limite_credito, total_venta
DETALLE_VENTA	nro_venta (PK), nombre_producto (PK), kilos, precio_kg, importe

Qué separó 1FN: Se elimina el grupo repetitivo: cada fila de DETALLE_VENTA es un producto vendido. Llave compuesta (nro_venta, nombre_producto).

Tabla 40*. Vista 2 · 2FN*

Tabla	Atributos
VENTA	nro_venta (PK), fecha_hora, vendedor, tipo_pago, medio_pago, nombre_cliente, ruc_cliente, limite_credito, total_venta
PRODUCTO	id_producto (PK), nombre
DETALLE_VENTA	nro_venta (PK), id_producto (PK), kilos, precio_kg, importe

Qué separó 2FN: El nombre del producto se identifica por sí mismo: se reemplaza por id_producto. precio_kg se queda en el detalle porque es el precio aplicado en esa venta (puede ser de remate) y depende de la llave completa.

Tabla 41*. Vista 2 · 3FN*

Tabla	Atributos
CLIENTE	id_cliente (PK), nombre_comercial, ruc, limite_credito
USUARIO	id_usuario (PK), nombre_completo
VENTA	id_venta (PK), uuid_dispositivo (UQ), id_usuario (FK), id_cliente (FK, nulo si es público), fecha_hora, tipo_pago, medio_pago, es_remate, estado
DETALLE_VENTA	id_detalle_venta (PK), id_venta (FK), id_producto (FK), kilos, precio_kg, costo_kg

Qué separó 3FN: ruc_cliente y limite_credito dependen del cliente: dependencia transitiva que se resuelve con CLIENTE. importe y total_venta se eliminan porque se calculan. Se agregan uuid_dispositivo (evita duplicados al sincronizar, RNF12) y costo_kg (costo promedio vigente al vender, para la ganancia del día).

Vista 3: Cobranza (P14 · RF31, RF35)



Figura 51. *Pantalla real P14 Cobranza con datos del día*

Fuente: Captura del sistema SVC-Canomar en ejecución.

Tabla 42. *Vista 3 · 0FN*

Tabla	Atributos
COBRANZA_CLIENTE	nombre_cliente, ruc, telefono, plazo_dias, limite_credito, saldo, { fecha_abono, monto_abono, medio, ventas_pagadas }, { fecha_compromiso, monto_compromiso, estado_compromiso }

Qué separó 0FN: Hay dos grupos repetitivos independientes: los abonos y los compromisos de pago del cliente.

Tabla 43. *Vista 3 · 1FN*

Tabla	Atributos
ABONO_CLIENTE	ruc (PK), nro_abono (PK), nombre_cliente, telefono, plazo_dias, limite_credito, saldo, fecha_abono, monto_abono, medio
ABONO_VENTA	ruc (PK), nro_abono (PK), nro_venta (PK), monto_aplicado
COMPROMISO_CLIENTE	ruc (PK), nro_compromiso (PK), nombre_cliente, fecha_compromiso, monto_compromiso, estado_compromiso

Qué separó 1FN: Cada grupo repetitivo pasa a su propia tabla. Un abono puede pagar varias ventas y una venta puede pagarse con varios abonos: esa relación se separa en ABONO_VENTA.

Tabla 44. *Vista 3 · 2FN*

Tabla	Atributos
CLIENTE	ruc (PK), nombre_cliente, telefono, plazo_dias, limite_credito, saldo
ABONO	ruc (PK), nro_abono (PK), fecha_abono, monto_abono, medio
ABONO_VENTA	ruc (PK), nro_abono (PK), nro_venta (PK), monto_aplicado
COMPROMISO_PAGO	ruc (PK), nro_compromiso (PK), fecha_compromiso, monto_compromiso, estado_compromiso

Qué separó 2FN: Los datos del cliente dependen solo del ruc (parte de la llave): dependencia parcial que se traslada a CLIENTE.

Tabla 45. *Vista 3 · 3FN*

Tabla	Atributos
CLIENTE	id_cliente (PK), nombre_comercial, ruc (UQ), telefono, limite_credito, plazo_dias, estado
ABONO	id_abono (PK), id_cliente (FK), id_usuario (FK), fecha, monto, medio, estado
ABONO_VENTA	id_abono (PK, FK), id_venta (PK, FK), monto_aplicado
COMPROMISO_PAGO	id_compromiso (PK), id_cliente (FK), fecha_compromiso, monto, estado

Qué separó 3FN: saldo se elimina porque se calcula (ventas al crédito menos abonos, RNF22). Se usa una llave subrogada id_cliente porque hay clientes sin RUC; el RUC queda como UNIQUE.

Vista 4: Gastos y cuadre (P16 · RF38, RF39)

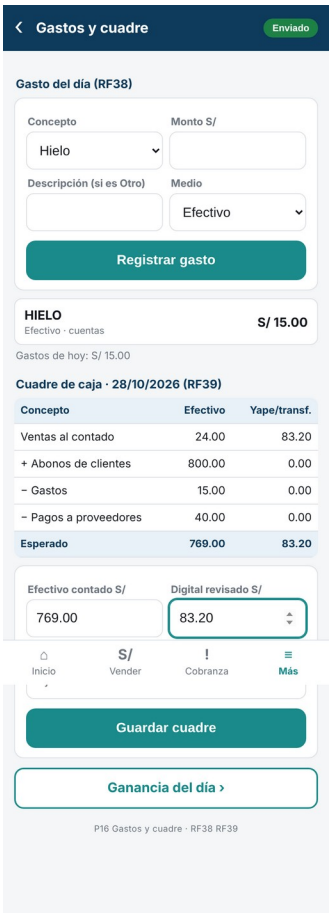


Figura 52. *Pantalla real P16 Gastos y cuadre con datos del día*

Fuente: Captura del sistema SVC-Canomar en ejecución.

Tabla 46*. Vista 4 · 0FN*

Tabla	Atributos
CUADRE_DIA	fecha, usuario_cuadre, { concepto_gasto, monto_gasto, medio_gasto, usuario_gasto }, efectivo_esperado, efectivo_contado, digital_esperado, digital_revisado, diferencia, motivo_diferencia, ganancia

Qué separó 0FN: El grupo repetitivo son los gastos del día.

Tabla 47*. Vista 4 · 1FN*

Tabla	Atributos
CUADRE	fecha (PK), usuario_cuadre, efectivo_esperado, efectivo_contado, digital_esperado, digital_revisado, diferencia, motivo_diferencia, ganancia
GASTO	fecha (PK), nro_gasto (PK), concepto_gasto, monto_gasto, medio_gasto, usuario_gasto

Qué separó 1FN: Se separan los gastos: cada fila es un gasto de un día.

Tabla 48*. Vista 4 · 2FN*

Tabla	Atributos
CUADRE	fecha (PK), usuario_cuadre, efectivo_esperado, efectivo_contado, digital_esperado, digital_revisado, diferencia, motivo_diferencia, ganancia
GASTO	fecha (PK), nro_gasto (PK), concepto_gasto, monto_gasto, medio_gasto, usuario_gasto

Qué separó 2FN: CUADRE tiene llave simple y en GASTO todos los atributos dependen de la llave completa: no hay dependencias parciales.

Tabla 49*. Vista 4 · 3FN*

Tabla	Atributos
USUARIO	id_usuario (PK), nombre_completo
CUADRE	id_cuadre (PK), id_usuario (FK), fecha (UQ), efectivo_esperado, efectivo_contado, digital_esperado, digital_revisado, motivo_diferencia, ganancia
GASTO	id_gasto (PK), id_usuario (FK), fecha, concepto, descripcion, monto, medio

Qué separó 3FN: diferencia se elimina porque se calcula (contado − esperado). efectivo_esperado, digital_esperado y ganancia se conservan como constancia del día: las ventas pueden anularse después y el cuadre debe mostrar el resultado con que se cerró. Los usuarios pasan a USUARIO.

Vista 5: Facturas (P18 · RF42, RF43)



Figura 53*. Pantalla real P18 Facturas con datos del día*

Fuente: Captura del sistema SVC-Canomar en ejecución.

Tabla 50*. Vista 5 · 0FN*

Tabla	Atributos
FACTURA_EMITIDA	serie, numero, fecha_emision, ruc_cliente, nombre_cliente, importe, sujeta_detraccion, { nro_venta, fecha_venta, total_venta }

Qué separó 0FN: El grupo repetitivo son las ventas que sustenta la factura.

Tabla 51*. Vista 5 · 1FN*

Tabla	Atributos
FACTURA	serie (PK), numero (PK), fecha_emision, ruc_cliente, nombre_cliente, importe, sujeta_detraccion
FACTURA_VENTA	serie (PK), numero (PK), nro_venta (PK), fecha_venta, total_venta

Qué separó 1FN: Se separan las ventas asociadas; la llave de FACTURA es compuesta (serie, numero).

Tabla 52*. Vista 5 · 2FN*

Tabla	Atributos
FACTURA	serie (PK), numero (PK), fecha_emision, ruc_cliente, nombre_cliente, importe, sujeta_detraccion
VENTA	nro_venta (PK), fecha_venta, total_venta
FACTURA_VENTA	serie (PK), numero (PK), nro_venta (PK)

Qué separó 2FN: fecha_venta y total_venta dependen solo de nro_venta (parte de la llave): dependencia parcial que se traslada a VENTA.

Tabla 53*. Vista 5 · 3FN*

Tabla	Atributos
CLIENTE	id_cliente (PK), nombre_comercial, ruc
FACTURA	id_factura (PK), id_cliente (FK), serie, numero, fecha_emision, importe, sujeta_detraccion · UQ (serie, numero)
VENTA	id_venta (PK), id_factura (FK, nulo hasta facturar), …

Qué separó 3FN: nombre_cliente depende del ruc: dependencia transitiva que se resuelve con CLIENTE. Como una venta se asocia a una sola factura (RF43), FACTURA_VENTA se reemplaza por la llave foránea id_factura en VENTA. total_venta se calcula desde DETALLE_VENTA.

5.2.2. Comparativo pantallas frente a base de datos

Se revisó cada campo de las 20 pantallas contra el modelo de datos. El comparativo completo está en el Anexo D; aquí solo se muestran los campos que no existían como columna o que estaban en el modelo del APF2 y se retiraron porque se calculan.

Tabla 54*. Campos de pantalla que quedaban libres y acción tomada*

Pantalla (RF)	Campo en pantalla	¿Existía en el modelo?	Acción tomada	Dónde quedó
P02 Inicio (RF26, RF29, RF34, RF46)	Ventas hoy	No	Es calculado: se obtiene por consulta	SUM de v_venta_total del día
P02 Inicio (RF26, RF29, RF34, RF46)	Ganancia hoy	No	Es calculado: se obtiene por consulta	Calculos.ganancia() en CajaServiceImpl
P02 Inicio (RF26, RF29, RF34, RF46)	Saldo (kg)	No	Es calculado: se obtiene por consulta	SUM(LOTE_SALDO.kilos_disponibles)
P02 Inicio (RF26, RF29, RF34, RF46)	Por cobrar	No	Es calculado: se obtiene por consulta	SUM de v_saldo_cliente.saldo
P02 Inicio (RF26, RF29, RF34, RF46)	Deuda proveedores	No	Es calculado: se obtiene por consulta	compras a crédito − PAGO_PROVEEDOR.monto
P02 Inicio (RF26, RF29, RF34, RF46)	Alerta «Bonito: 8 kg con 3 días»	No	Es calculado: se obtiene por consulta	LOTE_SALDO.fecha_ingreso vs CONFIGURACION.dias_alerta_saldo
P02 Inicio (RF26, RF29, RF34, RF46)	Indicador «Enviado / N pendientes»	No	Retirado del modelo: vive en el celular	localStorage del navegador (cola de ventas sin conexión)
P04 Mi cuenta (RF03, RF04)	Ventas sin enviar	No	Retirado del modelo: vive en el celular	localStorage del navegador
P06 Clientes (RF08, RF09, RF32)	Saldo	No	Es calculado: se obtiene por consulta	Vista V_SALDO_CLIENTE.saldo
P06 Clientes (RF08, RF09, RF32)	Disponible	No	Es calculado: se obtiene por consulta	limite_credito − saldo (SaldoCliente.disponible())
P06 Clientes (RF08, RF09, RF32)	Vencido	No	Es calculado: se obtiene por consulta	ventas con más días que plazo_dias (CreditoServiceImpl.saldo)
P06 Clientes (RF08, RF09, RF32)	Debe (por venta)	No	Es calculado: se obtiene por consulta	v_venta_total.total − ABONO_VENTA.monto_aplicado
P07 Proveedores y deudas (RF10, RF13, RF14)	Deuda por proveedor	No	Es calculado: se obtiene por consulta	SUM(DETALLE_COMPRA.costo_total) a crédito − SUM(PAGO_PROVEEDOR.monto)
P07 Proveedores y deudas (RF10, RF13, RF14)	Compra más antigua	No	Es calculado: se obtiene por consulta	MIN(COMPRA.fecha) con deuda

Pantalla (RF)	Campo en pantalla	¿Existía en el modelo?	Acción tomada	Dónde quedó
P08 Nueva compra (RF11, RF12)	Total de la compra (kg · S/)	Sí (APF2)	Retirado del modelo: es calculado	Se quitó COMPRA.total; se obtiene con SUM(DETALLE_COMPRA.costo_total)
P08 Nueva compra (RF11, RF12)	Costo por kilo del lote	No	Es calculado: se obtiene por consulta	costo_total ÷ kilos → LOTE_SALDO.costo_kg al registrar
P09 Precios del día (RF15, RF16)	Precio de ayer	No	Es calculado: se obtiene por consulta	PRECIO_DIA de la fecha anterior
P09 Precios del día (RF15, RF16)	Saldo kg / Agotado	No	Es calculado: se obtiene por consulta	SUM(LOTE_SALDO.kilos_disponibles) del producto
P10 Vender (RF16-RF19, RF21, RF24, RF25)	Importe por producto	Sí (APF2)	Retirado del modelo: es calculado	Se quitó DETALLE_VENTA.importe; kilos × precio_kg
P10 Vender (RF16-RF19, RF21, RF24, RF25)	Total	Sí (APF2)	Retirado del modelo: es calculado	Se quitó VENTA.total; vista V_VENTA_TOTAL.total
P10 Vender (RF16-RF19, RF21, RF24, RF25)	Disponible del restaurante	No	Es calculado: se obtiene por consulta	V_SALDO_CLIENTE (límite − saldo)
P11 Ventas del día (RF20, RF23)	Total por venta y del día	No	Es calculado: se obtiene por consulta	V_VENTA_TOTAL
P11 Ventas del día (RF20, RF23)	Totales por medio de pago	No	Es calculado: se obtiene por consulta	SUM(V_VENTA_TOTAL.total) por medio_pago
P12 Saldo de mercadería (RF22, RF28-RF30)	Días y color	No	Es calculado: se obtiene por consulta	hoy − fecha_ingreso (Calculos.colorSaldo)
P12 Saldo de mercadería (RF22, RF28-RF30)	Valor del ajuste	No	Es calculado: se obtiene por consulta	kilos × LOTE_SALDO.costo_kg → AJUSTE_SALDO.valor_costo
P13 Cierre del puesto (RF27)	Kilos esperados	No	Es calculado: se obtiene por consulta	SUM(LOTE_SALDO.kilos_disponibles)
P14 Cobranza (RF31, RF33-RF35)	Tramos 0-7, 8-15, 16-30, >30	No	Es calculado: se obtiene por consulta	Calculos.tramo(días de cada venta pendiente)
P14 Cobranza (RF31, RF33-RF35)	Saldo / vencido / disponible	No	Es calculado: se obtiene por consulta	V_SALDO_CLIENTE
P15 Estado de cuenta (RF36, RF37)	Saldo inicial	No	Es calculado: se obtiene por consulta	ventas a crédito − abonos antes del periodo
P15 Estado de cuenta (RF36, RF37)	Cargo / abono / saldo acumulado	No	Es calculado: se obtiene por consulta	V_VENTA_TOTAL y ABONO, ordenados por fecha
P16 Gastos y cuadre (RF38, RF39)	Diferencia	Sí (APF2)	Retirado del modelo: es calculado	Se quitó CUADRE.diferencia; contado − esperado (Calculos.diferencia)
P17 Ganancia del día (RF40, RF41)	Ventas, costo, pérdidas, gastos, ganancia	No	Es calculado: se obtiene por consulta	Calculos.ganancia(); costo desde DETALLE_VENTA.costo_kg
P18 Facturas (RF42-RF44)	Asociado / diferencia	No	Es calculado: se obtiene por consulta	SUM(V_VENTA_TOTAL.total) de sus ventas; importe − asociado
P18 Facturas (RF42-RF44)	Pendientes de facturar	No	Es calculado: se obtiene por consulta	ventas con id_factura NULL de clientes con RUC
P19 Cierre mensual y reportes (RF45, RF47, RF48)	Ventas, costo, gastos, compras, facturado, ganancia del mes	No	Es calculado: se obtiene por consulta	Consultas por rango de fechas
P19 Cierre mensual y reportes (RF45, RF47, RF48)	Días con ventas sin cuadre	No	Es calculado: se obtiene por consulta	días con VENTA sin CUADRE
P19 Cierre mensual y reportes (RF45, RF47, RF48)	Margen por producto	No	Es calculado: se obtiene por consulta	Calculos.margen()

Ningún campo de pantalla quedó libre: cada uno está en una columna del modelo, se calcula con una consulta o vista, o vive solo en el celular (la cola de ventas sin conexión). En el modelo final se retiraron cuatro columnas calculadas del APF2: compra.total, venta.total, detalle_venta.importe y cuadre.diferencia; no fue necesario agregar columnas nuevas.

5.2.3. Modelo de datos final

El diagrama entidad-relación en notación pata de gallo (archivo docs/diagramas/der_final.drawio) se muestra primero completo como vista general y luego en dos partes, en página horizontal, para que se lean las columnas.

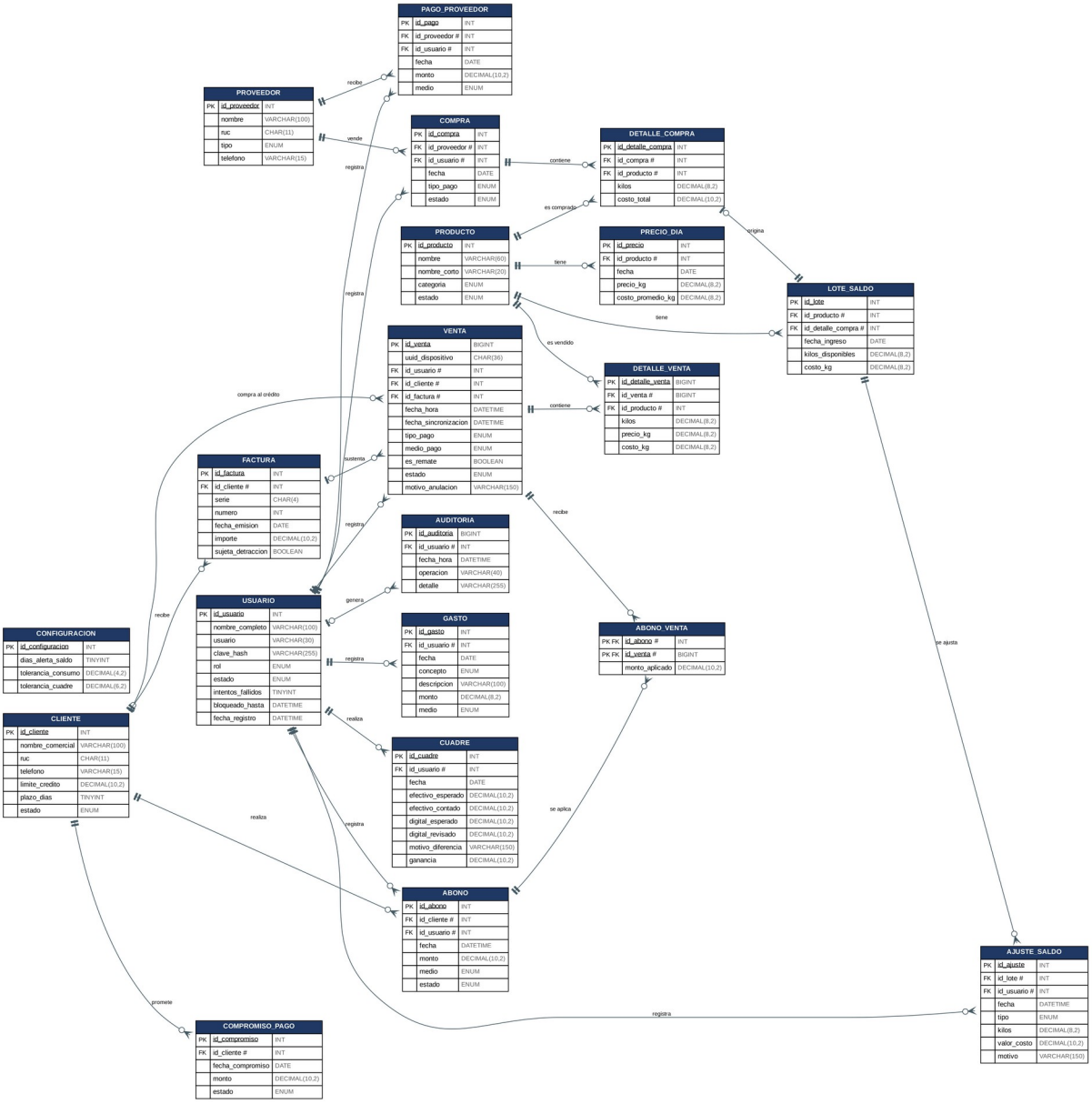


Figura 54. *Diagrama entidad-relación final completo (vista general)*

Fuente: Elaboración propia.

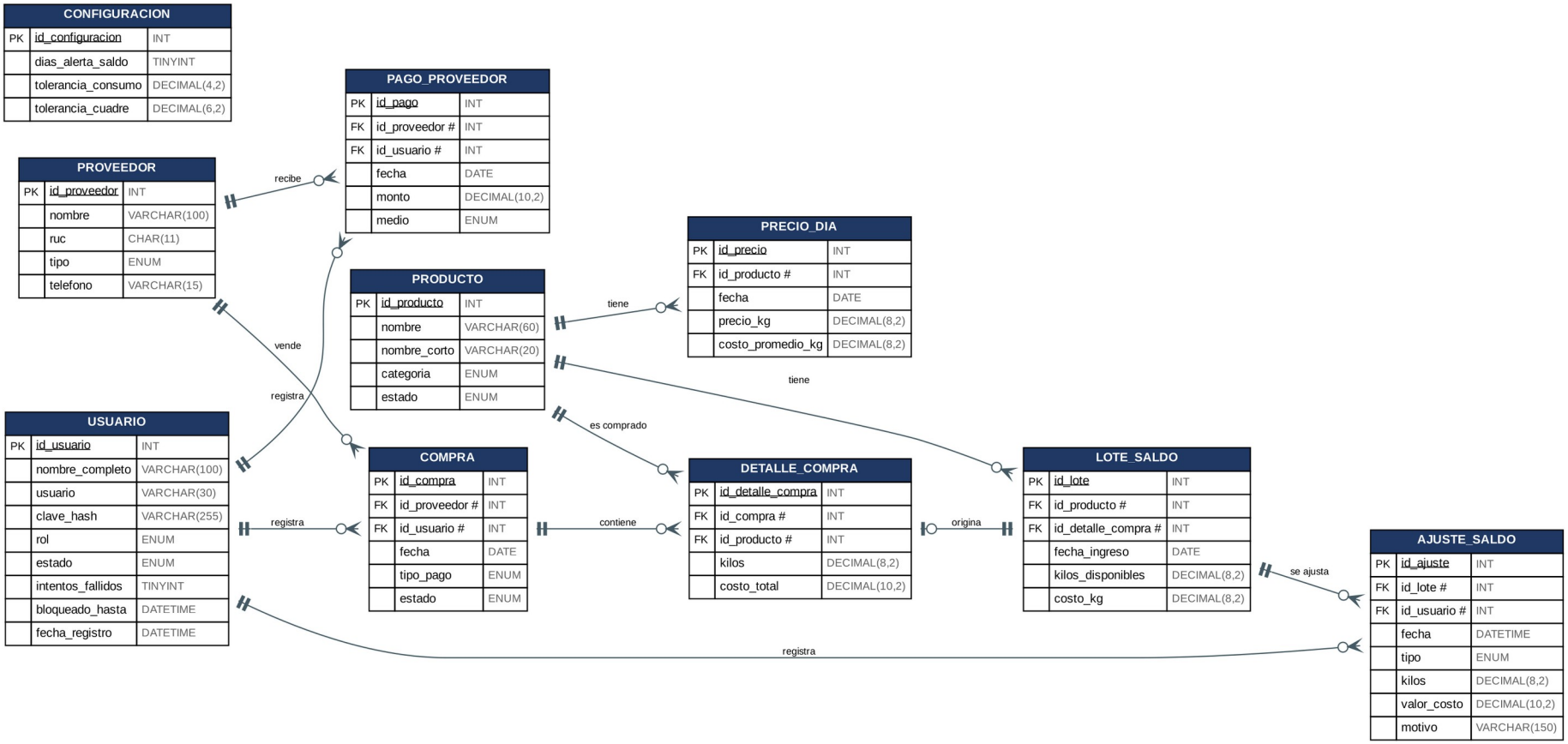


Figura 55*. DER final, parte A: compras, precios y saldo de mercadería (der_parte_a.drawio)*

Fuente: Elaboración propia.

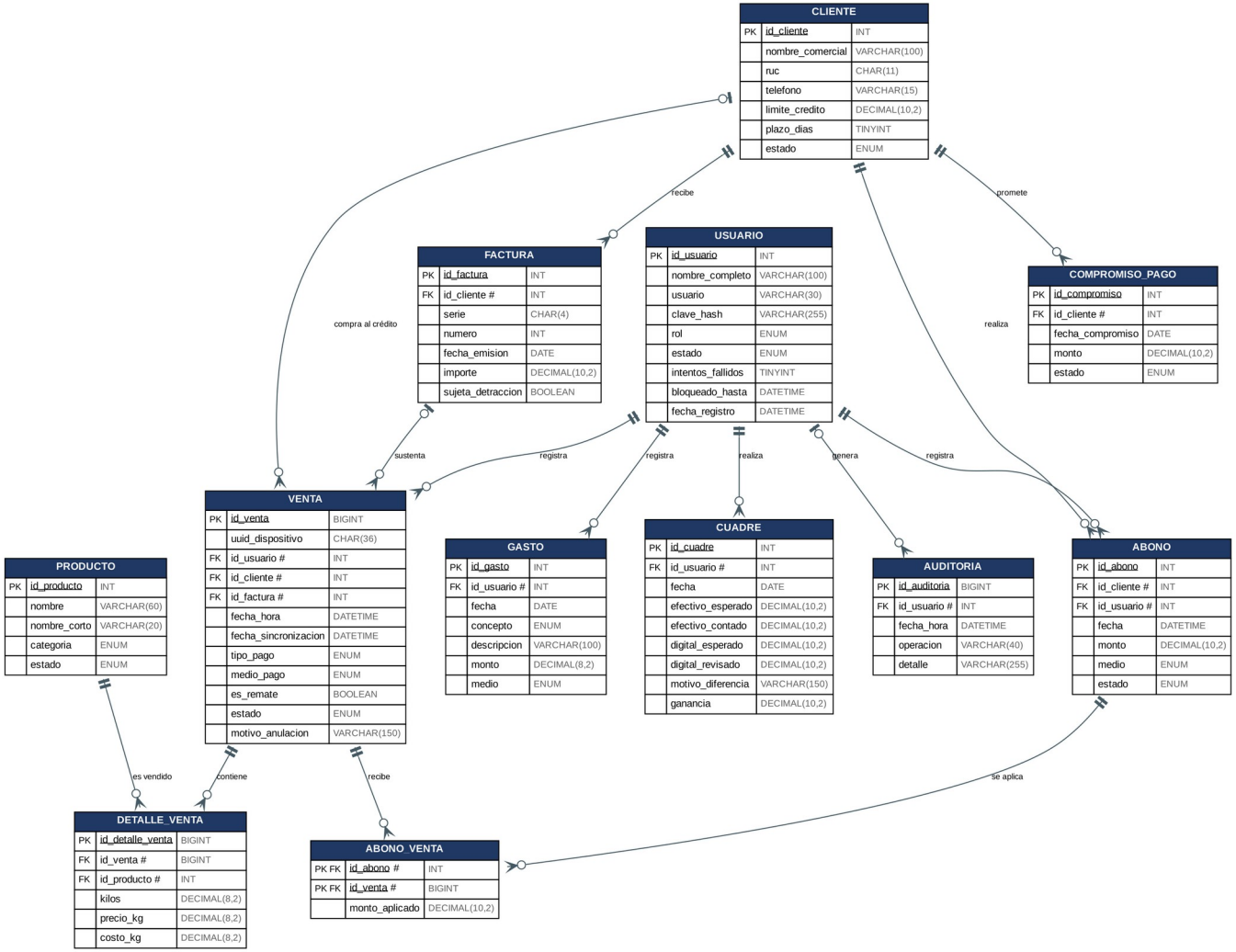


Figura 56. *DER final, parte B: ventas, créditos, caja, facturas y auditoría (der_parte_b.drawio)*

Fuente: Elaboración propia.

Cada relación se lee en los dos sentidos:

Tabla 55*. Relaciones del modelo final leídas de ida y vuelta*

Relación	Ida	Vuelta
USUARIO – COMPRA	Un usuario registra ninguna, una o muchas compras.	Cada compra corresponde a un solo usuario.
PROVEEDOR – COMPRA	Un proveedor vende ninguna, una o muchas compras.	Cada compra corresponde a un solo proveedor.
COMPRA – DETALLE_COMPRA	Una compra contiene ninguno, uno o muchos detalles de compra.	Cada detalle de compra corresponde a una sola compra.
PRODUCTO – DETALLE_COMPRA	Un producto aparece en ninguno, uno o muchos detalles de compra.	Cada detalle de compra corresponde a un solo producto.
PROVEEDOR – PAGO_PROVEEDOR	Un proveedor recibe ninguno, uno o muchos pagos a proveedor.	Cada pago a proveedor corresponde a un solo proveedor.
USUARIO – PAGO_PROVEEDOR	Un usuario registra ninguno, uno o muchos pagos a proveedor.	Cada pago a proveedor corresponde a un solo usuario.
PRODUCTO – PRECIO_DIA	Un producto tiene ninguno, uno o muchos precios del día.	Cada precio del día corresponde a un solo producto.
PRODUCTO – LOTE_SALDO	Un producto tiene ninguno, uno o muchos lotes de saldo.	Cada lote de saldo corresponde a un solo producto.
DETALLE_COMPRA – LOTE_SALDO	Un detalle de compra origina exactamente un lote de saldo.	Cada lote de saldo corresponde a ningún o un detalle de compra.
LOTE_SALDO – AJUSTE_SALDO	Un lote de saldo tiene ninguno, uno o muchos ajustes de saldo.	Cada ajuste de saldo corresponde a un solo lote de saldo.
USUARIO – AJUSTE_SALDO	Un usuario registra ninguno, uno o muchos ajustes de saldo.	Cada ajuste de saldo corresponde a un solo usuario.
USUARIO – VENTA	Un usuario registra ninguna, una o muchas ventas.	Cada venta corresponde a un solo usuario.
CLIENTE – VENTA	Un cliente tiene al crédito ninguna, una o muchas ventas.	Cada venta corresponde a ningún o un cliente.
FACTURA – VENTA	Una factura sustenta ninguna, una o muchas ventas.	Cada venta corresponde a ninguna o una factura.
VENTA – DETALLE_VENTA	Una venta contiene ninguno, uno o muchos detalles de venta.	Cada detalle de venta corresponde a una sola venta.
PRODUCTO – DETALLE_VENTA	Un producto aparece en ninguno, uno o muchos detalles de venta.	Cada detalle de venta corresponde a un solo producto.
CLIENTE – ABONO	Un cliente realiza ninguno, uno o muchos abonos.	Cada abono corresponde a un solo cliente.
USUARIO – ABONO	Un usuario registra ninguno, uno o muchos abonos.	Cada abono corresponde a un solo usuario.
ABONO – ABONO_VENTA	Un abono se reparte en ninguna, una o muchas aplicaciones de abono.	Cada aplicación de abono corresponde a un solo abono.
VENTA – ABONO_VENTA	Una venta recibe ninguna, una o muchas aplicaciones de abono.	Cada aplicación de abono corresponde a una sola venta.
CLIENTE – COMPROMISO_PAGO	Un cliente promete ninguno, uno o muchos compromisos de pago.	Cada compromiso de pago corresponde a un solo cliente.
CLIENTE – FACTURA	Un cliente recibe ninguna, una o muchas facturas.	Cada factura corresponde a un solo cliente.
USUARIO – GASTO	Un usuario registra ninguno, uno o muchos gastos.	Cada gasto corresponde a un solo usuario.
USUARIO – CUADRE	Un usuario realiza ninguno, uno o muchos cuadres.	Cada cuadre corresponde a un solo usuario.
USUARIO – AUDITORIA	Un usuario genera ninguno, uno o muchos registros de auditoría.	Cada registro de auditoría corresponde a ningún o un usuario.

Esquema lógico (PK subrayada en el diccionario; las llaves foráneas se marcan con #):

Tabla 56*. Esquema lógico del modelo final*

Tabla	Esquema
USUARIO	USUARIO (id_usuario [PK], nombre_completo, usuario, clave_hash, rol, estado, intentos_fallidos, bloqueado_hasta, fecha_registro)
CONFIGURACION	CONFIGURACION (id_configuracion [PK], dias_alerta_saldo, tolerancia_consumo, tolerancia_cuadre)
PRODUCTO	PRODUCTO (id_producto [PK], nombre, nombre_corto, categoria, estado)
CLIENTE	CLIENTE (id_cliente [PK], nombre_comercial, ruc, telefono, limite_credito, plazo_dias, estado)
PROVEEDOR	PROVEEDOR (id_proveedor [PK], nombre, ruc, tipo, telefono)

Tabla	Esquema
COMPRA	COMPRA (id_compra [PK], #id_proveedor, #id_usuario, fecha, tipo_pago, estado)
DETALLE_COMPRA	DETALLE_COMPRA (id_detalle_compra [PK], #id_compra, #id_producto, kilos, costo_total)
PAGO_PROVEEDOR	PAGO_PROVEEDOR (id_pago [PK], #id_proveedor, #id_usuario, fecha, monto, medio)
PRECIO_DIA	PRECIO_DIA (id_precio [PK], #id_producto, fecha, precio_kg, costo_promedio_kg)
LOTE_SALDO	LOTE_SALDO (id_lote [PK], #id_producto, #id_detalle_compra, fecha_ingreso, kilos_disponibles, costo_kg)
AJUSTE_SALDO	AJUSTE_SALDO (id_ajuste [PK], #id_lote, #id_usuario, fecha, tipo, kilos, valor_costo, motivo)
FACTURA	FACTURA (id_factura [PK], #id_cliente, serie, numero, fecha_emision, importe, sujeta_detraccion)
VENTA	VENTA (id_venta [PK], uuid_dispositivo, #id_usuario, #id_cliente, #id_factura, fecha_hora, fecha_sincronizacion, tipo_pago, medio_pago, es_remate, estado, motivo_anulacion)
DETALLE_VENTA	DETALLE_VENTA (id_detalle_venta [PK], #id_venta, #id_producto, kilos, precio_kg, costo_kg)
ABONO	ABONO (id_abono [PK], #id_cliente, #id_usuario, fecha, monto, medio, estado)
ABONO_VENTA	ABONO_VENTA (#id_abono [PK], #id_venta [PK], monto_aplicado)
COMPROMISO_PAGO	COMPROMISO_PAGO (id_compromiso [PK], #id_cliente, fecha_compromiso, monto, estado)
GASTO	GASTO (id_gasto [PK], #id_usuario, fecha, concepto, descripcion, monto, medio)
CUADRE	CUADRE (id_cuadre [PK], #id_usuario, fecha, efectivo_esperado, efectivo_contado, digital_esperado, digital_revisado, motivo_diferencia, ganancia)
AUDITORIA	AUDITORIA (id_auditoria [PK], #id_usuario, fecha_hora, operacion, detalle)

Diccionario de datos:

Tabla 57*. Diccionario de datos: USUARIO — Persona que usa el sistema con un rol*

Columna	Tipo	Llave	Restricciones	Descripción
id_usuario	INT	PK	NOT NULL AUTO_INCREMENT	Identificador del usuario
nombre_completo	VARCHAR(100)		NOT NULL	Nombre del usuario
usuario	VARCHAR(30)		NOT NULL UNIQUE	Nombre de acceso
clave_hash	VARCHAR(255)		NOT NULL	Contraseña cifrada con BCrypt
rol	ENUM('VENDEDOR','ADMINISTRADOR','CONTADOR')		NOT NULL	Rol del usuario
estado	ENUM('ACTIVO','INACTIVO')		NOT NULL DEFAULT 'ACTIVO'	Estado de la cuenta
intentos_fallidos	TINYINT		NOT NULL DEFAULT 0	Intentos fallidos consecutivos
bloqueado_hasta	DATETIME		NULL	Fin del bloqueo temporal
fecha_registro	DATETIME		NOT NULL	Fecha de creación de la cuenta

Tabla 58*. Diccionario de datos: CONFIGURACION — Parámetros del negocio (una sola fila)*

Columna	Tipo	Llave	Restricciones	Descripción
id_configuracion	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
dias_alerta_saldo	TINYINT		NOT NULL DEFAULT 3	Días para alertar mercadería antigua
tolerancia_consumo	DECIMAL(4,2)		NOT NULL DEFAULT 5.00	Diferencia % permitida al cerrar el puesto
tolerancia_cuadre	DECIMAL(6,2)		NOT NULL DEFAULT 5.00	Diferencia en soles permitida en el cuadre

Tabla 59*. Diccionario de datos: PRODUCTO — Especie o producto que se vende por kilo*

Columna	Tipo	Llave	Restricciones	Descripción
id_producto	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
nombre	VARCHAR(60)		NOT NULL UNIQUE	Nombre del producto
nombre_corto	VARCHAR(20)		NOT NULL	Nombre para la pantalla de venta

Columna	Tipo	Llave	Restricciones	Descripción
categoria	ENUM('PESCADO','MARISCO','MOLUSCO','OTRO')		NOT NULL	Categoría
estado	ENUM('ACTIVO','INACTIVO')		NOT NULL DEFAULT 'ACTIVO'	Disponible para venta

Tabla 60*. Diccionario de datos: CLIENTE — Restaurante o cliente frecuente con crédito*

Columna	Tipo	Llave	Restricciones	Descripción
id_cliente	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
nombre_comercial	VARCHAR(100)		NOT NULL	Nombre del cliente
ruc	CHAR(11)		NULL UNIQUE	RUC (obligatorio para facturar)
telefono	VARCHAR(15)		NULL	Teléfono de contacto
limite_credito	DECIMAL(10,2)		NOT NULL DEFAULT 0	Crédito máximo
plazo_dias	TINYINT		NOT NULL DEFAULT 7	Plazo de pago en días
estado	ENUM('ACTIVO','INACTIVO')		NOT NULL DEFAULT 'ACTIVO'	Estado

Tabla 61*. Diccionario de datos: PROVEEDOR — Mayorista o comerciante del terminal*

Columna	Tipo	Llave	Restricciones	Descripción
id_proveedor	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
nombre	VARCHAR(100)		NOT NULL	Nombre del proveedor
ruc	CHAR(11)		NULL	RUC si lo tiene
tipo	ENUM('PRINCIPAL','OCASIONAL')		NOT NULL	Tipo de proveedor
telefono	VARCHAR(15)		NULL	Teléfono

Tabla 62*. Diccionario de datos: COMPRA — Compra de mercadería de la madrugada*

Columna	Tipo	Llave	Restricciones	Descripción
id_compra	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
id_proveedor	INT	FK	NOT NULL · REFERENCES proveedor	Proveedor
id_usuario	INT	FK	NOT NULL · REFERENCES usuario	Quién registró
fecha	DATE		NOT NULL	Fecha de la compra
tipo_pago	ENUM('CONTADO','CREDITO')		NOT NULL	Forma de pago
estado	ENUM('VIGENTE','ANULADA')		NOT NULL DEFAULT 'VIGENTE'	Estado

Tabla 63*. Diccionario de datos: DETALLE_COMPRA — Producto comprado en una compra*

Columna	Tipo	Llave	Restricciones	Descripción
id_detalle_compra	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
id_compra	INT	FK	NOT NULL · REFERENCES compra ON DELETE CASCADE	Compra
id_producto	INT	FK	NOT NULL · REFERENCES producto	Producto
kilos	DECIMAL(8,2)		NOT NULL CHECK (kilos > 0)	Kilos comprados
costo_total	DECIMAL(10,2)		NOT NULL CHECK (costo_total > 0)	Costo del producto

Tabla 64*. Diccionario de datos: PAGO_PROVEEDOR — Pago total o parcial a un proveedor*

Columna	Tipo	Llave	Restricciones	Descripción
id_pago	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
id_proveedor	INT	FK	NOT NULL · REFERENCES proveedor	Proveedor
id_usuario	INT	FK	NOT NULL · REFERENCES usuario	Quién registró
fecha	DATE		NOT NULL	Fecha del pago
monto	DECIMAL(10,2)		NOT NULL CHECK (monto > 0)	Monto pagado
medio	ENUM('EFECTIVO','YAPE','TRANSFERENCIA')		NOT NULL	Medio de pago

Tabla 65*. Diccionario de datos: PRECIO_DIA — Precio de venta y costo promedio de un producto en un día*

Columna	Tipo	Llave	Restricciones	Descripción
id_precio	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
id_producto	INT	FK	NOT NULL · REFERENCES producto	Producto
fecha	DATE		NOT NULL	Día
precio_kg	DECIMAL(8,2)		NOT NULL	Precio de venta por kilo
costo_promedio_kg	DECIMAL(8,2)		NOT NULL	Costo promedio por kilo (RF12)

Tabla 66*. Diccionario de datos: LOTE_SALDO — Kilos de un producto según su fecha de ingreso (saldo con antigüedad)*

Columna	Tipo	Llave	Restricciones	Descripción
id_lote	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
id_producto	INT	FK	NOT NULL · REFERENCES producto	Producto
id_detalle_compra	INT	FK	NULL · REFERENCES detalle_compra	Compra de origen
fecha_ingreso	DATE		NOT NULL	Fecha de compra (antigüedad)
kilos_disponibles	DECIMAL(8,2)		NOT NULL CHECK (kilos_disponibles >= 0)	Kilos que quedan
costo_kg	DECIMAL(8,2)		NOT NULL	Costo por kilo del lote

Tabla 67*. Diccionario de datos: AJUSTE_SALDO — Cierre del puesto, autoconsumo o ajuste de kilos*

Columna	Tipo	Llave	Restricciones	Descripción
id_ajuste	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
id_lote	INT	FK	NOT NULL · REFERENCES lote_saldo	Lote afectado
id_usuario	INT	FK	NOT NULL · REFERENCES usuario	Quién registró
fecha	DATETIME		NOT NULL	Momento del ajuste
tipo	ENUM('CIERRE','AUTOCONSUMO','MERMA','DESECHO','CORRECCION')		NOT NULL	Tipo de ajuste
kilos	DECIMAL(8,2)		NOT NULL	Kilos (+ o −)
valor_costo	DECIMAL(10,2)		NOT NULL	Valor al costo
motivo	VARCHAR(150)		NULL	Motivo

Tabla 68*. Diccionario de datos: FACTURA — Factura emitida en SUNAT y registrada por el contador*

Columna	Tipo	Llave	Restricciones	Descripción
id_factura	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
id_cliente	INT	FK	NOT NULL · REFERENCES cliente	Cliente
serie	CHAR(4)		NOT NULL	Serie (E001)
numero	INT		NOT NULL	Número correlativo
fecha_emision	DATE		NOT NULL	Fecha de emisión
importe	DECIMAL(10,2)		NOT NULL	Importe total
sujeta_detraccion	BOOLEAN		NOT NULL DEFAULT FALSE	Control de detracción

Tabla 69*. Diccionario de datos: VENTA — Venta registrada en el puesto (contado, crédito o remate)*

Columna	Tipo	Llave	Restricciones	Descripción
id_venta	BIGINT	PK	NOT NULL AUTO_INCREMENT	Identificador
uuid_dispositivo	CHAR(36)		NOT NULL UNIQUE	Id generado en el celular (evita duplicados al sincronizar)
id_usuario	INT	FK	NOT NULL · REFERENCES usuario	Vendedor
id_cliente	INT	FK	NULL · REFERENCES cliente	Cliente (NULL = público)
id_factura	INT	FK	NULL · REFERENCES factura	Factura asociada (RF43)
fecha_hora	DATETIME		NOT NULL	Hora real de la venta
fecha_sincronizacion	DATETIME		NULL	Hora en que llegó al servidor
tipo_pago	ENUM('CONTADO','CREDITO')		NOT NULL	Forma de pago
medio_pago	ENUM('EFECTIVO','YAPE','TRANSFERENCIA')		NULL	Medio (solo contado)
es_remate	BOOLEAN		NOT NULL DEFAULT FALSE	Venta de mercadería antigua
estado	ENUM('VIGENTE','ANULADA')		NOT NULL DEFAULT 'VIGENTE'	Estado
motivo_anulacion	VARCHAR(150)		NULL	Motivo de anulación

Tabla 70*. Diccionario de datos: DETALLE_VENTA — Producto vendido en una venta*

Columna	Tipo	Llave	Restricciones	Descripción
id_detalle_venta	BIGINT	PK	NOT NULL AUTO_INCREMENT	Identificador
id_venta	BIGINT	FK	NOT NULL · REFERENCES venta ON DELETE CASCADE	Venta
id_producto	INT	FK	NOT NULL · REFERENCES producto	Producto
kilos	DECIMAL(8,2)		NOT NULL CHECK (kilos > 0)	Kilos de la balanza
precio_kg	DECIMAL(8,2)		NOT NULL	Precio aplicado
costo_kg	DECIMAL(8,2)		NOT NULL	Costo promedio al vender (ganancia)

Tabla 71*. Diccionario de datos: ABONO — Pago total o parcial de un cliente*

Columna	Tipo	Llave	Restricciones	Descripción
id_abono	INT	PK	NOT NULL AUTO_INCREMENT	Identificador

Columna	Tipo	Llave	Restricciones	Descripción
id_cliente	INT	FK	NOT NULL · REFERENCES cliente	Cliente
id_usuario	INT	FK	NOT NULL · REFERENCES usuario	Quién registró
fecha	DATETIME		NOT NULL	Fecha del abono
monto	DECIMAL(10,2)		NOT NULL CHECK (monto > 0)	Monto abonado
medio	ENUM('EFECTIVO','YAPE','TRANSFERENCIA')		NOT NULL	Medio de pago
estado	ENUM('VIGENTE','ANULADO')		NOT NULL DEFAULT 'VIGENTE'	Estado

Tabla 72*. Diccionario de datos: ABONO_VENTA — Parte de un abono aplicada a una venta al crédito*

Columna	Tipo	Llave	Restricciones	Descripción
id_abono	INT	PK, FK	NOT NULL · REFERENCES abono	Abono
id_venta	BIGINT	PK, FK	NOT NULL · REFERENCES venta	Venta pagada
monto_aplicado	DECIMAL(10,2)		NOT NULL	Monto aplicado a la venta

Tabla 73*. Diccionario de datos: COMPROMISO_PAGO — Fecha y monto que el cliente promete pagar*

Columna	Tipo	Llave	Restricciones	Descripción
id_compromiso	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
id_cliente	INT	FK	NOT NULL · REFERENCES cliente	Cliente
fecha_compromiso	DATE		NOT NULL	Fecha prometida
monto	DECIMAL(10,2)		NOT NULL	Monto prometido
estado	ENUM('PENDIENTE','CUMPLIDO','INCUMPLIDO')		NOT NULL DEFAULT 'PENDIENTE'	Estado

Tabla 74*. Diccionario de datos: GASTO — Gasto menor del día*

Columna	Tipo	Llave	Restricciones	Descripción
id_gasto	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
id_usuario	INT	FK	NOT NULL · REFERENCES usuario	Quién registró
fecha	DATE		NOT NULL	Día
concepto	ENUM('HIELO','BOLSAS','MOVILIDAD','ESTIBA','ESPACIO','OTRO')		NOT NULL	Concepto
descripcion	VARCHAR(100)		NULL	Detalle si es Otro
monto	DECIMAL(8,2)		NOT NULL CHECK (monto > 0)	Monto
medio	ENUM('EFECTIVO','YAPE','TRANSFERENCIA')		NOT NULL	Medio de pago

Tabla 75*. Diccionario de datos: CUADRE — Cuadre de caja y ganancia de un día*

Columna	Tipo	Llave	Restricciones	Descripción
id_cuadre	INT	PK	NOT NULL AUTO_INCREMENT	Identificador
id_usuario	INT	FK	NOT NULL · REFERENCES usuario	Quién cuadró
fecha	DATE		NOT NULL UNIQUE	Día cuadrado

Columna	Tipo	Llave	Restricciones	Descripción
efectivo_esperado	DECIMAL(10,2)		NOT NULL	Según el sistema
efectivo_contado	DECIMAL(10,2)		NOT NULL	Contado por el administrador
digital_esperado	DECIMAL(10,2)		NOT NULL	Yape / transferencias según el sistema
digital_revisado	DECIMAL(10,2)		NOT NULL	Revisado en las apps
motivo_diferencia	VARCHAR(150)		NULL	Explicación
ganancia	DECIMAL(10,2)		NOT NULL	Ganancia del día (RF40)

Tabla 76*. Diccionario de datos: AUDITORIA — Bitácora de operaciones (RF50)*

Columna	Tipo	Llave	Restricciones	Descripción
id_auditoria	BIGINT	PK	NOT NULL AUTO_INCREMENT	Identificador
id_usuario	INT	FK	NULL · REFERENCES usuario	Usuario (NULL = sistema)
fecha_hora	DATETIME		NOT NULL	Momento
operacion	VARCHAR(40)		NOT NULL	Tipo de operación
detalle	VARCHAR(255)		NOT NULL	Detalle

Vistas que reconstruyen datos calculados (no se guardan para respetar la 3FN):

Tabla 77*. Vistas SQL v_venta_total y v_saldo_cliente*

```
-- Total de cada venta = suma de kilos × precio de su detalle (RF19)
CREATE OR REPLACE VIEW v_venta_total AS
SELECT v.id_venta, v.fecha_hora, v.id_cliente, v.tipo_pago, v.estado,
       SUM(ROUND(d.kilos * d.precio_kg, 2)) AS total,
       SUM(ROUND(d.kilos * d.costo_kg, 2))  AS costo
FROM venta v
JOIN detalle_venta d ON d.id_venta = v.id_venta
GROUP BY v.id_venta, v.fecha_hora, v.id_cliente, v.tipo_pago, v.estado;

-- Saldo de cada cliente = ventas al crédito vigentes − abonos aplicados (RF32)
CREATE OR REPLACE VIEW v_saldo_cliente AS
SELECT c.id_cliente, c.nombre_comercial, c.limite_credito,
       COALESCE(t.vendido, 0) AS vendido_credito,
       COALESCE(a.pagado, 0)  AS abonado,
       COALESCE(t.vendido, 0) - COALESCE(a.pagado, 0) AS saldo
FROM cliente c
LEFT JOIN (SELECT vt.id_cliente, SUM(vt.total) AS vendido
           FROM v_venta_total vt
           WHERE vt.tipo_pago = 'CREDITO' AND vt.estado = 'VIGENTE'
           GROUP BY vt.id_cliente) t ON t.id_cliente = c.id_cliente
LEFT JOIN (SELECT ab.id_cliente, SUM(av.monto_aplicado) AS pagado
           FROM abono ab JOIN abono_venta av ON av.id_abono = ab.id_abono
           JOIN venta v ON v.id_venta = av.id_venta
           WHERE ab.estado = 'VIGENTE' AND v.estado = 'VIGENTE'
           GROUP BY ab.id_cliente) a ON a.id_cliente = c.id_cliente;
```

Script SQL completo del modelo final (src/main/resources/schema.sql, también en el Anexo E con los datos de demostración):

Tabla 78*. Script SQL del modelo final (schema.sql)*

```
-- SVC-Canomar · esquema MySQL 8 (mismo modelo físico del APF2, sección 2.6)
-- Se ejecuta en cada arranque. IF NOT EXISTS evita errores si las tablas ya existen.

CREATE TABLE IF NOT EXISTS usuario (
  id_usuario INT NOT NULL AUTO_INCREMENT,
  nombre_completo VARCHAR(100) NOT NULL,
  usuario VARCHAR(30) NOT NULL UNIQUE,
  clave_hash VARCHAR(255) NOT NULL,
  rol ENUM('VENDEDOR','ADMINISTRADOR','CONTADOR') NOT NULL,
```

```
  estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
  intentos_fallidos TINYINT NOT NULL DEFAULT 0,
  bloqueado_hasta DATETIME NULL,
  fecha_registro DATETIME NOT NULL,
  CONSTRAINT pk_usuario PRIMARY KEY (id_usuario)
) ENGINE=InnoDB;

-- Parámetros del negocio (una sola fila)

CREATE TABLE IF NOT EXISTS configuracion (
  id_configuracion INT NOT NULL AUTO_INCREMENT,
  dias_alerta_saldo TINYINT NOT NULL DEFAULT 3,
  tolerancia_consumo DECIMAL(4,2) NOT NULL DEFAULT 5.00,
  tolerancia_cuadre DECIMAL(6,2) NOT NULL DEFAULT 5.00,
  CONSTRAINT pk_configuracion PRIMARY KEY (id_configuracion)
) ENGINE=InnoDB;

-- Especie o producto que se vende por kilo

CREATE TABLE IF NOT EXISTS producto (
  id_producto INT NOT NULL AUTO_INCREMENT,
  nombre VARCHAR(60) NOT NULL UNIQUE,
  nombre_corto VARCHAR(20) NOT NULL,
  categoria ENUM('PESCADO','MARISCO','MOLUSCO','OTRO') NOT NULL,
  estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
  CONSTRAINT pk_producto PRIMARY KEY (id_producto)
) ENGINE=InnoDB;

-- Restaurante o cliente frecuente con crédito

CREATE TABLE IF NOT EXISTS cliente (
  id_cliente INT NOT NULL AUTO_INCREMENT,
  nombre_comercial VARCHAR(100) NOT NULL,
  ruc CHAR(11) NULL UNIQUE,
  telefono VARCHAR(15) NULL,
  limite_credito DECIMAL(10,2) NOT NULL DEFAULT 0,
  plazo_dias TINYINT NOT NULL DEFAULT 7,
  estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
  CONSTRAINT pk_cliente PRIMARY KEY (id_cliente)
) ENGINE=InnoDB;

-- Mayorista o comerciante del terminal

CREATE TABLE IF NOT EXISTS proveedor (
  id_proveedor INT NOT NULL AUTO_INCREMENT,
  nombre VARCHAR(100) NOT NULL,
  ruc CHAR(11) NULL,
  tipo ENUM('PRINCIPAL','OCASIONAL') NOT NULL,
  telefono VARCHAR(15) NULL,
  CONSTRAINT pk_proveedor PRIMARY KEY (id_proveedor)
) ENGINE=InnoDB;

-- Compra de mercadería de la madrugada

CREATE TABLE IF NOT EXISTS compra (
  id_compra INT NOT NULL AUTO_INCREMENT,
  id_proveedor INT NOT NULL,
  id_usuario INT NOT NULL,
  fecha DATE NOT NULL,
  tipo_pago ENUM('CONTADO','CREDITO') NOT NULL,
  estado ENUM('VIGENTE','ANULADA') NOT NULL DEFAULT 'VIGENTE',
  CONSTRAINT pk_compra PRIMARY KEY (id_compra),
  CONSTRAINT fk_compra_proveedor FOREIGN KEY (id_proveedor) REFERENCES proveedor(id_proveedor),
  CONSTRAINT fk_compra_usuario FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB;

-- Producto comprado en una compra

CREATE TABLE IF NOT EXISTS detalle_compra (
  id_detalle_compra INT NOT NULL AUTO_INCREMENT,
  id_compra INT NOT NULL,
  id_producto INT NOT NULL,
  kilos DECIMAL(8,2) NOT NULL CHECK (kilos > 0),
  costo_total DECIMAL(10,2) NOT NULL CHECK (costo_total > 0),
  CONSTRAINT pk_detalle_compra PRIMARY KEY (id_detalle_compra),
  CONSTRAINT fk_detalle_compra_compra FOREIGN KEY (id_compra) REFERENCES compra(id_compra) ON
DELETE CASCADE,
  CONSTRAINT fk_detalle_compra_producto FOREIGN KEY (id_producto) REFERENCES producto(id_producto)
) ENGINE=InnoDB;

-- Pago total o parcial a un proveedor
```

```
CREATE TABLE IF NOT EXISTS pago_proveedor (
  id_pago INT NOT NULL AUTO_INCREMENT,
  id_proveedor INT NOT NULL,
  id_usuario INT NOT NULL,
  fecha DATE NOT NULL,
  monto DECIMAL(10,2) NOT NULL CHECK (monto > 0),
  medio ENUM('EFECTIVO','YAPE','TRANSFERENCIA') NOT NULL,
  CONSTRAINT pk_pago_proveedor PRIMARY KEY (id_pago),
  CONSTRAINT fk_pago_proveedor_proveedor FOREIGN KEY (id_proveedor) REFERENCES
proveedor(id_proveedor),
  CONSTRAINT fk_pago_proveedor_usuario FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB;

-- Precio de venta y costo promedio de un producto en un día

CREATE TABLE IF NOT EXISTS precio_dia (
  id_precio INT NOT NULL AUTO_INCREMENT,
  id_producto INT NOT NULL,
  fecha DATE NOT NULL,
  precio_kg DECIMAL(8,2) NOT NULL,
  costo_promedio_kg DECIMAL(8,2) NOT NULL,
  CONSTRAINT pk_precio_dia PRIMARY KEY (id_precio),
  CONSTRAINT uq_precio_dia_id_producto_fecha UNIQUE (id_producto, fecha),
  CONSTRAINT fk_precio_dia_producto FOREIGN KEY (id_producto) REFERENCES producto(id_producto)
) ENGINE=InnoDB;

-- Kilos de un producto según su fecha de ingreso (saldo con antigüedad)

CREATE TABLE IF NOT EXISTS lote_saldo (
  id_lote INT NOT NULL AUTO_INCREMENT,
  id_producto INT NOT NULL,
  id_detalle_compra INT NULL,
  fecha_ingreso DATE NOT NULL,
  kilos_disponibles DECIMAL(8,2) NOT NULL CHECK (kilos_disponibles >= 0),
  costo_kg DECIMAL(8,2) NOT NULL,
  CONSTRAINT pk_lote_saldo PRIMARY KEY (id_lote),
  CONSTRAINT fk_lote_saldo_producto FOREIGN KEY (id_producto) REFERENCES producto(id_producto),
  CONSTRAINT fk_lote_saldo_detalle_compra FOREIGN KEY (id_detalle_compra) REFERENCES
detalle_compra(id_detalle_compra),
  INDEX ix_lote_producto (id_producto, fecha_ingreso)
) ENGINE=InnoDB;

-- Cierre del puesto, autoconsumo o ajuste de kilos

CREATE TABLE IF NOT EXISTS ajuste_saldo (
  id_ajuste INT NOT NULL AUTO_INCREMENT,
  id_lote INT NOT NULL,
  id_usuario INT NOT NULL,
  fecha DATETIME NOT NULL,
  tipo ENUM('CIERRE','AUTOCONSUMO','MERMA','DESECHO','CORRECCION') NOT NULL,
  kilos DECIMAL(8,2) NOT NULL,
  valor_costo DECIMAL(10,2) NOT NULL,
  motivo VARCHAR(150) NULL,
  CONSTRAINT pk_ajuste_saldo PRIMARY KEY (id_ajuste),
  CONSTRAINT fk_ajuste_saldo_lote FOREIGN KEY (id_lote) REFERENCES lote_saldo(id_lote),
  CONSTRAINT fk_ajuste_saldo_usuario FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB;

-- Factura emitida en SUNAT y registrada por el contador

CREATE TABLE IF NOT EXISTS factura (
  id_factura INT NOT NULL AUTO_INCREMENT,
  id_cliente INT NOT NULL,
  serie CHAR(4) NOT NULL,
  numero INT NOT NULL,
  fecha_emision DATE NOT NULL,
  importe DECIMAL(10,2) NOT NULL,
  sujeta_detraccion BOOLEAN NOT NULL DEFAULT FALSE,
  CONSTRAINT pk_factura PRIMARY KEY (id_factura),
  CONSTRAINT uq_factura_serie_numero UNIQUE (serie, numero),
  CONSTRAINT fk_factura_cliente FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
) ENGINE=InnoDB;

-- Venta registrada en el puesto (contado, crédito o remate)

CREATE TABLE IF NOT EXISTS venta (
  id_venta BIGINT NOT NULL AUTO_INCREMENT,
  uuid_dispositivo CHAR(36) NOT NULL UNIQUE,
  id_usuario INT NOT NULL,
  id_cliente INT NULL,
  id_factura INT NULL,
  fecha_hora DATETIME NOT NULL,
```

```
  fecha_sincronizacion DATETIME NULL,
  tipo_pago ENUM('CONTADO','CREDITO') NOT NULL,
  medio_pago ENUM('EFECTIVO','YAPE','TRANSFERENCIA') NULL,
  es_remate BOOLEAN NOT NULL DEFAULT FALSE,
  estado ENUM('VIGENTE','ANULADA') NOT NULL DEFAULT 'VIGENTE',
  motivo_anulacion VARCHAR(150) NULL,
  CONSTRAINT pk_venta PRIMARY KEY (id_venta),
  CONSTRAINT fk_venta_usuario FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
  CONSTRAINT fk_venta_cliente FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente),
  CONSTRAINT fk_venta_factura FOREIGN KEY (id_factura) REFERENCES factura(id_factura),
  INDEX ix_venta_fecha (fecha_hora),
  INDEX ix_venta_cliente (id_cliente, estado)
) ENGINE=InnoDB;

-- Producto vendido en una venta

CREATE TABLE IF NOT EXISTS detalle_venta (
  id_detalle_venta BIGINT NOT NULL AUTO_INCREMENT,
  id_venta BIGINT NOT NULL,
  id_producto INT NOT NULL,
  kilos DECIMAL(8,2) NOT NULL CHECK (kilos > 0),
  precio_kg DECIMAL(8,2) NOT NULL,
  costo_kg DECIMAL(8,2) NOT NULL,
  CONSTRAINT pk_detalle_venta PRIMARY KEY (id_detalle_venta),
  CONSTRAINT fk_detalle_venta_venta FOREIGN KEY (id_venta) REFERENCES venta(id_venta) ON DELETE
CASCADE,
  CONSTRAINT fk_detalle_venta_producto FOREIGN KEY (id_producto) REFERENCES producto(id_producto)
) ENGINE=InnoDB;

-- Pago total o parcial de un cliente

CREATE TABLE IF NOT EXISTS abono (
  id_abono INT NOT NULL AUTO_INCREMENT,
  id_cliente INT NOT NULL,
  id_usuario INT NOT NULL,
  fecha DATETIME NOT NULL,
  monto DECIMAL(10,2) NOT NULL CHECK (monto > 0),
  medio ENUM('EFECTIVO','YAPE','TRANSFERENCIA') NOT NULL,
  estado ENUM('VIGENTE','ANULADO') NOT NULL DEFAULT 'VIGENTE',
  CONSTRAINT pk_abono PRIMARY KEY (id_abono),
  CONSTRAINT fk_abono_cliente FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente),
  CONSTRAINT fk_abono_usuario FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
  INDEX ix_abono_cliente (id_cliente)
) ENGINE=InnoDB;

-- Parte de un abono aplicada a una venta al crédito

CREATE TABLE IF NOT EXISTS abono_venta (
  id_abono INT NOT NULL,
  id_venta BIGINT NOT NULL,
  monto_aplicado DECIMAL(10,2) NOT NULL,
  CONSTRAINT pk_abono_venta PRIMARY KEY (id_abono, id_venta),
  CONSTRAINT fk_abono_venta_abono FOREIGN KEY (id_abono) REFERENCES abono(id_abono),
  CONSTRAINT fk_abono_venta_venta FOREIGN KEY (id_venta) REFERENCES venta(id_venta)
) ENGINE=InnoDB;

-- Fecha y monto que el cliente promete pagar

CREATE TABLE IF NOT EXISTS compromiso_pago (
  id_compromiso INT NOT NULL AUTO_INCREMENT,
  id_cliente INT NOT NULL,
  fecha_compromiso DATE NOT NULL,
  monto DECIMAL(10,2) NOT NULL,
  estado ENUM('PENDIENTE','CUMPLIDO','INCUMPLIDO') NOT NULL DEFAULT 'PENDIENTE',
  CONSTRAINT pk_compromiso_pago PRIMARY KEY (id_compromiso),
  CONSTRAINT fk_compromiso_pago_cliente FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
) ENGINE=InnoDB;

-- Gasto menor del día

CREATE TABLE IF NOT EXISTS gasto (
  id_gasto INT NOT NULL AUTO_INCREMENT,
  id_usuario INT NOT NULL,
  fecha DATE NOT NULL,
  concepto ENUM('HIELO','BOLSAS','MOVILIDAD','ESTIBA','ESPACIO','OTRO') NOT NULL,
  descripcion VARCHAR(100) NULL,
  monto DECIMAL(8,2) NOT NULL CHECK (monto > 0),
  medio ENUM('EFECTIVO','YAPE','TRANSFERENCIA') NOT NULL,
  CONSTRAINT pk_gasto PRIMARY KEY (id_gasto),
  CONSTRAINT fk_gasto_usuario FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB;
```

```
-- Cuadre de caja y ganancia de un día

CREATE TABLE IF NOT EXISTS cuadre (
  id_cuadre INT NOT NULL AUTO_INCREMENT,
  id_usuario INT NOT NULL,
  fecha DATE NOT NULL UNIQUE,
  efectivo_esperado DECIMAL(10,2) NOT NULL,
  efectivo_contado DECIMAL(10,2) NOT NULL,
  digital_esperado DECIMAL(10,2) NOT NULL,
  digital_revisado DECIMAL(10,2) NOT NULL,
  motivo_diferencia VARCHAR(150) NULL,
  ganancia DECIMAL(10,2) NOT NULL,
  CONSTRAINT pk_cuadre PRIMARY KEY (id_cuadre),
  CONSTRAINT fk_cuadre_usuario FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB;

-- Bitácora de operaciones (RF50)

CREATE TABLE IF NOT EXISTS auditoria (
  id_auditoria BIGINT NOT NULL AUTO_INCREMENT,
  id_usuario INT NULL,
  fecha_hora DATETIME NOT NULL,
  operacion VARCHAR(40) NOT NULL,
  detalle VARCHAR(255) NOT NULL,
  CONSTRAINT pk_auditoria PRIMARY KEY (id_auditoria),
  CONSTRAINT fk_auditoria_usuario FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
) ENGINE=InnoDB;
```

```
Terminal · MariaDB 10.11 (compatible con MySQL 8)
$ mariadb -u root -p --default-character-set=utf8mb4 -vvv < Anexo_E_svc_canomar_final.sql   (base: svc_canomar_prueba)

CREATE TABLE IF NOT EXISTS usuario
CREATE TABLE IF NOT EXISTS configuracion
CREATE TABLE IF NOT EXISTS producto
CREATE TABLE IF NOT EXISTS cliente
   … (22 sentencias CREATE en total) …
CREATE TABLE IF NOT EXISTS auditoria
CREATE OR REPLACE VIEW v_venta_total AS
CREATE OR REPLACE VIEW v_saldo_cliente AS

Resultado del cliente: 34 sentencias con "Query OK", 0 errores
Query OK, 1 row affected (0.000 sec)
Query OK, 3 rows affected (0.002 sec)
Bye

$ mariadb -t svc_canomar_prueba -e "SELECT … tablas, vistas y v_saldo_cliente"
+--------+
| tablas |
+--------+
|     20 |
+--------+
+-----------------+
| vistas          |
+-----------------+
| v_saldo_cliente |
| v_venta_total   |
+-----------------+
+------------+---------------------------+-----------------+---------+---------+
| id_cliente | nombre_comercial          | vendido_credito | abonado | saldo   |
+------------+---------------------------+-----------------+---------+---------+
|          1 | Cevichería El Muelle      |          228.00 |    0.00 |  228.00 |
|          2 | Restaurante Sabor Norteño |         1372.00 |    0.00 | 1372.00 |
|          3 | Pollería Don Lucho        |            0.00 |    0.00 |    0.00 |
+------------+---------------------------+-----------------+---------+---------+
```

Figura 57. *Ejecución del script SQL del Anexo E: 20 tablas y 2 vistas creadas sin errores*

Fuente: Elaboración propia. Ejecutado en MariaDB 10.11, compatible con MySQL 8; las advertencias 1280 indican solo que el motor ignora el nombre de las restricciones PRIMARY KEY.

5.2.4. Trazabilidad de punta a punta

Tabla 79*. Trazabilidad de los 10 RF principales: documento, código, base de datos y prueba*

RF	CUS (Anexo A)	Clase y método	Tabla de la base de datos	Test
RF11 Registrar compra del día	CUS-RF11	CompraServiceImpl.registrarCompra()	compra, detalle_compra, lote_saldo	RF11RegistrarCompraDelDiaTest
RF15 Registrar precios de venta del día	CUS-RF15	PrecioServiceImpl.guardarPrecios()	precio_dia	RF15RegistrarPreciosDeVentaDelDiaTest
RF17 Registrar venta al contado	CUS-RF17	VentaServiceImpl.registrar()	venta, detalle_venta, lote_saldo	RF17RegistrarVentaAlContadoTest
RF18 Registrar venta al crédito	CUS-RF18	VentaServiceImpl.registrar() + CreditoServiceImpl.saldo()	venta, cliente, abono_venta	RF18RegistrarVentaAlCreditoTest
RF20 Anular venta	CUS-RF20	VentaServiceImpl.anular()	venta, lote_saldo, auditoria	RF20AnularVentaTest
RF25 Sincronizar ventas pendientes	CUS-RF25	VentaServiceImpl.sincronizar()	venta (uuid_dispositivo)	RF25SincronizarVentasPendientesTest
RF31 Registrar abono de cliente	CUS-RF31	CreditoServiceImpl.registrarAbono() + Calculos.aplicarAbono()	abono, abono_venta, compromiso_pago	RF31RegistrarAbonoDeClienteTest
RF39 Realizar cuadre de caja	CUS-RF39	CajaServiceImpl.realizarCuadre()	cuadre, gasto	RF39RealizarCuadreDeCajaTest
RF40 Calcular ganancia del día	CUS-RF40	CajaServiceImpl.ganancia() + Calculos.ganancia()	detalle_venta, ajuste_saldo, gasto	RF40CalcularGananciaDelDiaTest
RF42 Registrar factura emitida	CUS-RF42	FacturaServiceImpl.registrar()	factura	RF42RegistrarFacturaEmitidaTest

5.3. Buenas prácticas

1. Convenciones de nombres. Paquetes por capa (web, servicio, servicio/impl, dao, dao/jdbc, modelo, comun, seguridad, config); interfaces con el nombre de la entidad (VentaDao, VentaService) e implementaciones con el sufijo de su tecnología (JdbcVentaDao, VentaServiceImpl); tablas y columnas en minúsculas con guion bajo, igual que en el modelo físico.

2. Manejo de errores. Las reglas lanzan NegocioException con un mensaje para el usuario; un solo manejador lo devuelve a la pantalla anterior. Los errores técnicos no muestran la traza (server.error.include-stacktrace=never).

```
    @ExceptionHandler(NegocioException.class)
    public String negocio(NegocioException e, HttpServletRequest req) {
        RequestContextUtils.getOutputFlashMap(req).put("error", e.getMessage());
        return "redirect:" + volver(req);
    }
```

3. Patrones aplicados. DAO (sección 1.3), inyección de dependencias por constructor, Template Method en JdbcBase (insertarConLlave, entero) y una transacción por venta al sincronizar (TransactionTemplate) para que una venta rechazada no anule a las demás.

```
                // Cada venta en su propia transacción: si una falla, las demás
se guardan.
                Long id = tx.execute(s -> registrarInterno(v, idActor, true,
true, Fechas.ahora()));
                out.add(new ResultadoSync(v.uuid(), "SINCRONIZADA", "Venta " +
String.format("%04d", id)));
```

4. Configuración sin credenciales. La contraseña de la base se lee de una variable de entorno y los archivos locales con claves están excluidos (sección 3.4).

5. Registro con Logback. Cada operación del negocio queda en la bitácora de la base y en un archivo diario que se rota y conserva 30 días.

```
    <appender name="ARCHIVO"
class="ch.qos.logback.core.rolling.RollingFileAppender">
        <file>${LOG_DIR}/svc-canomar.log</file>
        <rollingPolicy
class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">

<fileNamePattern>${LOG_DIR}/svc-canomar.%d{yyyy-MM-dd}.log</fileNamePattern>
            <maxHistory>30</maxHistory>
        </rollingPolicy>
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss} %-5level %logger{40} -
%msg%n</pattern>
            <charset>UTF-8</charset>
        </encoder>
    </appender>
```

6. Pull requests con integración continua. Cada rama feature se une a develop con un pull request, y GitHub Actions compila con Maven y ejecuta las 188 pruebas antes de aceptar el cambio (.github/workflows/maven.yml). Se hicieron cinco: #1 código y pruebas, #2 y #4 documentos, #3 y #5 entrega de develop a main.

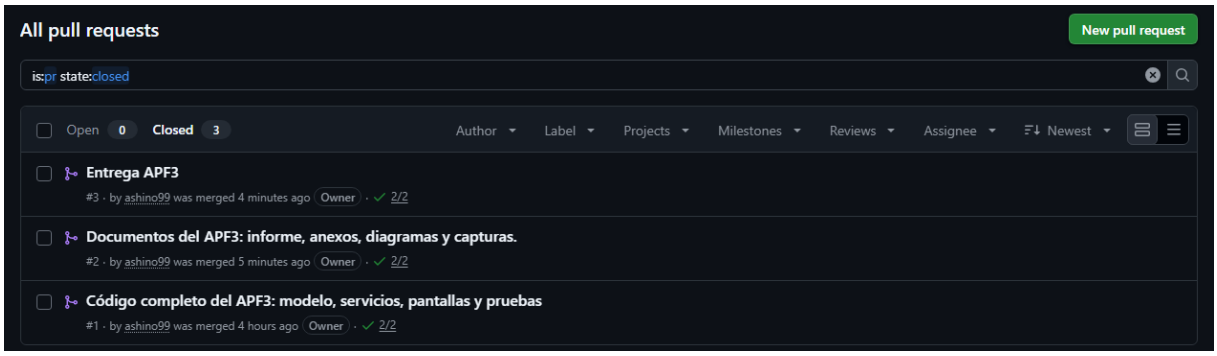


Figura 58*. Pull requests unidos con el check de GitHub Actions en verde (2/2)*

Fuente: Captura de GitHub.

5.4. Autoría

Tabla 80*. Responsable de cada módulo*

Módulo	Responsable	N° de commits	Commits en GitHub
Documentos APF1, APF2 y APF3	Doria Bautista, I.	3	docs(apf1-apf2), docs(apf3) ×2
Base del proyecto, modelo y DAO	Doria Bautista, I.	2	build, feat(modelo)
Servicios y cálculos del negocio	Doria Bautista, I.	1	feat(servicios)
Seguridad	Doria Bautista, I.	1	feat(seguridad)
Pantallas, PWA y modo sin conexión	Doria Bautista, I.	1	feat(pantallas)
Pruebas TDD e integración continua	Doria Bautista, I.	1	test(tdd)

Módulo	Responsable	N° de commits	Commits en GitHub
Total		9	+ 5 commits de merge (PR #1 a #5)

Todos los módulos fueron desarrollados por Doria Bautista, I.; en la sustentación explica o modifica en vivo cualquiera de ellos (guía docs/GUIA_DEL_CODIGO.md).

CAPÍTULO 6. SUSTENTACIÓN

La sustentación dura alrededor de 12 minutos y se hace con el sistema en ejecución. La presenta Doria Bautista, I., que desarrolló todos los módulos (sección 5.4).

Tabla 81*. Distribución de la sustentación*

Bloque	Tiempo	Responsable	Qué se muestra
1. Problema y arquitectura	2 min	Doria Bautista, I.	Problema de Canomar, arquitectura en capas y recorrido de RF18 (secciones 1.1 y 1.2).
2. Demostración del día de trabajo	4 min	Doria Bautista, I.	Compra, precios, venta al contado y al crédito (con la alerta de deuda vencida), venta sin señal y su sincronización, abono y cuadre.
3. Pruebas TDD	2 min	Doria Bautista, I.	Ejecutar en IntelliJ una clase RF## en verde y explicar ARRANGE/ACT/ASSERT y el DAO falso; mostrar un test en rojo.
4. Interfaces y modelo de datos	2 min	Doria Bautista, I.	Un hallazgo levantado (H01), la navegación por rol y una vista normalizada con su vista SQL.
5. GitHub y propuesta con hardware	2 min	Doria Bautista, I.	Ramas, pull requests con Actions en verde, release apf3 y la propuesta con hardware que se postula (Anexo F).

Modificación en vivo prevista (autoría): cambiar los días de alerta de mercadería antigua (CONFIGURACION.dias_alerta_saldo) o el texto de un mensaje de NegocioException, volver a ejecutar su test y ver el efecto en la pantalla.

Preguntas probables:

- ¿Por qué el controlador no tiene reglas? — porque así se prueban en el servicio sin pantalla ni base de datos.
- ¿Qué hace el DAO falso y por qué no se usa la base en los tests? — Mockito simula el DAO para probar solo la regla, rápido y sin datos de otra prueba.
- ¿Cómo se evita duplicar una venta guardada sin señal? — uuid_dispositivo UNIQUE y RF25-CP02.
- ¿Por qué el total de la venta no se guarda? — 3FN; se reconstruye con v_venta_total.
- ¿Qué pasa si se ingresa mal la contraseña cinco veces? — el usuario queda bloqueado 5 minutos (RF02-CP03).
- ¿Por qué se eligió esa propuesta con hardware para el semillero? — Anexo F, sección 1.9.

REFERENCIAS

Apache Software Foundation. (2024). *Apache POI: the Java API for Microsoft documents* (versión 5.3.0) [Software]. https://poi.apache.org/

Apache Software Foundation. (2024). *Apache Commons Lang* (versión 3.14) [Software]. https://commons.apache.org/proper/commons-lang/

Google. (2024). *Guava: Google core libraries for Java* (versión 33.3.1) [Software]. https://github.com/google/guava

JUnit Team. (2024). *JUnit 5 user guide*. https://junit.org/junit5/docs/current/user-guide/

Mockito. (2024). *Mockito framework* (versión 5) [Software]. https://site.mockito.org/

Nielsen, J. (1994). *10 usability heuristics for user interface design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

Oracle. (2024). *MySQL 8.0 reference manual*. https://dev.mysql.com/doc/refman/8.0/en/

OWASP Foundation. (2021). *OWASP Top 10: 2021*. https://owasp.org/Top10/

QOS.ch. (2024). *Logback manual*. https://logback.qos.ch/manual/

VMware Tanzu. (2024). *Spring Boot reference documentation* (versión 3.3). https://docs.spring.io/spring-boot/

VMware Tanzu. (2024). *Spring Security reference* (versión 6.3). https://docs.spring.io/spring-security/reference/

Thymeleaf. (2024). *Tutorial: Thymeleaf + Spring* (versión 3.1). https://www.thymeleaf.org/doc/tutorials/3.1/thymeleafspring.html

ANEXOS

Anexo	Contenido	Formato
A	Casos de uso del sistema (CUS) de los 50 RF, con 3 eventos alternativos cada uno	Excel: Anexo_A_CUS_50_RF.xlsx
B	Fichas de casos de prueba de los 50 RF y clasificación de aptitud	Excel: Anexo_B_Fichas_CP_50_RF.xlsx
C	Matriz de trazabilidad vista / RF de los 50 RF	Excel: Anexo_C_Trazabilidad_Vista_RF.xlsx
D	Comparativo completo de los campos de las 20 pantallas frente a la base de datos	Excel: Anexo_D_Comparativo_Pantallas_BD.xlsx
E	Script SQL del modelo final con datos de demostración	Archivo .sql: Anexo_E_svc_canomar_final.sql
F	Fichas completas de las Propuestas 2 y 3 con hardware (confidencial)	PDF aparte, solo por UTP+Class: APF3_Equipo7_Anexo_F_Confidencial.pdf
—	Diagramas fuente: arquitectura, navegación, DER final y bloques	docs/diagramas/*.drawio y *.dot